Differential Privacy with OpenDP

The OpenDP library from a first release to synthetic data

Manuel V. Loureiro

Version 2026-11-04

Table of contents

Preface 1
How this book is organized . . . 1

I. Getting started 2

1. A first release 4
1.1. Install the library . . . 4
1.2. One noisy number . . . 5
1.3. The five steps of an analysis . . . 5
1.4. A count and a mean . . . 7
1.5. What the context refuses . . . 8
1.6. Summary . . . 8
1.7. References . . . 9

2. The data and the context 10
2.1. The Labour Force Survey sample . . . 10
2.2. Loading the data lazily . . . 11
2.3. A context on the frame . . . 12
2.4. A query is not a frame . . . 13
2.5. A margin, in preview . . . 15
2.6. Summary . . . 15
2.7. References . . . 16

II. Tabular data 17

3. Essential statistics 19
3.1. The context for this chapter . . . 19
3.2. Counting responses . . . 20
3.3. Where bounds come from . . . 21
3.4. The sum of hours . . . 21
3.5. The mean . . . 23
3.6. The median and the quartiles . . . 24
3.7. A check against the truth . . . 25
3.8. Summary . . . 26
3.9. References . . . 27

4. Grouping 28
4.1. Why grouping keys are dangerous . . . 28
4.2. Stable keys . . . 29

4.3. Explicit keys . . . 30
4.4. Invariant keys . . . 31
4.5. Margins as public descriptors . . . 33
4.6. The noise per group and the ungrouped count . . . 33
4.7. Checking the release against the truth . . . 34
4.8. Summary . . . 34
4.9. References . . . 35

5. Identifier truncation and bounds **36**
5.1. A privacy unit with an identifier . . . 36
5.2. Truncating each person's rows . . . 38
5.3. Truncating the groups a person touches . . . 39
5.4. One row per person . . . 41
5.5. Bounds in the privacy unit . . . 42
5.6. The check against the truth . . . 43
5.7. Summary . . . 44
5.8. References . . . 45

6. Preparing microdata **46**
6.1. Prepare inside the query . . . 46
6.2. Columns, filters and groups . . . 47
6.3. The types OpenDP tracks . . . 48
6.4. Casting, filling and clipping . . . 49
6.5. Predicates and arithmetic . . . 51
6.6. Strings and dates . . . 52
6.7. A worked preparation: hours lost to absence . . . 53
6.8. What a refused expression looks like . . . 54
6.9. Summary . . . 55
6.10. References . . . 56

7. Assessing utility and choosing the noise **57**
7.1. What an accuracy statement means . . . 57
7.2. A target error bar on the survey . . . 58
7.3. Laplace under pure DP, Gaussian under zCDP . . . 59
7.4. One person in many groups: L1 against L2 . . . 60
7.5. Stating the guarantee in epsilon . . . 61
7.6. Signed and unsigned counts . . . 62
7.7. Which noise to choose . . . 63
7.8. The releases of one run . . . 63
7.9. Summary . . . 64
7.10. References . . . 65

III. The framework **66**

8. A framework to understand differential privacy **68**
8.1. Two kinds of object . . . 68
8.2. Functions . . . 69

8.3. Domains . . . 69
8.4. Metrics . . . 70
8.5. Measures . . . 71
8.6. Stability maps and privacy maps . . . 71
8.7. Chaining a transformation and a measurement . . . 72
8.8. The typical workflow in both APIs . . . 74
8.9. Summary . . . 76
8.10. References . . . 77

9. Transformations **78**
9.1. Constructors and chains . . . 78
9.2. Preprocessing . . . 79
9.3. The sum . . . 81
9.4. The mean and the variance . . . 84
9.5. Quantile scores . . . 85
9.6. Reading a chain . . . 86
9.7. Summary . . . 87
9.8. References . . . 88

10. Measurements **89**
10.1. Additive noise . . . 89
10.2. Canonical noise . . . 92
10.3. Thresholded noise . . . 92
10.4. The approximate Laplace projection . . . 94
10.5. Noisy max and selection . . . 95
10.6. Randomized response . . . 97
10.7. Summary . . . 99
10.8. References . . . 100

11. Combinators **102**
11.1. Chaining . . . 102
11.2. Non-adaptive composition . . . 104
11.3. Adaptive composition . . . 105
11.4. Odometers . . . 106
11.5. Privacy filters . . . 107
11.6. Chaining and nesting compositors . . . 108
11.7. Measure casting . . . 109
11.8. Amplification by subsampling . . . 110
11.9. Summary . . . 111
11.10. References . . . 112

12. Utilities and the Context API inside **114**
12.1. Types . . . 114
12.2. Parameter search . . . 116
12.3. Serialization . . . 117
12.4. The compositor behind a context . . . 118
12.5. The query behind a release . . . 119
12.6. Nested compositors and other measures . . . 120

12.7. Summary . . . 122
12.8. References . . . 123

IV. Extending the library **124**

13. Plugins **126**
13.1. Trust and the honest-but-curious flag . . . 126
13.2. A measurement plugin . . . 127
13.3. A transformation plugin . . . 128
13.4. Your own domains, metrics and measures . . . 130
13.5. Registering a constructor for the Context API . . . 131
13.6. A Theil-Sen regression . . . 132
13.7. Choosing grouping columns privately . . . 135
13.8. Summary . . . 137
13.9. References . . . 138

14. Statistical modelling **139**
14.1. A model or a table? . . . 139
14.2. Linear regression . . . 140
14.3. Principal component analysis . . . 142
14.4. The NumPy domain underneath . . . 143
14.5. What a reviewer should ask . . . 144
14.6. The releases of one run . . . 145
14.7. Summary . . . 146
14.8. References . . . 147

15. Limitations and troubleshooting **148**
15.1. What OpenDP does not promise . . . 148
15.2. The feature flags . . . 149
15.3. What lies outside the guarantee . . . 151
15.4. The error messages you will meet . . . 151
15.5. Reading an OpenDPException . . . 154
15.6. What a reviewer verifies . . . 155
15.7. Summary . . . 156
15.8. References . . . 157

V. Synthetic data **158**

16. Contingency tables **160**
16.1. From grouped counts to marginals . . . 160
16.2. The context under zCDP, with a delta . . . 161
16.3. Keys and cuts . . . 162
16.4. The Fixed algorithm and its workload . . . 162
16.5. The released table . . . 163
16.6. Adaptive estimation . . . 164
16.7. Checking the release against the truth . . . 165
16.8. Summary . . . 167

16.9. References 168

17. Synthetic data with AIM and MST **169**
17.1. When you do not know the questions 169
17.2. The AIM release 170
17.3. A spanning tree of pairs with MST 172
17.4. A first look at the one-way marginals 172
17.5. Nulls and the key set 173
17.6. Guaranteeing marginals with Sequential 174
17.7. Generating the rows 175
17.8. The privacy statement 176
17.9. The measured marginals against the real ones 177
17.10. Summary 178
17.11. References 179

18. Evaluating and tuning synthetic data **180**
18.1. What good means 180
18.2. The workload error 182
18.3. A query nobody measured 183
18.4. Train on synthetic, test on real 184
18.5. What the budget and the algorithm buy 185
18.6. How many rows, and how to draw them 186
18.7. What to publish 187
18.8. Summary 188
18.9. References 189

Appendices **190**

A. Glossary **190**

B. Index **191**

Preface

This book teaches the OpenDP library, from installation and a first noisy release to synthetic data generation. OpenDP implements differential privacy (DP), a guarantee that limits how much a release reveals about any one person. The documentation is organized by library components; this book is organized by learning sequence. Each chapter identifies the documentation pages it covers so readers can consult both.

All code has been run. Each chapter uses a script in the repository's `code/` directory; tests check every number quoted in the text. Short code blocks are executable snippets or excerpts from the chapter script. OpenDP uses an unseeded cryptographic generator for noise, so releases vary between runs. Chapters quote reproducible values, such as noise scales, error bars and thresholds, and label sample releases as single runs.

The book uses the French Labour Force Survey (LFS) sample from the OpenDP tutorials, with about 3.8 million quarterly responses from 1.7 million people. Chapters count, sum, group, bound and model the data, then generate a synthetic table.

Readers need Python and basic familiarity with NumPy or Polars, but no prior knowledge of differential privacy. Each chapter defines the results it uses and links to the full edition of *Privacy-Preserving Machine Learning* for derivations.

How this book is organized

1. **Getting started** installs the library, releases a first number and creates a context for later queries.
2. **Tabular data** releases statistics and grouped tables with Polars, bounds contributions, prepares data within the guarantee and interprets error bars.
3. **The framework** defines domains, metrics, measures, transformations and measurements; explains chaining and composition; and covers parameter-search utilities.
4. **Extending the library** implements transformations and measurements as plugins, fits regression and principal component models, and describes OpenDP's limitations.
5. **Synthetic data** releases contingency tables, generates synthetic microdata with AIM and MST, and evaluates the results.

The repository pins OpenDP 0.16.0 and Polars 1.36.1. Rerun a chapter from the repository root with `uv run python code/<part>/opendp_<topic>.py`.

Part I.

Getting started

This part introduces OpenDP contexts, which hold a dataset and privacy budget. It assumes no prior knowledge of differential privacy. A differentially private release adds noise calibrated to the maximum effect of one person's data. OpenDP calibrates the noise and tracks the budget.

The first chapter installs the library and releases a noisy number, first with a manually built mechanism and then with the Context API. The context uses the privacy unit and budget to calibrate noise. The chapter also presents the five steps of an OpenDP analysis.

The second chapter loads the French Labour Force Survey (LFS) sample used in the OpenDP tutorials and builds a context for it. Because a person may have rows from multiple quarters, the privacy unit must cover all of that person's rows. The chapter also explains how contexts divide a budget among queries and how to define a margin.

Table 1.: The chapters of this part and the questions they answer.

Chapter	Question it answers
A first release	How do I install OpenDP, and what happens when I release one noisy number?
The data and the context	How do I put a real dataset behind a context, and what do its unit, budget and margins mean?

By the end of this part, you can specify a release's privacy unit and budget and build a context for data with multiple rows per person.

1. A first release

Consider the scores of a hundred people. The goal is to publish their count and average while limiting what the release reveals about any individual's score. This chapter uses OpenDP to make these releases and introduces its central guarantee. A **differentially private** release is a randomized computation whose noise is calibrated to how much one person can change the answer. OpenDP calibrates the noise, rejects computations it cannot calibrate and tracks the privacy budget.

Following the documentation's quickstart, the chapter first builds a Laplace mechanism by hand and reads its guarantee. It then uses an OpenDP context to release a count and a mean.

The script opendp_first_release.py runs every step on OpenDP 0.16.0, using a seeded generator to produce a hundred integers. Noise scales, error bars and privacy maps are reproducible. Released values vary because OpenDP uses an unseeded cryptographic generator, so this page labels them as one run.

```
uv run python code/part01-getting-started/opendp_first_release.py
```

1.1. Install the library

OpenDP is a Python package with a Rust core. Install the Polars extra to read tables:

```
pip install 'opendp[polars]'
```

The repository pins `opendp[polars]==0.16.0` and Polars 1.36.1; all numbers in this book use those versions. Next, import the prelude, which gathers the library's modules under one name, and enable the `contrib` feature.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.__version__)  # 0.16.0
```

The feature flag marks components whose written proofs have not completed review. OpenDP runs a `contrib` component only when you opt in. In version 0.16.0, almost every component used in this book is still `contrib`, so every script enables it.

1.2. One noisy number

The simplest private computation adds noise to one number. OpenDP builds it from a **space**, a pair of a domain and a metric. A constructor then turns the space into a **measurement**, which is a randomized function with a privacy guarantee.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print(laplace.map(1.0))  # 1.0
print(laplace.map(2.0))  # 2.0
print(laplace.check(1.0, 0.5))  # False
```

The domain restricts inputs to finite floats. The metric measures their absolute difference. The constructor `then_laplace` adds Laplace noise with scale 1, and `>>` attaches it to the space.

The **privacy map** takes a bound on the distance between inputs, called the **sensitivity** of the released quantity. It returns the largest privacy loss ε for inputs within that distance. For Laplace noise, the map divides sensitivity by the noise scale. At scale 1, the cost equals the sensitivity. The `check` method tests whether the cost at a given sensitivity is at most a given ε. Doubling the scale halves the cost.

```
laplace scale 1: map(1.0)=1.0 map(2.0)=2.0
laplace scale 1: check(1.0, 1.0)=True check(1.0, 0.5)=False
laplace scale 2: map(1.0)=0.5
```

The value ε bounds the factor by which a change to one person's data can alter the probability of any output: at most e^{ε}. The full edition's chapter on the Laplace and Gaussian mechanisms derives this guarantee. It depends on both the noise and a bound on one person's effect. This bound is the argument to `map`; the rest of the chapter shows how OpenDP finds it.

1.3. The five steps of an analysis

The documentation's typical workflow has five steps: specify the privacy unit, budget and public information; provide the data through a context; then submit queries through it.

First identify the **privacy unit**, the entity the guarantee protects. The documentation calls it the unit of privacy. Here each person contributes one row, so neighbouring datasets differ by one row. OpenDP represents this relation as symmetric distance 1.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.unit_of(contributions=1))  # (SymmetricDistance(), 1)
print(dp.loss_of(epsilon=1.0))  # (MaxDivergence, 1.0)
```

```
print(dp.loss_of(epsilon=1.0, delta=1e-7))  # (Approximate(MaxDivergence), (1.0,
 ↪  1e-07))
print(dp.loss_of(rho=0.1))  # (ZeroConcentratedDivergence, 0.1)
```

Next set the **privacy budget**, which the documentation calls the privacy loss. It pairs a measure, which compares the distributions of two releases, with a bound on that comparison. Pure differential privacy uses max divergence bounded by ε, so `loss_of(epsilon=1.0)` returns the printed pair. The other two lines show budgets covered later: δ gives approximate differential privacy, and ρ gives zero-concentrated differential privacy, introduced in Assessing utility and choosing the noise. A common starting budget is $\varepsilon = 1$; the OpenDP deployments registry lists budgets from real releases.

Then collect **public information**: facts about the data that the guarantee need not protect. Here the score bounds are 0 and 100, and 50 is the value for a missing score. Take bounds from the domain or another public source, never from private data; deriving a bound from those data would leak information.

The fourth step is to **mediate access** through a **context**. It holds the data, privacy unit and budget, and defines how to split the budget. Every later computation on the data goes through it.

```
def make_context(data: list[float]) -> dp.Context:
    """The context: the data behind a privacy unit of one row, a budget of epsilon = 1,
     ↪  split evenly over 3 queries."""
    return dp.Context.compositor(
        data=data,
        privacy_unit=dp.unit_of(contributions=1),  # neighbours add or remove one
 ↪  person's row
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 3
    )
```

The context is a **compositor**: it adds the costs of multiple releases. Three queries that each use $\varepsilon/3$ cost ε in total, by the basic composition rule proved in the full edition's chapter on properties. The script prints the split.

```
privacy unit: (SymmetricDistance(), 1)
privacy loss: (MaxDivergence, 1.0)
bounds: (0.0, 100.0)  imputed value: 50
budget per query: [0.3333, 0.3333, 0.3333]
```

Finally, submit queries through the context. Delete other references to the data after creating it. The context's guarantee does not cover computations outside it.

1.4. A count and a mean

Build a query by chaining methods on `context.query()`. Each method is a framework constructor without an explicit space; the context supplies that space. The count query counts rows and adds Laplace noise.

```
count_query = context.query().count().laplace()
count_scale = count_query.param()  # the scale the budget allows: sensitivity 1 over
 ↪  epsilon / 3
count_accuracy = dp.discrete_laplacian_scale_to_accuracy(scale=count_scale, alpha=0.05)
dp_count = count_query.release()
```

`laplace()` does not specify a scale. The context chooses the smallest scale whose privacy map at the unit's distance fits the query's budget share; `param` reports that scale. For a count, one row has sensitivity 1. The share is $\varepsilon/3$, so the scale is 3. Because the count is an integer, OpenDP adds discrete Laplace noise. The discrete accuracy function converts the scale to an error bar: with probability 0.95, the released count is within about 9.4 of the true count.

```
count: scale=3.0000 95% accuracy=9.4457
```

Check both values before release. They depend on the budget and sensitivity, not on the data, and therefore incur no privacy cost.

The mean needs more preparation. Its sensitivity is bounded only after you address missing values, bound each value and fix the list size.

```
mean_query = (
    context.query()
    .impute_constant(IMPUTED)  # a missing value becomes the public midpoint
    .clamp(BOUNDS)  # every value is clipped into the public bounds
    .resize(size=dp_count, constant=IMPUTED)  # the mean needs a known size: the
 ↪  released count
    .mean()
    .laplace()
)
mean_scale = mean_query.param()
dp_mean = mean_query.release()
```

Imputation replaces missing values, for which the mean is undefined. Clamping bounds every value; without bounds, one person's contribution could change the mean without limit. Resizing fixes the row count, which is the mean's denominator and is needed to bound sensitivity. Here the size is the previously released count. Reusing that public value costs nothing by the post-processing property proved in the full edition. If the noisy count is too small, resizing pads with the imputed value; if it is too large, resizing drops rows. The resulting mean is therefore computed on a modified list. Its error bar does not include this change.

The mean's scale changes from run to run because it uses the released count. Since this toy dataset is public, the script also reports the scale and error bar at its true size of 100.

```
mean at size 100: scale=3.0000 95% accuracy=8.9872
```

An error bar of about 9 points is wide on a range of 100. The query receives only $\varepsilon/3$, and one person's row can move the mean by a fixed fraction of the full range. Essential statistics shows how releasing a sum and count on a larger dataset improves accuracy.

1.5. What the context refuses

The count and mean consume two of the three budget shares. The script reads the remaining shares from the context.

```
budget spent after two queries: [0.3333, 0.3333]
```

The third query uses the last share. A fourth query exceeds the budget, so the context raises an error and releases nothing.

```
try:
    context.query().count().laplace().release()
    refusal = "released"
except ValueError as err:
    refusal = str(err)
```

```
fourth query: Privacy allowance has been exhausted
```

The context enforces the declared budget. Without it, the framework does not prevent repeated calls to a measurement or track their cumulative cost.

Here are the releases of one run.

```
released (one run):
  count 105  (true 100, within 9.4 with probability 0.95)
  mean scale 2.8571  mean 53.32  (true 50.93)
  third query, a second count 108
```

The count and mean fall within their error bars in this run. The count's 95% error bar covers the true count in about 19 runs out of 20, not every run. The mean's scale is below 3 because the released count exceeds 100. You can compare with the true values only because this dataset is a public toy. For real data, test on public or historical data during development and release the private data once.

1.6. Summary

Table 1.1.: The calls of a first release, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.enable_features("contrib")`	Switches on the components whose proofs are not yet vetted	Put it at the top of every script; leave it out to see what is vetted
`space >> dp.m.then_laplace(scale)`	A Laplace measurement on a domain and a metric	The scale is the sensitivity over ε
`measurement.map(d_in)`	The largest ε for inputs at most `d_in` apart	Read it before releasing; it costs nothing
`measurement.check(d_in, d_out)`	Whether the map at `d_in` stays within `d_out`	What the context runs for you when it searches for a scale
`dp.unit_of(contributions=1)`	The privacy unit: neighbours add or remove one row	Choose it before anything else; one row per person is the default case
`dp.loss_of(epsilon=1.0)`	The privacy budget under pure differential privacy	$\varepsilon = 1$ is a common start; the registry lists real values
`dp.Context.compositor(...)`	Holds the data, the unit, the budget and its split	One context per release; delete your own reference to the data
`context.query().count().laplace()`	A query whose scale the context fills in from the budget	Read `param()` and the accuracy before `release()`
`impute_constant`, `clamp`, `resize`	Make the mean's sensitivity finite	Bounds and imputed values come from public information
A fourth query on a budget split in three	`ValueError: Privacy allowance has been exhausted`	The refusal is the point of the context

1.7. References

- OpenDP documentation: "Quickstart", "Typical Workflow", and "Context" in the API user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism and the definition it satisfies.
- Gaboardi, Hay and Vadhan, "A Programming Framework for OpenDP" (2020): the framework of domains, metrics, measures and maps behind every call.
- The full edition's chapters on the definition in the full edition, post-processing and composition in the full edition and the Laplace and Gaussian mechanisms in the full edition derive the results this chapter states.

2. The data and the context

The first chapter released a count and a mean from a hundred numbers that fit in one line of Python. Real data are different: a table of millions of rows, read from a file, in which one person appears many times. This chapter puts such a table behind a context. The one idea it adds is that the privacy unit must cover everything that one person can contribute. So a person who answers a survey every quarter needs a unit of many rows, and every noise scale grows with it.

The chapter follows the documentation's introduction to tabular data and its page on how OpenDP's Polars differs from plain Polars. You meet the dataset of the whole book and load it lazily. Then you build a context with a unit of 36 rows and a budget split five ways. You see how a query on a context differs from a query on a frame, and you add the margin that sums need. The first release on this data is the subject of Essential statistics.

The script opendp_data_context.py runs every step on OpenDP 0.16.0 with Polars 1.36.1. Every number on this page is the same on every run: the sizes of the sample, the budget splits, the noise scales and the margin. The page quotes no released value, because OpenDP's noise cannot be seeded and the releases start in the next chapter.

```
uv run python code/part01-getting-started/opendp_data_context.py
```

2.1. The Labour Force Survey sample

Eurostat and national statistical offices survey working hours across the European Union in the Labour Force Survey (LFS). Eurostat publishes a public microdata sample. OpenDP's tutorials use selected columns from the French sample for 2005–2013. Statistical disclosure control has already protected these data through recoded categories, suppression of rare values and added noise. Releases from this sample therefore pose no risk to respondents and can be compared with the true values. The book uses it for these reasons.

Table 2.1.: The columns of the LFS sample, with the coding of the survey's user guide.

Column	Meaning	Coding
`PIDENT`	Person identifier	An integer, generated by the OpenDP project
`YEAR`	Reference year	2005 to 2013
`QUARTER`	Reference quarter	1 to 4
`SEX`	Sex	1 male, 2 female
`AGE`	Age during the reference week	Single years, as a float with a few missing values

Column	Meaning	Coding
`HWUSUAL`	Hours per week usually worked in the main job	1 to 98 hours; 0 when the hours vary too much to give; 99 when not applicable; missing when there was no answer
`HWACTUAL`	Hours actually worked in the reference week	Coded like `HWUSUAL`
`ILOSTAT`	Labour status during the reference week	1 worked for pay; 2 had a job but was absent; 3 laid off; 4 conscript; 5 neither worked nor had a job; 9 under 15 years old

The survey publishes no person identifier, so OpenDP generated `PIDENT`. This synthetic column links rows as if they belonged to one person over time. It demonstrates how the library handles people with multiple rows and identifies no real person. The script reads sample sizes before creating a context; this is permitted because the sample is public.

```
rows: 3,811,932  people: 1,682,306
years: 2005 to 2013  quarters: 36
responses per person: max 15  distinct quarters per person: max 4
```

The sample has about 3.8 million responses and 1.7 million identifiers. A typical identifier appears two or three times. The busiest appears 15 times and in at most four distinct quarters, so some identifiers link several rows from one quarter. Later chapters do not use these counts. The privacy unit follows the survey's public design; these counts only confirm that the design's bound holds with room to spare.

2.2. Loading the data lazily

Polars reads files eagerly or lazily. An eager read loads the table into memory. A lazy scan records the file location and column names, then reads rows when a query runs. OpenDP targets the lazy API, so the script uses `scan_parquet` to create a `LazyFrame`. Polars reads only the file header to print its schema.

```
frame = pl.scan_parquet(practitioner_datasets.france_lfs())  # a LazyFrame: the file is
 ↪  read only when a query runs
print("schema: " + ", ".join(f"{name} {dtype}" for name, dtype in
 ↪  frame.collect_schema().items()))
```

```
schema: PIDENT Int64, YEAR Int64, QUARTER Int64, SEX Int64, AGE Float64, HWUSUAL
 ↪  Float64, HWACTUAL Float64, ILOSTAT Int64
```

The documentation reads the same sample from CSV with `dp.examples.get_france_lfs_path()` and `scan_csv`. The repository caches it as Parquet because CSV files do not store types. Polars infers types from values, so a column's type depends on its data: whole numbers produce an integer column, while one decimal value makes the column floating point.

```
import io

import polars as pl

integers = pl.scan_csv(io.BytesIO(b"hours\n35\n40\n"))
decimals = pl.scan_csv(io.BytesIO(b"hours\n35\n40.5\n"))
strings = pl.scan_csv(io.BytesIO(b"hours\n35\n40\n"), infer_schema=False)
print(integers.collect_schema())  # Schema({'hours': Int64})
print(decimals.collect_schema())  # Schema({'hours': Float64})
print(strings.collect_schema())  # Schema({'hours': String})
```

The privacy guarantee covers only queries submitted through the context. Data loading is outside it; the documentation assigns that step to a trusted curator. If one person's unusual value changes an inferred type or causes parsing to fail, an observer outside that trusted environment could learn about the person through a channel the guarantee does not cover. Use a format with a stored schema, such as Parquet, or load CSV columns as strings with `infer_schema=False` and cast them yourself.

Pass the `LazyFrame` to the context rather than a collected table. OpenDP can then inspect each query plan before reading rows, which remain out of memory until a release needs them.

2.3. A context on the frame

The first chapter's context took a list; this one takes a frame. The arguments are the same, but their values differ.

```
def make_context(frame: pl.LazyFrame, margins: Sequence[dp.polars.Margin] | None =
↪  None) -> dp.Context:
    """A context on the LazyFrame: a privacy unit of 36 rows, a budget of epsilon = 1
    ↪  split evenly over 5 queries."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or
↪  remove one person's 36 rows
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 5
        margins=margins,  # public facts about the frame; the count needs none
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (MaxDivergence, 1.0)
budget split evenly over 5: [0.2, 0.2, 0.2, 0.2, 0.2]
budget split by weights [2, 1, 1, 1]: [0.4, 0.2, 0.2, 0.2]
```

Set the privacy unit to `dp.unit_of(contributions=36)`, using the survey's design. A person can answer once per quarter for nine years, so they contribute at most 36 rows. The unit must cover all rows one person can contribute. Neighbouring datasets therefore differ by up to 36 rows from one person, represented in OpenDP as symmetric distance 36. Identifier truncation and bounds replaces this fixed bound with `dp.unit_of(contributions=1, identifier="PIDENT")`, which enforces the bound rather than assuming it.

This unit increases the privacy cost. One person's addition or removal can change a count by up to 36, so counts are 36 times as sensitive as with a one-row unit, and their noise scales increase by the same factor. This protects all of a person's responses. A one-row unit would protect each row separately; for someone with 15 rows, group privacy would weaken the guarantee by a factor of 15, as proved in the full edition's chapter on properties.

Set the budget to `dp.loss_of(epsilon=1.0)`, which specifies pure differential privacy with $\varepsilon = 1$. Changing all 36 rows from one person changes the probability of any output by at most a factor of e^{ε}. The full edition's chapter on the definition states this guarantee. The documentation calls the argument privacy loss; this book calls it the budget because the context spends it.

The context splits the budget because privacy losses add across releases. With `split_evenly_over=5`, each query gets $\varepsilon/5$, and the five shares of 0.2 sum to the full budget by the basic composition rule in the same chapter on properties. Without a query limit, the total loss would be unbounded. The documentation recommends equal shares for similar queries and weights when some queries matter more. The script builds a second context to show a weighted split.

```
weighted = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON),
    split_by_weights=WEIGHTS,  # the first query gets twice the share of each of the
 ↪  other three
)
```

The first query gets 0.4; each of the other three gets 0.2. These weights are budget shares, not values of ε. The documentation recommends using one context for all queries.

```
context = make_context(frame)
del frame  # from here on, every computation on the data goes through a context
```

The documentation recommends deleting other references to the frame once the context exists. Computations made directly from the frame are outside the guarantee. The sizes printed above are such computations, permitted because the sample is public.

2.4. A query is not a frame

With a `LazyFrame` as its data, `context.query()` returns a `LazyFrameQuery`. The documentation's comparison of Polars and OpenDP describes four differences; a small frame demonstrates the first two.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": [35, 40, 0, 99]}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=5,
)
query = context.query().select(dp.len(signed=True))
print(type(query).__name__)  # LazyFrameQuery
scale = query.summarize()["scale"].item()
print(f"scale {scale:.1f}")  # scale 5.0
```

First, specify the data through `context.query()` rather than accessing the frame directly. The documentation calls this object a mock of the data.

```
query type: LazyFrameQuery
query schema: Schema({'PIDENT': Int64, 'YEAR': Int64, 'QUARTER': Int64, 'SEX': Int64,
 ↪  'AGE': Float64, 'HWUSUAL': Float64, 'HWACTUAL': Float64, 'ILOSTAT': Int64})
collect before release: You must call `.release()` before executing a query.
plain Polars mean: MakeMeasurement("Expr is not recognized at this time:
 ↪  col("AGE").mean(). If you would like to see this supported, please file an issue.")
```

The mock knows the frame's schema, so OpenDP checks queries against the real columns and types. The documentation describes it as an empty `LazyFrame` with extra methods. In version 0.16.0, it is a separate class rather than a `LazyFrame` subclass, but its query interface is similar.

Second, OpenDP extends the Polars API with query methods and an expression namespace. `summarize` reports the noise for each aggregate; `release` spends the budget. The `.dp` namespace provides noisy aggregates such as `dp.len`, `.dp.sum` and `.dp.median`. In the snippet, count sensitivity 1 and budget share $\varepsilon/5$ give a scale of 5.

Third, query execution requires `release` before `collect`. A release spends the query's budget share and returns a `OnceFrame`, which yields its rows once. A second `collect` fails, so retain the collected frame rather than the released query.

```
second collect: FailedFunction("OnceFrame has been exhausted")
```

Fourth, OpenDP allows only query plans and expressions covered by its proofs. It rejects the plain Polars `mean` of the age column because it adds no noise. The private alternative is `.dp.mean`, used in the next chapter. OpenDP also changes some accepted expressions' arguments to prevent data-dependent errors.

2.5. A margin, in preview

A count needs only the privacy unit. The working-hours sum in the next chapter also needs a public bound on frame length. The script first requests the sum without this bound, then builds a context that includes it.

```
margin = dp.polars.Margin(
    # at most 150,000 responses per quarter (50,000 households of under three people,
    ↪ both public figures)
    # over the 36 quarters of the survey: a bound on the length of the frame that costs
    ↪ no privacy
    max_length=QUARTER_RESPONSES * CONTRIBUTIONS,
)
bounded = make_context(frame, margins=[margin])
```

```
margin: Margin(by=[], max_length=5400000, max_groups=None, invariant=None)
sum without a margin: MakeTransformation("must specify 'max_length' in a margin with
↪ by=[]")
sum with the margin: scale=14,405.2  (36 x 80 over epsilon / 5 = 14,400)
```

A **margin** declares public facts about a frame, grouped by selected columns. With `by=[]`, the group is the whole frame, and `max_length` bounds its row count. The sum needs this bound because OpenDP's guarantee accounts for overflow and floating-point rounding. Summing millions of floats loses some precision; proving that this loss does not leak data requires a bound on the number of values. Without a margin, OpenDP rejects the sum. With it, the clipped usual-hours sum has a scale slightly above 14,400, its sensitivity divided by the budget share. The small increase accounts for rounding.

The bound must be public. Eurostat reports about 50,000 French households per quarter, and public statistics put the average household below three people. Thus 150,000 responses per quarter is a generous bound, or 5.4 million rows over 36 quarters. Do not derive the bound from the observed row count; that would leak private data. A public bound lets OpenDP prove that arithmetic does not weaken the guarantee. Margins can also bound group counts and declare group keys or lengths invariant, as discussed in Grouping.

2.6. Summary

Table 2.2.: The calls and concepts of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
The LFS sample	Public microdata of the French Labour Force Survey, 3,811,932 responses from 1,682,306 synthetic identifiers	Already protected by statistical disclosure control, so releases can be checked against the truth

Call or concept	What it does	Rule of thumb
`pl.scan_parquet(path)`	A `LazyFrame` that reads rows only when a query runs	Give the context the lazy frame, not a collected table
`pl.scan_csv` and schema inference	Types inferred from the values, so they depend on the data	Read a format with a schema, or load strings with `infer_schema=False` and cast
`dp.unit_of(contributions=36)`	The privacy unit: neighbours add or remove up to 36 rows of one person	Take the bound from the design of the data, never from the data
`dp.loss_of(epsilon=1.0)`	The budget, under pure differential privacy	One person's data changes any output's probability by at most e^{ε}
`split_evenly_over`, `split_by_weights`	How the budget is divided; `context.d_mids` lists the shares	Weights are shares, not values of ε; the shares sum to the budget
`del frame`	Removes your own reference to the data	Anything computed outside the context is outside the guarantee
`context.query()`	A `LazyFrameQuery`: a mock that knows the schema and accepts only proven expressions	Build and summarize against the mock; `collect` works only after `release`
`summarize()`	Scale and distribution of each aggregate, before any release	It depends on the budget and the unit, not on the data
`release().collect()`	Spends the share and returns the released frame once	Keep the collected frame; a second `collect` fails
`dp.polars.Margin(max_length=...)`	A public bound on the length of the frame, which sums need	Build it from published figures, as 150,000 responses a quarter here

2.7. References

- OpenDP documentation: "Working with Tabular Data" (the dataset description, loading data, and mediating access with `Context`), "Polars vs. OpenDP" and "Context" in the API user guide, and the "Why OpenDP?" paragraph of the front page.
- Eurostat, *EU Labour Force Survey*, public microdata and the EU-LFS database user guide: the survey, its columns and their coding.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism behind the scales of this chapter.
- The full edition's chapters on the definition in the full edition and on post-processing, group privacy and composition in the full edition derive the results this chapter states.

Part II.

Tabular data

This part covers private statistics on tables. OpenDP analyses Polars lazy queries before execution, so filters, casts and aggregations are included in the privacy guarantee. Each statistic needs a bound on each person's contribution and a budget share. OpenDP uses both to set the noise scale.

The chapters follow the documentation's tabular-data tutorial. Essential statistics releases counts, sums, means, medians and quantiles, then interprets the scales and error bars from `summarize`. Grouping releases one row per group, using public or privately selected keys. Identifier truncation and bounds limits each person's rows and affected groups. Preparing microdata covers supported pre-aggregation expressions and data types. Assessing utility and choosing the noise converts scales to error bars and compares Laplace and Gaussian noise under zero-concentrated differential privacy.

Table 2.3.: The chapters of this part and the questions they answer.

Chapter	Question it answers
Essential statistics	How do I release a count, a sum, a mean or a quantile, and how large is its error?
Grouping	How do I release one statistic per group, when the groups are known and when they are not?
Identifier truncation and bounds	What do I do when one person owns many rows?
Preparing microdata	Which transformations may I apply before aggregating, and how do types and nulls behave?
Assessing utility and choosing the noise	How do I read an error bar, and when is Gaussian noise the better choice?

By the end of this part, you can release statistics on data with multiple rows per person, interpret cell-level error bars and state the table's privacy guarantee.

3. Essential statistics

This chapter answers four questions on the Labour Force Survey (LFS) sample: how many responses are there, how many hours do people work, and what are the mean and typical work week? OpenDP's Polars queries release each statistic with noise. The library sets the noise scale from a bound on one person's contribution and the query's budget share. Read the scale and error bar before release.

The chapter follows the documentation's tutorial on essential statistics and its aggregation guide. It counts rows, chooses bounds, sums a column, releases a mean in two ways and selects a median and quartiles from candidates. The practitioner edition's workflow on releasing statistics applies these methods to retail data.

The script opendp_essential_statistics.py runs every query on OpenDP 0.16.0. It reads the sample lazily through `practitioner_datasets.france_lfs()`, as the documentation reads the same file through `dp.examples.get_france_lfs_path()`. The scales, error bars, refusals and true values that it prints are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so this page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_essential_statistics.py
```

3.1. The context for this chapter

The context is the one that the previous chapter builds, with a privacy unit of 36 rows, one per quarter. Its budget is a privacy parameter ε of 1 under pure differential privacy (DP), split evenly over five queries. It also has that chapter's margin, a public bound on the number of rows. The new argument `invariant` is for the section on the mean.

```
def make_context(frame: pl.LazyFrame, invariant: Literal["keys", "lengths"] | None =
↪  None) -> dp.Context:
    """The chapter's context: 36 rows per person, epsilon = 1 split evenly over 5
    ↪  queries, and one public margin."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or
↪  remove one person's 36 rows
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 5
        margins=[dp.polars.Margin(max_length=MAX_LENGTH, invariant=invariant)],  # a
↪  public bound on the rows
    )
```

The five releases use one context. Building a query reserves its budget share, so scale comparisons use separate contexts with the same settings. In a real analysis, plan the shares before building queries in one context.

3.2. Counting responses

The first release counts the survey rows. The query selects `dp.len`, OpenDP's private version of Polars' `len`; the context calibrates the noise.

```
count_query = context.query().select(dp.len(signed=True))
count_summary = count_query.summarize(alpha=0.05)
dp_count = count_query.release().collect().item()
```

Call `summarize` before `release` to inspect the measurement without running it. The summary has one row per statistic.

```
shape: (1, 5)
┌────────┬────────────┬──────────────────┬─────────┬────────────┐
│ column ┆ aggregate  ┆ distribution     ┆ scale   ┆ accuracy   │
│ ---    ┆ ---        ┆ ---              ┆ ---     ┆ ---        │
│ str    ┆ str        ┆ str              ┆ f64     ┆ f64        │
╞════════╪════════════╪══════════════════╪═════════╪════════════╡
│ len    ┆ Frame Length ┆ Integer Laplace ┆ 180.0   ┆ 539.731115 │
└────────┴────────────┴──────────────────┴─────────┴────────────┘
```

The aggregate is frame length, and its distribution is integer Laplace because the count is an integer. One person's addition or removal changes up to 36 rows, so count **sensitivity** is 36. The Laplace scale is sensitivity divided by the query's ε share: 36/0.2. The full edition's chapter on the Laplace and Gaussian mechanisms derives this calibration. The accuracy is the error bar for the requested `alpha`; at 95% confidence, the released count is within 539.73 of the true count.

The error bar depends on `alpha`, not the data, so reading it has no privacy cost. Greater confidence requires a wider bar: at `alpha=0.01`, this scale gives 829.43.

```
count: scale=180.0 95% accuracy=539.73 99% accuracy=829.43
sensitivity 36 over epsilon 0.2: scale 180.0
```

The bar covers noise only. It excludes sampling error between this sample and France's population, as well as Eurostat's recoding and suppression. It also counts responses, not people. `signed=True` allows a negative released count when noise exceeds a small true count. Without this argument, OpenDP clamps negative counts to zero, which biases small counts upward; the library warns that this default will change.

```
without signed=True: FutureWarning: The default value is currently signed=False, and
 ↪  counts are clamped at zero, consistent with past releases, but the default will be
 ↪  changing to signed=True.
```

Columns support additional counts described in the documentation's aggregation guide. `.dp.len` counts rows including nulls, `.dp.count` counts non-null rows, and `.dp.null_count` counts nulls. These give the number of hours responses and the non-response count. `.dp.n_unique` counts distinct values, treating null as one value. Each count has sensitivity at most 36, so its scale matches frame length when queried alone. The excerpt requests all five counts together.

```
five_counts = variants.query().select(
    dp.len(signed=True),  # rows of the frame
    hwusual.dp.len(signed=True).alias("expr_len"),  # rows of the column, nulls
 ↪  included
    hwusual.dp.count(signed=True).alias("count"),  # rows of the column that are not
 ↪  null
    hwusual.dp.null_count(signed=True).alias("null_count"),  # rows of the column that
 ↪  are null
    hwusual.dp.n_unique(signed=True).alias("n_unique"),  # distinct values, null among
 ↪  them
)
five_summary = five_counts.summarize(alpha=0.05)
```

```
HWUSUAL len alone: scale=180.0
HWUSUAL count alone: scale=180.0
HWUSUAL null_count alone: scale=180.0
HWUSUAL n_unique alone: scale=180.0
five counts in one query: scale=900.0 each, 95% accuracy=2696.7
```

Five statistics in one query split its budget equally, giving each a fifth of the share and five times the scale. The queries on this page use equal splits unless their scales are set explicitly. Grouping releases null and non-null counts with one count's noise by grouping on null status.

3.3. Where bounds come from

Statistics other than counts need value bounds or candidates. Sums and means need lower and upper bounds; medians need candidate values. The bounds determine sensitivity: one person's rows can change a sum by at most their count times the interval width. The full edition's chapter on sensitivity derives sensitivity under clipping. Wide bounds add unnecessary noise. Narrow bounds clip valid values and bias the result; the error bar does not include that bias.

Choose bounds from domain knowledge, a comparable public dataset or a budgeted coarse-quantile query. Never estimate them from private data: a bound such as the observed maximum depends on individual values and leaks information outside the guarantee. This chapter uses domain knowledge: 0 to 80 hours for a work week and whole-hour candidates from 20 to 59.

3.4. The sum of hours

`HWUSUAL` records usual weekly hours; 99 means "not applicable." The query filters out those rows, and OpenDP accounts for the filter. `.dp.sum` then clips values to the bounds, sums them and adds noise.

```
hours: Any = pl.col("HWUSUAL").cast(int)  # Any: the .dp namespace is added at run time
sum_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)  # 99 means "not applicable"
    .select(hours.fill_null(35).dp.sum(bounds=(0, 80)))  # fill, clip, sum, add noise
)
sum_summary = sum_query.summarize(alpha=0.05)
dp_sum = sum_query.release().collect().item()
```

```
sum HWUSUAL: Sum, Integer Laplace, scale=14,400.00 95% accuracy=43,139.0
sensitivity 36 x 80 = 2,880 over epsilon 0.2: scale 14,400
```

One person contributes at most 36 rows of at most 80 hours each, so sensitivity is their product. The 43,139-hour error bar is small relative to the total of about 56 million. Noise does not grow with the row count, so relative error decreases as the table grows.

The sum needs the margin, and it says so when the margin is missing.

```
without a margin: must specify 'max_length' in a margin with by=[]
```

The previous chapter explains that a sum needs a row-count bound to account for overflow and rounding. The message's `by=[]` identifies the whole-table margin; Grouping adds margins for groups. The refusal is data-independent, as the three-row snippet demonstrates.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": [35.0, 39.0, 40.0]}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
)
try:
    context.query().select(pl.col("hours").dp.sum(bounds=(0.0, 80.0))).summarize()
except dp.OpenDPException as err:
    print(err.message)  # must specify 'max_length' in a margin with by=[]
```

The second adjustment casts the float column to integers. A float sum accounts for rounding across up to `max_length` additions. The cost is small at this chapter's bound, but a bound of one billion rows raises the float scale to 192,035.68. Integer casting avoids that rounding cost and its dependence on `max_length`; use it when the values are whole numbers.

```
float sum, max_length 5,400,000: HWUSUAL: Sum, Float Laplace, scale=14,405.18 95%
 ↪  accuracy=43,154.1
float sum, max_length 1,000,000,000: HWUSUAL: Sum, Float Laplace, scale=192,035.68 95%
 ↪  accuracy=575,287.5
int sum, max_length 1,000,000,000: HWUSUAL: Sum, Integer Laplace, scale=14,400.00 95%
 ↪  accuracy=43,139.0
```

The third adjustment is `fill_null`. A sum cannot process a null row, so OpenDP rejects a nullable column when you write the clipping and sum manually. `.dp.sum` instead fills nulls with the bounds' midpoint, 40 here. `fill_null(35)` uses a legal French work week, a public fact. This choice does not affect this dataset: Polars treats a null comparison with `!= 99` as null, so the filter drops every null row. The script also sums the whole column to confirm the midpoint rule.

```
sum without fill_null: scale=14,400.0 (accepted; a null becomes the midpoint 40)
clip, sum and laplace by hand, without fill_null: input data (HWUSUAL) might contain
 ↪  nulls. Preprocess your data with `.fill_null`.
```

The error bar applies to the *clipped* sum. Values above 80 become 80, and the error bar does not capture this change. The final check shows that no applicable value exceeds the bound in this dataset.

3.5. The mean

A mean is a sum divided by a count. Under the add/remove relation, both are private, so the documentation recommends releasing them in one query and dividing afterwards. This division is post-processing and incurs no additional cost, as proved in the full edition's chapter on properties.

```
joint_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)
    .select(hours.dp.sum(bounds=(0, 80)), dp.len(signed=True))  # a null becomes the
 ↪  midpoint, 40
)
joint_summary = joint_query.summarize(alpha=0.05)
joint = joint_query.release().collect().with_columns(mean=pl.col("HWUSUAL") /
 ↪  pl.col("len"))
```

```
joint HWUSUAL: Sum, Integer Laplace, scale=28,800.00 95% accuracy=86,277.6
joint len: Frame Length, Integer Laplace, scale=360.00 95% accuracy=1,079.0
```

The query splits its share between the two statistics, doubling each scale compared with a standalone query. Together they cost one share, and the released count can divide sums from other columns. In this context, `.dp.mean` releases the same noisy sum and length.

```
dp.mean on the add/remove context: HWUSUAL: Sum, Integer Laplace, scale=28,800.00 95%
 ↪  accuracy=86,277.6
dp.mean on the add/remove context: HWUSUAL: Length, Integer Laplace, scale=360.00 95%
 ↪  accuracy=1,079.0
```

Alternatively, declare the row count public, which the documentation calls bounded DP. Neighbours then have equal row counts and differ in one person's values (the replace-one relation). Setting `invariant="lengths"` in a margin makes the length public, so `.dp.mean` adds noise only to the sum.

```
applicable_frame = frame.filter(pl.col("HWUSUAL") != 99.0)  # filtered outside the
↪  context: see the text
bounded = make_context(applicable_frame, invariant="lengths")  # the number of rows is
↪  public
mean_query = bounded.query().select(hours.dp.mean(bounds=(0, 80)))
mean_summary = mean_query.summarize(alpha=0.05)
dp_mean = mean_query.release().collect().item()
```

```
bounded mean HWUSUAL: Sum, Integer Laplace, scale=7,200.00 95% accuracy=21,569.8
bounded mean HWUSUAL: Length, Integer Laplace, scale=0.00 95% accuracy=none
```

The sum's scale is half the add/remove scale for a full budget share. With fixed length, changing one row is one removal and one addition, so distance 36 allows at most 18 changed rows. The length has scale zero because it is released exactly. This weaker guarantee hides a person's answers but not their presence in the table.

The excerpt also filters out code 99 outside the context. Filtering inside the context makes the row count unknown, so the invariant no longer holds and `.dp.mean` uses a noisy sum and length. Filter outside the context only when the number of applicable responses is public. Otherwise, use the joint query.

```
dp.mean after a filter inside the bounded context: HWUSUAL: Sum, Integer Laplace,
↪  scale=28,800.00 95% accuracy=86,277.6
dp.mean after a filter inside the bounded context: HWUSUAL: Length, Integer Laplace,
↪  scale=360.00 95% accuracy=1,079.0
```

3.6. The median and the quartiles

Direct noise on a median has no finite sensitivity bound: if few values lie near the middle, one person's rows can shift it arbitrarily. OpenDP instead chooses among supplied **candidates**. It scores each candidate by the imbalance between counts below and above it, adds exponential noise to the scores and releases the candidate with the smallest noisy score. The full edition's chapter on selection derives this exponential mechanism. Here the candidates are whole hours from 20 to 59.

```
candidates = list(range(20, 60))  # whole hours from 20 to 59
median_query = context.query().filter(pl.col("HWUSUAL") !=
↪  99.0).select(hours.dp.median(candidates))
median_summary = median_query.summarize(alpha=0.05)
dp_median = median_query.release().collect().item()
```

```
median HWUSUAL: 0.5-Quantile, ExponentialMin, scale=360.00 95% accuracy=none
score sensitivity 36: scale 2 x 36 / 0.2 = 360
median without a margin: Must know max_length
```

The aggregate is labelled 0.5-quantile because `.dp.median` calls `.dp.quantile` with `alpha=0.5`. Its scale is twice the score sensitivity divided by the query's budget share, as required by the exponential mechanism. Accuracy is empty because noise perturbs scores, not the answer, so there is no error bar in hours. A candidate with a much worse score is unlikely to win; the final check quantifies this gap. The quantile also needs a margin because its integer scores depend on the row count.

For other quantiles, `alpha` is the fraction of values below the target. The excerpt requests all three quartiles in one query.

```
quartile_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)
    .select(hours.dp.quantile(a, candidates).alias(f"{a}-Quantile") for a in [0.25,
 ↪  0.5, 0.75])
)
quartile_summary = quartile_query.summarize()
dp_quartiles = quartile_query.release().collect().row(0)
```

```
quartiles 0.25-Quantile: 0.25-Quantile, ExponentialMin, scale=3,240.00 95%
 ↪  accuracy=none
quartiles 0.5-Quantile: 0.5-Quantile, ExponentialMin, scale=1,080.00 95% accuracy=none
quartiles 0.75-Quantile: 0.75-Quantile, ExponentialMin, scale=3,240.00 95%
 ↪  accuracy=none
0.25-quantile alone: scale=1,080.0
```

Three quantiles split the query's share equally, tripling the median's scale. The quartile scales are three times larger again, without increasing the median's budget share. A quantile score is an integer multiplied by the denominator of `alpha`: four for a quartile and two for the median. One person's rows therefore move a quartile score three times farther. The final line shows that the first quartile's standalone scale is one-third its scale in the trio.

3.7. A check against the truth

Because the sample is public, the script computes true values and compares them with one run's releases. For private data, perform this check during development on public or historical data, then release the private data once.

```
true responses: 3,811,932
true responses with HWUSUAL not 99: 1,496,192 (nulls among them: 0; in the whole
 ↪  column: 20,819)
true sum of hours after the filter, clipped to [0, 80]: 56,315,594 (values above 80: 0)
true sum of the whole column, clipped: 240,742,034 with nulls as 40, 239,909,274 with
 ↪  nulls dropped
true mean: 37.64  (bounded mean 95% accuracy: 0.0144)
true quartiles: 35, 37, 40
score gap from the true quartile to the nearest other candidate: 0.25: 219,506  0.5:
 ↪  74,383  0.75: 116,811
```

The second and third lines show that the filter leaves no nulls and no applicable value exceeds 80. The bounded mean's error bar is the sum's error bar divided by the exact length. For quantiles, the final line replaces an error bar: the nearest alternative candidate scores at least 74,383 above the true quartile, while the maximum noise scale is 3,240. A wrong candidate is therefore unlikely, and the test requires the exact quartiles.

```
released (one run):
  responses 3,811,961  (true 3,811,932, within 539.7 with probability 0.95)
  sum of hours 56,308,788  (true 56,315,594, within 43,139.0)
  joint: sum 56,335,491  count 1,495,681  mean 37.67  (true 37.64)
  bounded mean 37.6411  (true 37.6393, within 0.0144)
  median 37  quartiles 35, 37, 40  (true 37; 35, 37, 40)
  sum of the whole column 240,726,542  (true with nulls as 40: 240,742,034)
```

In this run, the count and sum fall within their 95% error bars; each does so in about 19 runs out of 20. A count error of a few hundred is small relative to 3.8 million responses. The joint mean differs in the second decimal because it divides two noisy values. The bounded mean falls within an error bar of about a hundredth of an hour, the benefit of a public length. The median and quartiles match exactly. The whole-column sum is close to the total that imputes 40 hours for each null, not the total that drops nulls.

3.8. Summary

Table 3.1.: The calls of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.unit_of(contributions=36)`	The privacy unit: neighbours differ by one person's rows, up to 36	Count the most rows one person can own, not the typical number
`dp.polars.Margin(max_length=A public bound on the number of rows		Sums, means and quantiles refuse to run without it; take it from public facts
`dp.len(signed=True)`	The number of rows, with Laplace noise of scale sensitivity over ε	Without `signed=True` a negative count is clamped to zero
`.dp.len`, `.dp.count`, `.dp.null_count`, `.dp.n_unique`	A column's rows, non-nulls, nulls and distinct values	Each is as sensitive as the frame length; several in one query share its ε
`query.summarize(alpha=0.05)`	The column, aggregate, distribution, scale and error bar of each statistic	Read it before `release`; a smaller `alpha` widens the bar
`.dp.sum(bounds=(0, 80))`	Fills nulls with the midpoint, clips, sums, adds noise	Cast to an integer; the error bar excludes the bias of clipping

Call or concept	What it does	Rule of thumb
Sum and `dp.len` in one query	A mean under add/remove, by dividing afterwards	Each statistic gets twice the scale; the division is free post-processing
`invariant="lengths"` and `.dp.mean`	A bounded-DP mean: noise on the sum, the exact length	Weaker guarantee; filter outside the context only if the row count is truly public
`.dp.median(candidates)`, `.dp.quantile(alpha, candidates)`	The candidate nearest the quantile, by the exponential mechanism	No error bar; candidates from domain knowledge; several quantiles split the share evenly

3.9. References

- OpenDP documentation: "Essential Statistics" in the tutorial on tabular data, and "Aggregation" in the Polars user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism behind the counts and sums.
- McSherry and Talwar, "Mechanism Design via Differential Privacy" (2007): the exponential mechanism behind the median and the quantiles.
- The full edition's chapters on the Laplace and Gaussian mechanisms in the full edition, post-processing and composition in the full edition, selection in the full edition and sensitivity in the full edition derive the results this chapter states.

4. Grouping

Many private releases are grouped tables, such as counts by age and employment status or mean hours by sex. Their keys can reveal information: a group containing only one person's data identifies that person by its presence. This chapter compares three ways OpenDP releases grouped tables and the privacy cost of each.

The first method selects keys and retains groups whose noisy counts exceed a threshold. The second uses a public key list. The third declares keys or group sizes public through margins. The choice determines what information the release protects.

The script opendp_grouping.py runs every query of this chapter on OpenDP 0.16.0 against the full survey sample. The scales, error bars, thresholds, error messages and true group counts that it prints are the same on every run. The released counts and means are not, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_grouping.py
```

4.1. Why grouping keys are dangerous

The data are the France Labour Force Survey (LFS) sample described in The data and the context. The privacy unit matches that chapter's context: up to 36 rows per person, one per quarter.

Grouping responses by `AGE` and `ILOSTAT` (labour status) produces 16 groups. The smallest group, with no recorded age, has 255 rows. Noise with an error bar of a few hundred can hide its count but not its existence. In a finer grouping by age, sex and town, one person could fill a group alone. Noise would hide the count, but the row would reveal that someone with those keys answered the survey. Group keys therefore need protection as well as counts.

OpenDP enforces this. On a context with a pure budget, `summarize` refuses a count by `AGE` and `ILOSTAT` before it computes anything.

```
without delta: The key-set of {col("AGE"), col("ILOSTAT")} is private and cannot be
 ↪  released without a filter or join. Please pass a filtering threshold into
 ↪  make_private_lazyframe or conduct a join against a public key-set.
```

The message gives two options. A filter retains groups whose noisy counts pass a threshold and requires a δ budget. A join uses a public key list. The snippet summarizes both methods on a toy frame.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
data = pl.LazyFrame({"region": ["A", "A", "B"], "hours": [35, 40, 20]})
unit = dp.unit_of(contributions=1)
pure = dp.Context.compositor(data=data, privacy_unit=unit,
 ↪  privacy_loss=dp.loss_of(epsilon=1.0), split_evenly_over=1)
counts = pure.query().group_by("region").agg(dp.len(signed=True))
try:
    counts.summarize()
except dp.OpenDPException as err:
    print("cannot be released without a filter or join" in str(err))  # True
loss = dp.loss_of(epsilon=1.0, delta=1e-6)
approx = dp.Context.compositor(data=data, privacy_unit=unit, privacy_loss=loss,
 ↪  split_evenly_over=1)
summary =
 ↪  approx.query().group_by("region").agg(dp.len(signed=True)).summarize(alpha=0.05)
print(summary.columns)  # ['column', 'aggregate', 'distribution', 'scale', 'accuracy',
 ↪  'threshold']
print(int(summary["threshold"].item()))  # 14
keyed = counts.with_keys(pl.LazyFrame({"region": ["A", "B", "C"]}))
print(keyed.summarize().columns)  # ['column', 'aggregate', 'distribution', 'scale']
```

4.2. Stable keys

The first method lets OpenDP select the groups to release. The documentation calls retained groups **stable keys**: a group is stable if it remains after removing any one person. OpenDP releases a group only when its noisy count exceeds a threshold. It sets the threshold so a group filled by one person appears with probability at most δ. This chapter uses the documentation's budget, `dp.loss_of(epsilon=0.25, delta=1e-7)`. This method, **partition selection**, satisfies approximate differential privacy; the full edition's chapter on sensitivity derives the guarantee.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # a person answers in at
 ↪  most 36 quarters
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),  # the delta pays for the
 ↪  threshold
    split_evenly_over=1,
)
stable = context.query().group_by("AGE", "ILOSTAT").agg(dp.len(signed=True))
stable_summary = stable.summarize(alpha=0.05)
stable_hat = stable.release().collect()
```

```
stable keys: privacy loss (Approximate(MaxDivergence), (0.25, 1e-07))
stable keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
 ↪  threshold=2773
```

The summary gives sensitivity 36 because one person's rows can all belong to one group. Dividing by $\varepsilon = 0.25$ gives scale 144. With probability 0.95, each released count is within 431.9 of its true value. The threshold is 2773; groups with noisy counts at or below it are dropped.

The threshold accounts for the worst case across one person's rows. All 36 rows could enter one empty group, giving it a true count of 36. Discrete Laplace noise with scale 144 raises that count above 2773 with probability 2.77×10^{-9}. Alternatively, the rows could enter 36 separate one-row groups. OpenDP adds the 36 probabilities, giving 9.98×10^{-8}, just below $\delta = 10^{-7}$. A threshold of 2772 exceeds δ, so 2773 is the smallest threshold that meets the guarantee.

```
one person's 36 rows clear 2773 with probability 2.77e-09; over 36 groups 9.98e-08 <=
 ↪  delta 1e-07
at threshold 2772: over 36 groups 1.01e-07 > delta
```

The threshold decreases when δ is larger or the noise scale is smaller. A larger privacy unit raises both scale and threshold because one person can contribute more rows to a group.

```
threshold with delta=1e-05: scale=144.00 threshold=2110
threshold with epsilon=1: scale=36.00 threshold=720
```

In this public dataset, 14 of 16 groups have true counts above 2773, so the release contains 14 rows. The number of released groups is itself private information. Do not inspect the data to identify missing groups; that inspection is outside the guarantee.

4.3. Explicit keys

If the keys are known in advance, provide them to the library and use a pure privacy budget. **Explicit keys** are a supplied key list; the release includes every listed key, whether or not it occurs in the data. Construct the list without inspecting private data, because keys selected from the data are a release. Here the list contains every survey year crossed with every status code, including codes absent from the sample.

```
explicit = (
    context.query()
    .group_by("YEAR", "ILOSTAT")
    .agg(dp.len(signed=True))
    .with_keys(public_keys())  # the years crossed with the codebook's codes, fixed
 ↪  without the data
)
explicit_summary = explicit.summarize(alpha=0.05)
explicit_hat = explicit.release().collect()
```

```
explicit keys: 54 keys, 36 with rows in the data, 18 without
explicit keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

The scale and accuracy are unchanged, and there is no threshold because no group is dropped. The 18 empty keys receive signed noise around zero, so readers cannot distinguish empty groups from small ones. A released key set is public, so the documentation reuses the keys from a stable release in a later query.

`with_keys` performs a right join from the aggregation to the key list. Writing the join directly gives the same summary.

```
right join len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

Only left or right joins against a key list can release a key set. An inner join retains only keys present in the data and therefore reveals which keys occur; OpenDP rejects it.

```
inner join: only left or right joins can be used to release key-sets
```

4.4. Invariant keys

The third method declares grouping keys public through a **margin**, introduced in The data and the context. The margin `dp.polars.Margin(by=["YEAR", "QUARTER"], invariant="keys")` declares the set of quarters public. OpenDP then releases every group key in the clear, without a threshold or δ.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["YEAR", "QUARTER"], invariant="keys")],  # the
 ↪  quarters are public
)
quarterly = context.query().group_by("YEAR", "QUARTER").agg(dp.len(signed=True))
quarterly_summary = quarterly.summarize(alpha=0.05)
quarterly_hat = quarterly.release().collect()
```

```
invariant keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true quarterly counts: 36 quarters, smallest 82,504 (2006 Q3), largest 131,656 (2011
 ↪  Q4)
error bar as a share of the smallest quarter: 0.52%, of the largest: 0.33%
```

An **invariant** is a public fact excluded from the guarantee, which makes the guarantee weaker. Public keys could reveal a quarter containing a single respondent. Here the quarter set comes from the survey design and was known before responses were collected. In other settings, use invariants cautiously, as the documentation advises. The error bar is unchanged.

4.4.1. Invariant lengths

A margin can also declare group sizes public with `invariant="lengths"`. Public sizes imply public keys, so this reveals more. The documentation's example computes mean `HWUSUAL` (usual hours) by sex. Code 99 means not applicable and would bias the mean, so remove those rows first. A filter inside the query changes group sizes and invalidates the margin, causing OpenDP to treat the keys as private again.

```
filter inside the query: The key-set of {col("SEX")} is private and cannot be released
 ↪  without a filter or join. Please pass a filtering threshold into
 ↪  make_private_lazyframe or conduct a join against a public key-set.
```

Filter the `LazyFrame` before building the context; the margin then describes the filtered data, which has 1,496,192 rows.

```
context = dp.Context.compositor(
    data=working,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=MEAN_EPSILON),
    split_evenly_over=1,
    margins=[
        dp.polars.Margin(
            by=["SEX"],
            invariant="lengths",  # the number of responses per sex is public, keys
 ↪  included
            max_length=MAX_LENGTH,  # needed by any sum: a public bound on the rows in
 ↪  a group
            max_groups=3,  # male, female or null
        )
    ],
)
mean_hours = context.query().group_by("SEX").agg(hours().dp.mean(HOURS_BOUNDS))
mean_summary = mean_hours.summarize(alpha=0.05)
mean_hat = mean_hours.release().collect()
```

```
invariant lengths HWUSUAL: Sum, Integer Laplace, scale=1,764.00 95% accuracy=5,285.0
invariant lengths HWUSUAL: Length, Integer Laplace, scale=0.00 95% accuracy=none
true lengths: SEX=1 779,714  SEX=2 716,478
true mean hours: SEX=1 40.778  SEX=2 34.224
mean 95% accuracy: SEX=1 0.0068  SEX=2 0.0074
```

OpenDP releases the mean as a sum divided by a length and reports each separately. The public length has scale 0. At $\varepsilon = 1$, the sum's scale is 1764, its sensitivity. Because group sizes are fixed, one person's rows can only replace other rows: at most 18 swaps, each changing the sum by up to 98 hours. The documentation's prose gives scale 7056, but its table and OpenDP 0.16.0 give 1764.

Dividing the sum's 5285-hour error bar by a public group length of several hundred thousand gives a mean error below 0.01 hours for either sex. In version 0.16.0, released means are floats; the documentation rounds them to whole hours.

4.5. Margins as public descriptors

A margin describes one grouping through its `by` columns, and a context accepts multiple margins. One context can therefore protect most groupings fully and relax selected ones. In addition to `invariant`, a margin has two bounds. `max_groups` publicly bounds the number of groups and lowers the threshold for stable keys: one person's rows can affect at most that many groups, so OpenDP divides δ among fewer groups. With a bound of 16, the threshold falls from 2773 to 2656.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["AGE", "ILOSTAT"], max_groups=GROUPS_IN_TRUTH)],  #
 ↪  at most 16 groups
)
bounded_summary = context.query().group_by("AGE",
 ↪  "ILOSTAT").agg(dp.len(signed=True)).summarize()
```

```
max_groups=16: threshold=2656
```

`max_length` publicly bounds the rows in each group. A sum needs this bound to account for overflow; an unbounded sum could overflow. Counts need neither bound. OpenDP rejects a sum of hours by sex if the margin lacks `max_length`.

```
sum per SEX without max_length: must specify 'max_length' in a margin with
 ↪  by=[col("SEX")]
```

OpenDP does not verify either bound. The bound of 16 comes from the public sample, which is suitable for a tutorial but not a private release. Set both bounds conservatively from the survey design. If the data violate a bound, the guarantee fails without an error.

4.6. The noise per group and the ungrouped count

Every count in this chapter has scale 144, whether it counts one of the 16 groups or all rows.

```
ungrouped count len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
error bar as a share of the true count: all rows 0.011%, largest group 0.063%, smallest
 ↪  group above the threshold 3.3%
```

One person can contribute all 36 rows to a group, so its count is as sensitive as the total. Relative error differs: it is 3.3% for the smallest group above the threshold. Finer groupings reduce utility twice: more groups fall below the threshold, and those that remain have larger relative errors. Use only the grouping detail the question requires, and request several statistics in one query.

Assessing utility and choosing the noise compares Gaussian noise, whose scale depends on how one person's rows spread across groups. Identifier truncation and bounds shows how to bound that spread.

4.7. Checking the release against the truth

The LFS sample is public, so the script compares each release with the true table. Run this check on public or historical data during development. For stable keys, the smallest group above the threshold has 13,157 rows; the largest below it has 847. No group is near the threshold, so every run returns 14 groups. A group within one or two error bars of the threshold would appear only on some runs.

```
smallest true count above the threshold: 13,157  largest below it: 847
test bound for a table: 4 x accuracy = 1,727.5
released (one run):
  stable groups released: 14  smallest released count: 13,321  smallest true count
 ↪  among them: 13,157
  stable AGE=47 ILOSTAT=1: 652,892  (true 652,801)
  stable AGE=65 ILOSTAT=2: 13,321  (true 13,157)
  stable largest |released - true|: 349
  explicit YEAR=2005 ILOSTAT=4: 312  (true 0)
  explicit YEAR=2013 ILOSTAT=1: 188,396  (true 188,349)
  explicit keys released: 54  largest |released - true|: 530
  quarter 2005 Q1: 87,307  (true 87,372)
  quarters released: 36  within their 95% accuracy: 36 of 36  largest |released -
 ↪  true|: 307
  mean hours: SEX=1 40.786 (true 40.778)  SEX=2 34.229 (true 34.224)
```

Compare the run with the 431.9 error bar. The stable release contains 14 groups, all within the bar. The explicit release contains all 54 keys, including the 2005 conscript group, whose true count is zero and whose released count is pure noise. Its largest error slightly exceeds the bar, as expected for 54 counts when each misses a 95% bar about once in twenty. All 36 quarters fall within their bars, and the means are within 0.01 hours of the true values.

The test checks deterministic outputs exactly and each named count and mean within three error bars. It allows four bars for the largest table error because a table has up to 54 counts. The practitioner edition's Release statistics with a library in the practitioner edition performs a similar check on retail customers.

4.8. Summary

Table 4.1.: The calls and concepts of a grouped release, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`group_by(...).agg(dp.len(...))` without keys or margins	Releases the key set is private	Choose stable keys, explicit keys or a margin before you query
Stable keys, `dp.loss_of(epsilon, delta)`	Releases only groups whose noisy count exceeds a threshold	The threshold costs δ; read it in `summarize`
The `threshold` column	The smallest value such that a one-person group clears it with probability at most δ, over every group that person could create	Grows with the scale, with the privacy unit and with a smaller δ
`with_keys(keys)`	Joins a public key list; every key is released, zeros as noise	The list is fixed without the data; a released key set counts as public
`join(keys, how="right")`	The join that `with_keys` stands for	Only left or right joins release a key set
`Margin(by, invariant="keys")`	The keys of that grouping are released in the clear	Presence and absence of a key are unprotected; use for a design the public knows
`Margin(by, invariant="lengths")`	The group sizes are public, keys included	Gives up more than keys; filter before the context, not inside the query
`max_groups`, `max_length`	Public bounds on the number of groups and the rows per group	`max_groups` lowers the threshold; a sum needs `max_length`; set both from the design
One scale for every group	A person's rows can all sit in one group	Finer groups keep the noise and lose relative accuracy

4.9. References

- OpenDP documentation: "Grouping" in the tabular-data tutorial, and "Aggregation" and "Polars" in the API user guide.
- Korolova, Kenthapadi, Mishra and Ntoulas, "Releasing Search Queries and Clicks Privately" (2009): thresholded noisy counts as a way to release keys privately.
- Desfontaines, Voss, Gipson and Mandayam, "Differentially Private Partition Selection" (2022): the threshold, its δ, and why one person's groups add up.
- The full edition's chapters on sensitivity in the full edition, whose stability-based release is the thresholded count, and on post-processing and composition in the full edition, which is why a released key set can be reused for free.

5. Identifier truncation and bounds

A survey respondent may own several rows, one per quarter. Earlier chapters used the fixed unit from The data and the context, allowing up to 36 rows per person and increasing every noise scale by that factor. This chapter uses `PIDENT` to identify a person's rows and bounds their contribution separately for each query. The guarantee still protects the whole record, while the noise reflects the query's contribution bound.

The chapter follows the documentation's identifier-truncation page and user guide on truncation and bounds. It shows how to bound each person's rows and groups under differential privacy (DP), and distinguishes bounds enforced by the library from bounds it trusts.

The script opendp_identifier_truncation.py runs every query on the Labour Force Survey (LFS) sample with OpenDP 0.16.0. The scales, error bars, thresholds, error messages and facts about the data that it prints are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_identifier_truncation.py
```

5.1. A privacy unit with an identifier

Previously, the privacy unit was a number of rows. With an identifier, it is a column value. Datasets D and D' are neighbours when one is obtained from the other by adding or removing every row with one `PIDENT` value. This is **user-level privacy**: it protects a person's whole record, not just one row. The full edition's chapter on user-level privacy defines and analyses it. OpenDP represents this unit with a metric on identifiers and distance one.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.unit_of(contributions=1, identifier="PIDENT"))  # (SymmetricIdDistance {
 ↪  identifier: col("PIDENT") }, 1)
```

Here, `contributions=1` counts identifiers rather than rows. In this survey, each person has at most one `PIDENT` value. The unit does not bound a person's row count. The earlier bound of 36 had to hold across the entire dataset before querying; now each query specifies a bound that the library enforces through truncation.

The script first counts rows per identifier. This inspection is safe only because the data are public. For private data, derive the bound from the survey design; a bound calculated from the data leaks information.

```
identifiers: 1,682,306  rows: 3,811,932
rows per identifier: p50=2 p90=4 p99=7 max=15
identifiers with more than 10 rows: 195
quarters per identifier: max 4  rows per identifier and quarter: max 8
```

The 36-row worst case is far above the observed maximum. An identifier spans at most four quarters but can own up to eight rows in one quarter.

The context follows the documentation and includes the public row-count bound from The data and the context. It splits an approximate-DP budget of $\varepsilon = 1$ and $\delta = 10^{-8}$ evenly across four queries. The quarterly table needs δ because the library selects its keys from the data.

```
def make_context(frame: pl.LazyFrame, privacy_unit: tuple[dp.Metric, Any]) ->
↪  dp.Context:
    """The documentation's context: approximate differential privacy, split evenly over
    ↪  four queries."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=privacy_unit,  # dp.unit_of(contributions=1, identifier="PIDENT")
↪  below
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
        split_evenly_over=QUERIES,
        margins=[dp.polars.Margin(max_length=MAX_LENGTH)],
    )
```

```
budget per query: epsilon=0.25 delta=2.5e-09
```

OpenDP rejects queries on this context unless they truncate, because it cannot otherwise bound one identifier's effect on a statistic. In version 0.16.0, the error message does not identify the missing truncation.

```
try:
    context.query().filter(pl.col("HWUSUAL") !=
↪  99).select(hours().dp.mean(HOURS)).summarize()
    untruncated = "accepted"
except dp.OpenDPException as err:
    untruncated = reason(err)
```

```
mean without truncation: A step in your query is not recognized at this time: "DF
↪  ["PIDENT", "YEAR", "QUARTER", "SEX", ...]; PROJECT */8 COLUMNS". ""If you would
↪  like to see this supported, please file an issue.
```

The message blames the data scan rather than the missing truncation. On an identifier-based context, check that every statistic follows a truncation.

5.2. Truncating each person's rows

The documentation first computes the mean of `HWUSUAL` (usual hours), excluding code 99 ("not applicable"). The mean needs public bounds of 0 to 80 hours and a bound on rows per person. `truncate_per_group(10)` keeps at most ten random rows per identifier before aggregation.

```
mean_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)  # 99 means "not applicable"
    .truncate_per_group(ROWS_PER_IDENTIFIER)  # at most 10 rows per identifier, chosen
 ↪  at random
    .select(hours().dp.mean(HOURS))
)
mean_summary = mean_query.summarize(alpha=0.05)
dp_mean = mean_query.release().collect().item()
```

```
mean, 10 rows per identifier: HWUSUAL Sum, Integer Laplace, scale=6,400.0 95%
 ↪  accuracy=19,173.2
mean, 10 rows per identifier: HWUSUAL Length, Integer Laplace, scale=80.0 95%
 ↪  accuracy=240.2
```

A mean uses a noisy sum and count, as in Essential statistics. They split the query's budget share, so each gets $\varepsilon/8$. Truncation limits count sensitivity to 10, giving scale $10/(\varepsilon/8)$. Each row contributes at most 80 hours, so sum sensitivity and scale are 80 times larger. The full edition's chapter on sensitivity derives how truncation and clipping affect sensitivity and the answer.

With a 36-row unit, the same mean has scales 23,040 and 288. Truncation can bias the mean by dropping rows after the tenth. Here it drops only 20 of about 1.5 million rows.

```
rows with HWUSUAL not 99: 1,496,192 by 809,710 identifiers
identifiers with more than 10 such rows: 18  rows dropped: 20
unit of 36 contributions, no identifier: scales 23,040.0, 288.0
```

The `keep` argument selects which rows remain. The default, `"sample"`, retains ten random rows by shuffling each identifier's rows and keeping the first ten positions. `"first"` and `"last"` follow file order; they are cheaper but can be biased when that order matters. `SortBy` retains rows ordered by an expression such as employment status, `ILOSTAT`. The scales are unchanged because each method retains at most ten rows.

```
truncate_per_group(10) plan: FILTER
 ↪  [(0.int_range([len().cast(Int64)]).shuffle().over([col("PIDENT")])) < (10)]
keep=first: scales 6,400.0, 80.0
keep=last: scales 6,400.0, 80.0
keep=SortBy(ILOSTAT): scales 6,400.0, 80.0
ILOSTAT values among them: [1]
```

Every retained row has `ILOSTAT=1`, so sorting makes no difference in this dataset.

5.3. Truncating the groups a person touches

The second query groups responses by `QUARTER` and `YEAR`, releasing each quarter's count and sum of hours. Noise depends on two bounds: rows one person can add to a group and the number of groups they can affect. Sensitivity is the product of these bounds. The full edition's section on contribution bounding derives this result.

The query applies two truncations. `truncate_per_group(1, by=quarterly)` keeps one row per identifier and quarter; `truncate_num_groups(10, by=quarterly)` keeps at most ten quarters per identifier. Without the second truncation, OpenDP rejects the query and identifies the missing bound. The script omits the message's suggestion because its column order changes between runs.

```
per quarter without truncate_num_groups: num_groups (None), total contributions (None),
 ↪  and per_group (Some(1)) are not sufficiently well-defined. This is likely due to a
 ↪  missing truncation earlier in the data pipeline.
```

The query follows the documentation but sets `keep="first"` for group truncation; the reason appears below.

```
quarter_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)
    .truncate_per_group(1, by=QUARTERLY)  # one row per identifier and quarter
    .truncate_num_groups(GROUPS_PER_IDENTIFIER, by=QUARTERLY, keep="first")  # at most
 ↪  10 quarters each
    .group_by(QUARTERLY)
    .agg(dp.len(signed=True), hours().dp.sum(HOURS))
)
quarter_summary = quarter_query.summarize(alpha=0.05)
quarters_hat = quarter_query.release().collect()
```

```
per quarter, 10 quarters per identifier: len Frame Length, Integer Laplace, scale=80.0
 ↪  95% accuracy=240.2 threshold=1,714
per quarter, 10 quarters per identifier: HWUSUAL Sum, Integer Laplace, scale=6,400.0
 ↪  95% accuracy=19,173.2
```

These scales match the mean's: one row in each of ten quarters gives ten rows per person. Because the query has no public key list, OpenDP releases only quarters whose noisy counts exceed the threshold, as described in Grouping. Each quarter has at least 27,228 responses, so all 36 appear.

OpenDP 0.16.0 has two problems with `truncate_num_groups`. With the default `keep="sample"`, `summarize` succeeds and `release` spends the budget share, but Polars fails during `collect`.

```
sample_query = grouped_query(context, keep="sample")  # the same table, with the
 ↪  default keep
sample_summary = sample_query.summarize(alpha=0.05)
released = sample_query.release()  # spends the query's share of the budget
```

```
try:
    released.collect()
    sample_outcome = "collected"
except dp.OpenDPException as err:
    sample_outcome = reason(err)
```

```
keep=sample: summarize scales 80.0, 6,400.0; collect: multiple fields with name
 ↪  'QUARTER' found
shares spent after the failed collect: 4 of 4
```

The share is spent when the measurement is built. Polars can check the plan without reading data, so `explain()` detects the same error before `release`, at no cost. The toy frame demonstrates this check.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
toy = pl.LazyFrame({"PIDENT": [7, 7, 7], "YEAR": [2005] * 3, "QUARTER": [1, 2, 3],
 ↪  "HWUSUAL": [35.0, 36.0, 37.0]})
context = dp.Context.compositor(
    data=toy,
    privacy_unit=dp.unit_of(contributions=1, identifier="PIDENT"),
    privacy_loss=dp.loss_of(epsilon=1.0, delta=1e-8),
    split_evenly_over=1,
    margins=[dp.polars.Margin(max_length=100)],
)
quarterly = [pl.col("QUARTER"), pl.col("YEAR")]
query = context.query().truncate_per_group(1, by=quarterly).truncate_num_groups(10,
 ↪  by=quarterly)
query = query.group_by(quarterly).agg(dp.len(signed=True))
print(f"{query.summarize()['scale'].item():.1f}")  # 10.0
try:
    query.explain()
except pl.exceptions.DuplicateError as err:
    print(str(err).splitlines()[0])  # multiple fields with name 'QUARTER' found
```

`keep="first"` and `keep="last"` run, but their plans retain ranks below the requested group count. Polars ranks from one, so the truncation keeps one fewer group than requested, as the two numberings show.

```
import polars as pl

toy = pl.DataFrame({"PIDENT": [7, 7, 7], "YEAR": [2005] * 3, "QUARTER": [1, 2, 3]})
quarterly = [pl.col("QUARTER"), pl.col("YEAR")]
numbered = toy.with_columns(
    index=pl.int_range(pl.len()).over("PIDENT"),  # what truncate_per_group compares
 ↪  with k
    rank=pl.struct(quarterly).rank("dense").over("PIDENT"),  # what truncate_num_groups
 ↪  compares with k
```

```
)
print(numbered["index"].to_list(), numbered["rank"].to_list())  # [0, 1, 2] [1, 2, 3]
```

No identifier spans more than four quarters, so the script tests smaller values of `k`. Requesting four keeps at most three quarters and removes one quarter for each of the 13,948 identifiers with four. Requesting five keeps all four, with the scale and threshold for five groups.

```
truncate_num_groups(10, keep=first) plan: FILTER
 ↪  [(col("QUARTER").as_struct([col("YEAR")]).rank().over([col("PIDENT")])) < (10)]
identifier-quarters after the filter: 1,241,896  identifiers with 4 quarters: 13,948
truncate_num_groups(4, keep=first): keeps at most 3 quarters, 1,227,948
 ↪  identifier-quarters; scale 32.0 threshold 657
truncate_num_groups(5, keep=first): keeps at most 4 quarters, 1,241,896
 ↪  identifier-quarters; scale 40.0 threshold 830
```

In version 0.16.0, request one more group than you intend to retain and accept the corresponding larger noise. The guarantee remains valid because OpenDP calibrates to the requested count. The estimate can be biased because the dropped group is always last in sort order. The practitioner edition's release workflow avoids this call.

Place truncations next to each other, with group truncation immediately before aggregation. A filter between them makes the truncation expression unrecognized.

```
a filter between the two truncations: Expr is not recognized at this time: dyn int:
 ↪  0.int_range([len()]).shuffle().over([col("PIDENT"), col("QUARTER"), col("YEAR")]).
 ↪  If you would like to see this supported, please file an issue.
```

5.4. One row per person

For a mean, grouping by identifier can be preferable to retaining ten rows per person. Aggregate each person's rows to one row with a Polars expression, then take the private mean. After grouping, each identifier contributes one row.

```
person_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)
    .group_by("PIDENT")  # one row per identifier: the truncation
    .agg(pl.col("HWUSUAL").mean())  # any expression may summarize the person's rows
    .select(hours().dp.mean(HOURS))
)
person_summary = person_query.summarize(alpha=0.05)
dp_person_mean = person_query.release().collect().item()
```

```
mean of one row per identifier: HWUSUAL Sum, Integer Laplace, scale=640.0 95%
 ↪  accuracy=1,917.8
mean of one row per identifier: HWUSUAL Length, Integer Laplace, scale=8.0 95%
 ↪  accuracy=24.5
```

Count sensitivity is now 1 and sum sensitivity is 80, so both scales are one-tenth of those with ten rows. No rows are dropped.

This changes the estimand. The first query averages responses, so a person with four responses contributes four times. This query averages person-level means, giving each person equal weight: 37.24 hours rather than 37.64. Casting to integers truncates each person's mean toward zero and lowers the true value to 37.10. With an error bar of about 0.0035 hours, this choice matters far more than the noise.

```
true mean of one row per identifier: 37.0970  (before the cast to int: 37.2359; all
 ↪  rows: 37.6393)
per-identifier mean error bar at the true size: 0.0035
```

5.5. Bounds in the privacy unit

The unit can encode more detailed knowledge in a list of `dp.polars.Bound` objects. Each object specifies a grouping, the maximum identifiers per group and the maximum groups one person can affect. The documentation considers data merged from two sources that assigned identifiers separately. A person may then have two identifiers, increasing noise, but never two within the same quarter.

```
two_sources = dp.unit_of(
    contributions=[
        dp.polars.Bound(per_group=2),  # a person may appear under two identifiers
        dp.polars.Bound(by=QUARTERLY, per_group=1),  # but under one identifier in any
 ↪  quarter
    ],
    identifier="PIDENT",
)
bounded = make_context(frame, two_sources)
bounded_query = grouped_query(bounded, keep="first", k=5)  # each identifier may affect
 ↪  up to 5 quarters
bounded_summary = bounded_query.summarize(alpha=0.05)
bounded_hat = bounded_query.release().collect()
```

```
two identifiers, one per quarter, 5 quarters: len Frame Length, Integer Laplace,
 ↪  scale=80.0 95% accuracy=240.2 threshold=1,714
two identifiers, one per quarter, 5 quarters: HWUSUAL Sum, Integer Laplace,
 ↪  scale=6,400.0 95% accuracy=19,173.2
one identifier as a bound, 5 quarters: scales 40.0, 3,200.0
```

The query keeps one row per identifier and quarter and up to five quarters per identifier. Two identifiers across five quarters give ten rows per person, so the scale is again 80. The second bound says the identifiers never occur in the same quarter; one person therefore adds at most one row per quarter. This does not lower count sensitivity because the ten rows may occupy ten different quarters. A single-identifier bound halves the scale.

`num_groups` can also bound the number of groups one person affects. The query then needs no `truncate_num_groups`, and a promise of ten quarters gives the same scale as truncating at ten.

Unlike truncation, however, OpenDP does not enforce this bound. The script declares that no identifier spans more than three quarters, although the data contain 13,948 identifiers spanning four; the release proceeds without warning.

```
promised = dp.unit_of(
    contributions=[
        dp.polars.Bound(per_group=1),
        dp.polars.Bound(by=QUARTERLY, per_group=1, num_groups=3),  # a promise: at most
 ↪  3 quarters each
    ],
    identifier="PIDENT",
)
promise_query = (
    make_context(frame, promised)
    .query()
    .filter(pl.col("HWUSUAL") != 99)
    .truncate_per_group(1, by=QUARTERLY)  # no truncate_num_groups: the unit already
 ↪  bounds the quarters
    .group_by(QUARTERLY)
    .agg(dp.len(signed=True), hours().dp.sum(HOURS))
)
promise_summary = promise_query.summarize(alpha=0.05)
promise_hat = promise_query.release().collect()
```

```
num_groups=10 promised in the unit: len Frame Length, Integer Laplace, scale=80.0 95%
 ↪  accuracy=240.2 threshold=1,714
num_groups=3 promised in the unit: len Frame Length, Integer Laplace, scale=24.0 95%
 ↪  accuracy=72.4 threshold=486
num_groups=3 promised in the unit: HWUSUAL Sum, Integer Laplace, scale=1,920.0 95%
 ↪  accuracy=5,752.3
identifiers that break the promise of 3 quarters: 13,948
```

The noise assumes three quarters, but 13,948 identifiers span four. Their privacy loss exceeds the declared ε, with no warning in the output. Prefer truncation in the query, where OpenDP enforces the bound. Declare a unit bound only when truncation cannot express it, as when two identifiers represent one person and a filter cannot link them. Likewise, clip values in the query instead of declaring their range in the input domain. Assessing utility and choosing the noise revisits per-group bounds under Gaussian noise, whose scale depends on them differently.

5.6. The check against the truth

Because the data are public, the script computes each query's value without noise. The first mean uses its own seeded sample of ten rows. For quarterly values, the check uses the first row per identifier and quarter, while the release selects a random row. The count check is exact; the hours check is approximate.

```
true mean, 10 rows per identifier: 37.6393  (1,496,172 rows)
true per quarter (first row per identifier and quarter): 36 quarters, rows from 27,228
 ↪  to 41,868
```

```
true 2005Q1: rows 29,561  hours 1,110,610
true 2005Q2: rows 29,406  hours 1,106,170
true 2005Q3: rows 28,364  hours 1,070,826
true 2005Q4: rows 28,681  hours 1,080,009
```

The following values come from one run. Each mean's error bar combines the sum and count error bars and holds whenever both component bars hold.

```
released (one run):
  mean, 10 rows per identifier: 37.6366  (true 37.6393, within 0.0189 with probability
↪  at least 0.9)
  per quarter: 36 quarters released; rows within their 95% accuracy: 36 of 36; hours:
↪  35 of 36
  2005Q1: rows 29,644  hours 1,105,518
  2005Q2: rows 29,460  hours 1,098,965
  2005Q3: rows 28,244  hours 1,063,846
  2005Q4: rows 28,645  hours 1,087,028
  mean of one row per identifier: 37.0970  (true 37.0970, within 0.0035 with
↪  probability at least 0.9)
  two bounds, 5 quarters: 36 quarters released; rows within four times their 95%
↪  accuracy: 36 of 36
  num_groups=3 promised: 36 quarters released; rows within four times their 95%
↪  accuracy: 36 of 36
```

Both means fall within their error bars. So do all quarterly counts and 35 of the 36 sums, consistent with 95% coverage across 36 values. The test allows three error bars for the means and four for the quarterly values, so a correct run rarely fails.

5.7. Summary

Table 5.1.: The calls of identifier truncation and bounds, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.unit_of(contributions=1, identifier="PIDENT")`	The privacy unit is every row of one identifier	`contributions` now counts identifiers; the rows per person are bounded by the query
A query with no truncation	Refused, with a message that blames the data scan	On a context with an identifier, every statistic sits behind a truncation
`truncate_per_group(k, keep=...)`	Keeps at most `k` rows per identifier, and per group with `by`	`"sample"` is unbiased; `"first"`, `"last"` and `SortBy` are cheaper or deliberate, and biased when order matters

Call or concept	What it does	Rule of thumb
`truncate_num_groups(k, by=...)`	Keeps at most `k` groups per identifier	In 0.16.0, `"sample"` fails at `collect` after the budget is spent, and `"first"` keeps `k - 1` groups; call `explain()` before `release()`
Truncations in a query	Must sit next to each other, the group truncation last	A filter between them makes the query unrecognized
`group_by("PIDENT").agg(...)` before a statistic	One row per person, so the sensitivity is that of one row	The best choice for a mean, if the mean over people is the one you want
`dp.polars.Bound(by, per_group, num_groups)` in `unit_of`	Finer bounds on identifiers per group and groups per person	A promise the library trusts; a person who breaks it loses more than ε
Truncate or declare	The query enforces a truncation; the unit declares a bound	Truncate, and declare only what no truncation can express

5.8. References

- OpenDP documentation: "Identifier Truncation" in the tabular-data tutorial, and "Identifier Truncation and Bounds" in the Polars user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): sensitivity and the Laplace mechanism.
- Amin, Kulesza, Muñoz Medina and Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (2019): why truncating a person's rows trades bias for variance.
- Wilson, Zhang, Lam, Desfontaines, Simmons-Marengo and Gipson, "Differentially Private SQL with Bounded User Contribution" (2020): bounding rows per group and groups per user in grouped queries.
- The full edition's chapters on sensitivity in the full edition and user-level privacy in the full edition derive the results this chapter states.

6. Preparing microdata

Survey data rarely arrive in the form a statistic requires. Work hours are floats, 99 means "not applicable," and null means no answer. Response dates are split across two integer columns. Preparation may require casting, filling, clipping, filtering or recoding. This chapter shows where to perform those steps. OpenDP inspects the full lazy query and accounts for each step, so prepare data inside the query to keep that work within the guarantee.

The chapter follows the documentation's tutorial on preparing microdata and its guides to data types and expressions. It describes the supported Polars contexts and expressions, OpenDP's changes to them, and its treatment of nulls, undefined floats and bounds. The final example prepares Labour Force Survey (LFS) data to estimate the mean difference between actual and usual work hours.

The script opendp_preparing_microdata.py runs every step on OpenDP 0.16.0 with Polars 1.36.1. The noise scales, error bars, thresholds, error messages and true values that it prints are the same on every run. The released counts and mean are not, because OpenDP's noise cannot be seeded, so this page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_preparing_microdata.py
```

6.1. Prepare inside the query

A context on a Polars `LazyFrame` reads only its schema, which OpenDP treats as public, and accesses the data only at release. Before execution, OpenDP parses the query plan into a chain of transformations, as Transformations explains. It bounds each person's effect at every step, so filters and fills in the plan are included in the accounting.

If you prepare data before creating the context, OpenDP sees only the prepared frame and guarantees privacy for that frame. A fixed row-wise function, such as renaming a column or parsing a date with a known format, preserves each person's rows. The guarantee still applies, but you must justify that claim because OpenDP does not check the preparation. Data-dependent steps are unsafe: imputing with a column mean, bounding by its maximum or filtering at a quantile carries private information into the release outside the accounting. Before creating the context, use only public facts such as the schema, codebook and published sample-size bound.

The context uses the privacy unit and row-count bound from The data and the context: at most 36 rows per person. It splits an approximate differential privacy budget of $\varepsilon = 1$ and $\delta = 10^{-7}$ evenly over four queries.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The documentation's context: 36 rows per person, (epsilon = 1, delta = 1e-7)
     ↪ split evenly over 4 queries."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
        split_evenly_over=QUERIES,
        margins=[dp.polars.Margin(max_length=MAX_LENGTH)],  # public: a bound on the
 ↪ number of rows
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(MaxDivergence), (1.0, 1e-07))
budget per query: epsilon=0.25 delta=2.5e-08
schema: PIDENT=Int64  YEAR=Int64  QUARTER=Int64  SEX=Int64  AGE=Float64
 ↪ HWUSUAL=Float64  HWACTUAL=Float64  ILOSTAT=Int64
```

At this point, the context knows only the schema and assumes that every column can contain nulls.

6.2. Columns, filters and groups

Polars evaluates expressions in five contexts, which OpenDP divides into two groups. Expressions in `.select` and `.agg` may change row count or order, as aggregations do. Expressions in `.with_columns`, `.filter` and `.group_by` must operate **row by row**. This preserves the privacy unit: each person's 36 input rows remain their own 36 output rows. Sorting or shifting could mix rows from different people, making one person's effect impossible to bound.

The documentation first adds a column with `.with_columns` and groups by it. The row-wise `cut` expression bins usual hours at fixed breaks and replaces `HWUSUAL` with its bin. Because the labels depend only on the breaks, they are public keys.

```
def binned_counts(context: dp.Context) -> LazyFrameQuery:
    """How many responses fall in each band of usual hours: a row-by-row `cut`, then a
     ↪ count per bin."""
    labels = pl.Series("HWUSUAL", cut_labels(BREAKS, left_closed=True),
 ↪ dtype=pl.Categorical)
    return (
        lazy_query(context)
        .with_columns(pl.col("HWUSUAL").cut(breaks=BREAKS, left_closed=True))  #
 ↪ shadows HWUSUAL with its bin
        .group_by("HWUSUAL")
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame([labels]))  # the bin labels are public, so no
 ↪ threshold is needed
    )
```

```
binned counts: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
without keys: len Frame Length, Integer Laplace, scale=144.00 threshold=2,973
true bins: null=20,819  [-inf, 0)=0  [0, 20)=89,495  [20, 40)=908,061  [40, 60)=406,947
↪  [60, 80)=78,469  [80, 98)=13,220  [98, inf)=2,294,921
```

Scale is 36 rows per person divided by the query's budget share. With public keys, every bin is released, including the empty bin below zero, whose signed count may be negative. Without public keys, OpenDP must determine from the data which bins exist, so it applies a threshold as described in Grouping. Null `HWUSUAL` values form a null bin absent from the key list and are not released. A margin on `HWUSUAL` also applies only to the original column, not the new bin column.

`.filter` removes rows using a boolean row-wise expression, such as the documentation's filter for code 99 before a sum. A filter preserves most margin information but invalidates invariants about keys and group sizes. It also does not prove which values remain: a column filtered to non-null values is still nullable in its domain.

```
filtered sum: HWUSUAL Sum, Integer Laplace, scale=11,520.00 95% accuracy=34,511.3
```

The tutorial's final context applies `.group_by` and `.agg` *before* private aggregation. In its sample-and-aggregate example, rows enter 1,000 groups by `PIDENT`, and the query releases the mean of each group's minimum usual hours. The inner `.agg` accepts any expression, but privacy adds noise. The script compares mean scales under the current context and with a `Bound` from Identifier truncation and bounds that assigns each person's rows to one group.

```
sample and aggregate: HWUSUAL Sum, Integer Laplace, scale=17,280.00
sample and aggregate: HWUSUAL Length, Integer Laplace, scale=576.00
one group per person: HWUSUAL Sum, Integer Laplace, scale=480.00
one group per person: HWUSUAL Length, Integer Laplace, scale=16.00
```

The first context does not use `PIDENT` to link a person's rows, so it allows 36 changed groups. The inner aggregation doubles noise even with the bound: a changed group counts as one value removed and another added.

6.3. The types OpenDP tracks

OpenDP assumes columns may contain nulls until an expression proves otherwise. It also tracks **domain descriptors**, facts about values that later aggregations need. Numeric descriptors include lower and upper bounds, set by `.clip` and removed by some later steps. OpenDP supports signed integers and unsigned 32- and 64-bit integers, but rejects `UInt8`. Float descriptors also record whether values may be NaN, an undefined numeric value; a float sum therefore needs both `.fill_null` and `.fill_nan`. Booleans and strings have no descriptors.

Categorical columns store integer codes and a label table. Two operations can make the table data-dependent. An expression such as `.fill_null` may add or remove categories, causing Polars to rewrite codes and emit a data-dependent warning. Also, codes follow the values' order of first

appearance, so grouping by them can reveal row order. OpenDP rejects both cases. An `Enum` fixes categories in the schema, avoiding these problems; `cut` is also safe because its encoding depends only on fixed breaks.

```
import polars as pl
import opendp.prelude as dp

dp.enable_features("contrib")
status = ["working", "absent", "other"] * 100
unit, loss = dp.unit_of(contributions=1), dp.loss_of(epsilon=1.0, delta=1e-7)


def context_for(column: pl.Series) -> dp.Context:
    return dp.Context.compositor(data=pl.LazyFrame([column]), privacy_unit=unit,
     ↪  privacy_loss=loss, split_evenly_over=1)


categorical = context_for(pl.Series("status", status, dtype=pl.Categorical))
try:
    categorical.query().group_by("status").agg(dp.len(signed=True)).summarize()
except dp.OpenDPException as err:
    print(err.message)  # Categories are data-dependent, which may reveal sensitive
     ↪  record ordering. Cast status to string before grouping.
enum = context_for(pl.Series("status", status, dtype=pl.Enum(["working", "absent",
 ↪  "other"])))
print(f"{enum.query().group_by('status').agg(dp.len(signed=True)).summarize()['scale']
 ↪  .item():.4f}")  # 1.0000
```

The error names the fix, a cast to string, and the enum query goes through. Dates, datetimes and times are also supported, with one time zone per datetime column.

6.4. Casting, filling and clipping

The documentation recommends casting because floating-point sums incur rounding error. OpenDP accounts for that error, which can increase the noise scale for many rows and wide bounds. The script compares both sum types.

```
def hours_sum(context: dp.Context, as_integers: bool) -> pl.DataFrame:
    """The noise scale of a sum of usual hours, on the float column or after casting it
     ↪  to integers."""
    usual: Any = pl.col("HWUSUAL")  # typed Any: OpenDP adds the `.dp` namespace to
 ↪  expressions at run time
    column = usual.cast(int) if as_integers else usual
    return lazy_query(context).select(column.dp.sum(HOUR_BOUNDS)).summarize(alpha=0.05)
```

```
float sum: HWUSUAL Sum, Float Laplace, scale=14,405.18 95% accuracy=43,154.1
integer sum: HWUSUAL Sum, Integer Laplace, scale=14,400.00 95% accuracy=43,139.0
```

For integers, the scale is exactly 36 times the upper bound of 100 divided by the budget share; for floats it is slightly larger. Casting also changes error handling. Polars normally raises a data-dependent error when a value cannot be converted. OpenDP instead returns null, so every cast column is nullable and needs `.fill_null` before aggregation.

```
import polars as pl
import opendp.prelude as dp

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": ["40", "35", "n/a"] * 100}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(max_length=1000)],
)
hours = pl.col("hours").cast(int)  # OpenDP turns "n/a" into a null instead of raising
 ↪  an error
try:
    context.query().select(hours.clip(0, 80).sum().dp.noise()).summarize()
except dp.OpenDPException as err:
    print(err.message)  # input data (hours) might contain nulls. Preprocess your data
     ↪  with `.fill_null`.
query = context.query().select(hours.fill_null(0).clip(0, 80).sum().dp.noise())
print(query.summarize()["scale"].item())  # 80.0
```

Earlier chapters' `.dp.sum` handles imputation and clipping automatically. This snippet prepares the column manually and adds noise to a plain `.sum`, showing the checks OpenDP applies. A sum needs bounds from `.clip`; without them, one person's effect is unbounded. `.fill_null` and `.fill_nan` remove bounds, so fill first and clip afterward. The script applies this sequence to the survey in `.with_columns` and tests incorrect orders.

```
def prepared_sum(context: dp.Context, steps: pl.Expr) -> pl.DataFrame:
    """A sum with noise on a column prepared in with_columns: OpenDP checks the steps
     ↪  left it non-null and bounded."""
    total: Any = pl.col("HWUSUAL").sum()
    return lazy_query(context).with_columns(steps).select(total.dp.noise()).summarize()
```

```
prepared sum: HWUSUAL Sum, Integer Laplace, scale=14,400.00
cast without fill_null: input data (HWUSUAL) might contain nulls. Preprocess your data
 ↪  with `.fill_null`.
clip before fill_null: input domain must consist of bounded data. Either specify bounds
 ↪  in the input domain or use make_clamp.
fill_null before fill_nan: input data (HWUSUAL) might contain nans. Preprocess your
 ↪  data with `.fill_nan`.
float prepared sum: HWUSUAL Sum, Float Laplace, scale=14,405.18
```

You can fill from another column, such as actual hours for missing usual hours. Both columns must be row-wise so their rows align, and the fill column must contain no nulls.

```
filled from a column: HWUSUAL Sum, Integer Laplace, scale=14,400.00
filled from a nullable column: fill expression must not be nullable
```

`.drop_nulls` and `.drop_nans` change column length by removing rows, so OpenDP forbids them in a row-wise context.

```
drop_nulls in with_columns: drop_nulls is not allowed in a row-by-row context
```

Recoding uses `.replace` or `.replace_strict`. `.replace` swaps selected values without changing the type. `.replace_strict` maps every value and may change the type; unmapped values receive `default`. The documentation says OpenDP sets a missing `default` to null, but version 0.16.0 does not. With three toy codes and a mapping for two, `summarize` succeeds but `release` fails. Always supply `default` to `.replace_strict`.

```
replace_strict without default: summarize scale=2.0; release:
 ↪  InvalidOperation(ErrString("incomplete mapping specified for
 ↪  `replace_strict`\n\nHint: Pass a `default` value to set unmapped values."))
replace_strict with default: released keys one other two
```

6.5. Predicates and arithmetic

OpenDP accepts Polars comparisons, conjunctions and arithmetic as row-wise expressions. Both operands of a binary operator must be row-wise, even in `.select` or `.agg`; otherwise a length mismatch could raise a data-dependent error. Tests such as `is_null` and `is_nan` are also row-wise and can define filters or grouping columns. The documentation counts respondents over retirement age by grouping on a comparison.

```
def over_retirement_age(context: dp.Context) -> LazyFrameQuery:
    """How many responses come from people over 64: a boolean expression as the
     ↪  grouping column."""
    return (
        lazy_query(context)
        .group_by(pl.col("AGE") > RETIREMENT_AGE)
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame({"AGE": [True, False]}))  # a null age is in neither
 ↪  group and is dropped
    )
```

```
over 64: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true over 64: true=1,190,845  false=2,620,832  null=255
```

Comparisons with nulls return null, so the 255 rows without an age form a third group omitted from the keys. One grouped query answers both sides of the comparison.

Version 0.16.0 does not support `is_in`, which tests membership in a list. Replace it with a disjunction of equality tests; the script uses this form to count employed respondents.

```
is_in: IsIn { nulls_equal: false } is not currently supported
employed: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

The other is the absolute value, in Section 6.8.

6.6. Strings and dates

OpenDP supports string parsing into temporal types through `.str.strptime` and its variants `to_date`, `to_datetime` and `to_time`. The `format` argument is required because inferring a format from data could make it depend on one person's rows. OpenDP rejects nanosecond datetimes because the parser can panic on some inputs, producing a data-dependent error.

```
to_date without format: format must be specified; otherwise Polars will attempt to
 ↪  infer the format from the data, resulting in an unstable transformation
nanosecond datetime: Nanoseconds are not currently supported due to potential panics
 ↪  when parsing inputs. Please open an issue on the OpenDP repository if you would
 ↪  find this functionality useful. Otherwise, consider parsing into micro- or
 ↪  millisecond datetimes instead.
```

The survey stores dates in `YEAR` and `QUARTER`, not a date column. The documentation combines them with `concat_str` before the context because OpenDP rejects concatenation inside a query.

```
concat_str: Expr is not recognized at this time: ConcatHorizontal { delimiter: "-",
 ↪  ignore_nulls: false }. If you would like to see this supported, please file an
 ↪  issue.
```

Arithmetic is supported, so you can construct the date inside the query. Multiplying the year by 100 and adding three times the quarter gives an integer such as 200503. Parsing the string with `%Y%m` yields the first day of the quarter's final month. `.dt` expressions then extract date components.

```
def quarter_end() -> pl.Expr:
    """The last month of the quarter as a date, built from YEAR and QUARTER and parsed
     ↪  with a fixed format."""
    year_month = pl.col("YEAR") * 100 + pl.col("QUARTER") * 3  # 2005 and 1 become
 ↪  200503
    return year_month.cast(str).str.to_date(format="%Y%m")


def responses_per_month(context: dp.Context) -> LazyFrameQuery:
    """How many responses carry each quarter-end month, read off the parsed date."""
    return (
        lazy_query(context)
        .with_columns(DATE=quarter_end())
        .group_by(pl.col("DATE").dt.month().alias("MONTH"))
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame({"MONTH": pl.Series(QUARTER_END_MONTHS,
 ↪  dtype=pl.Int8)}))
```

```
    )
```

```
first dates: 2005 Q1 -> 2005-03-01  2005 Q2 -> 2005-06-01
per month: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true per month: 3=953,695  6=958,396  9=933,257  12=966,584
```

Each quarter ends in one of four public months, so the query uses those months as keys without a threshold. Their type is `Int8`, matching the output of `.dt.month`.

6.7. A worked preparation: hours lost to absence

The statistic is the mean difference between actual and usual hours for employed respondents. A negative mean means holidays, illness and short weeks outweigh overtime. The query performs each preparation step: it keeps `ILOSTAT=1` (working for pay or profit), casts both hour columns to integers for integer noise (Section 6.4), fills missing usual hours with 0, then keeps rows with known usual hours, excluding 0 (variable hours) and 99 (not applicable).

Fill before filtering because a filter does not prove non-nullness to OpenDP (Section 6.2). The filtered column still has a nullable domain, and the next step uses it as a fill value, which must not be nullable. Missing actual hours are replaced by usual hours, making the difference zero. Finally, the query clips differences to the public range of ±40 hours, as required for a bounded sum. Using ±80 would double the noise.

```
def prepare[Frame: (LazyFrameQuery, pl.LazyFrame)](frame: Frame) -> Frame:
    """The rows of people who work, with their usual hours known, and the hours they
     ↪  lost that week, clipped."""
    low, high = LOST_BOUNDS
    return (
        frame.filter(pl.col("ILOSTAT") == 1)  # working for pay or profit
        .with_columns(pl.col("HWUSUAL").cast(int).fill_null(0),
↪   pl.col("HWACTUAL").cast(int))
        .filter((pl.col("HWUSUAL") > 0) & (pl.col("HWUSUAL") < 99))  # 0 is "varies",
↪   99 "not applicable"
        .with_columns(pl.col("HWACTUAL").fill_null(pl.col("HWUSUAL")))  # unknown
↪   actual hours: assume the usual
        .with_columns(LOST=(pl.col("HWACTUAL") - pl.col("HWUSUAL")).clip(low, high))
    )
```

The function accepts a query or a plain `LazyFrame`, so the release and validation use the same preparation. The query ends with `pl.col("LOST").dp.mean(LOST_BOUNDS)`; `summarize` reports its privacy cost.

```
hours lost: LOST Sum, Integer Laplace, scale=11,520.00 95% accuracy=34,511.3
hours lost: LOST Length, Integer Laplace, scale=288.00 95% accuracy=863.3
hours lost, bounds (-80, 80): LOST Sum, Integer Laplace, scale=23,040.00 95%
 ↪  accuracy=69,022.2
```

The mean uses a noisy sum and count, which split the query's budget share. The sum's scale is 36 rows times the bound of 40, divided by one-eighth of the budget. With bounds of ±80, its scale doubles. Clipping trades bias against noise: narrower bounds reduce noise but can increase bias. The full edition's chapter on sensitivity derives the sum's sensitivity.

Because the survey is public, the script measures clipping bias. On private data, use a public or historical copy for this check.

```
true prepared rows: 1,490,113  true sum of hours lost: -9,289,648  true mean: -6.2342
rows the clip changes: 42,830 (2.874%)  mean without the clip: -6.5319
mean error bar at 95%: 0.0268
```

Clipping changes fewer than 3% of rows but shifts the true mean from -6.53 to -6.23 hours, a bias of about one-third of an hour. This exceeds the noise. The script combines the sum and count accuracies into a mean error bar: the sum's bar plus the mean times the count's bar, divided by the count. The result is 0.027 hours. At this sample size, clipping bias, not noise, limits accuracy.

```
released (one run):
  bins: [-inf, 0)=-129  [0, 20)=89,491  [20, 40)=907,895  [40, 60)=406,594  [60,
↪  80)=78,436  [80, 98)=13,452  [98, inf)=2,294,708
  over 64: true=1,190,894  false=2,620,771
  per month: 3=953,684  6=958,313  9=933,077  12=966,807
  hours lost: mean -6.2147  (true -6.2342, within 0.0268 with probability 0.95)
budget spent: 4 of 4 queries
```

Every count falls within its error bar of about 432. The empty bin is negative, which signed counts allow. The mean falls within its bar relative to the clipped truth. These four releases use all four budget shares. Assessing utility and choosing the noise develops the error-bar analysis further.

6.8. What a refused expression looks like

An unsupported expression raises an `OpenDPException` with a variant and message when you build the query, before OpenDP reads any data. Absolute difference in hours is one example.

```
abs: Expr is not recognized at this time: [(col("HWACTUAL")) - (col("HWUSUAL"))].abs().
↪  If you would like to see this supported, please file an issue.
```

The messages fall into two categories. "Not recognized at this time" means the expression has no implementation, as with `concat_str` and `abs`. Other messages identify a rule, such as a required format or non-null fill, and give the in-query fix. Rewrite unsupported expressions with supported operators when possible, as with `is_in`. Otherwise, apply the expression before the context only if it is a fixed row-wise function. Disclose this step with the release because OpenDP did not check it.

6.9. Summary

Table 6.1.: The contexts and expressions of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
Preparation inside the query	OpenDP parses every step and accounts for it	Prepare before the context only with fixed row-by-row steps and public facts, and say so
`.with_columns`, `.filter`, `.group_by`	Row-by-row contexts: each person's rows stay their own	A new column has no margins; a filter drops key and length invariants and proves nothing about what remains
`.select`, `.agg`	May change the number and order of rows	Binary operators and fills inside them still need row-by-row inputs
`.cut(breaks)`	Bins a number into categories with a data-independent encoding	Pass the labels as keys to avoid a threshold
Domain descriptors	Nullable, may be NaN, bounds	Every column starts nullable; floats need `.fill_null` and `.fill_nan`; `.clip` sets bounds
`.cast`	A failed cast returns null instead of raising	Follow it with `.fill_null`; cast to integers for a cheaper sum
`.fill_null`, `.fill_nan`	Impute; a column fill must be row-by-row and non-nullable	Fill first, then clip, because filling drops the bounds
`pl.Categorical` vs `pl.Enum`	Categories from the data vs fixed in the schema	Group by enums; cast categoricals to string
`.replace_strict(mapping)`	Recodes every value	Pass a `default`: in 0.16.0 a missing one fails at release
Comparisons, `&`, `|`, arithmetic	Row-by-row predicates and new columns	`is_in` and `abs` are not supported in 0.16.0
`.str.to_date(format=...)`	Parses a string into a date	The format is mandatory; nanosecond datetimes are refused
`.dt.month()` and the other components	Reads a part of a date or time	Keys for a component must match its type (`Int8` for a month)
`OpenDPException`	Raised when the plan is parsed, before any data are read	"Not recognized" is unimplemented; the other messages state a rule and its fix

6.10. References

- OpenDP documentation: “Preparing Microdata” in the tabular data tutorial; “Data Types” and the “Expressions” pages “Boolean”, “Columns”, “Manipulation”, “Operators”, “String” and “Temporal” in the Polars user guide.
- Nissim, Raskhodnikova and Smith, “Smooth Sensitivity and Sampling in Private Data Analysis” (2007): the sample and aggregate framework behind the stable `group_by/agg`.
- Gaboardi, Hay and Vadhan, “A Programming Framework for OpenDP” (2020): the stable transformations that each expression is parsed into.
- The full edition’s chapter on sensitivity in the full edition derives how the bounds of a clipped column fix the noise of its sum and mean.

7. Assessing utility and choosing the noise

Earlier chapters reported each query's noise scale and accuracy before release. This chapter explains how to interpret and adjust them. **Accuracy** is an error bar's half-width: with the stated probability, the released value is within that distance of the value to which noise was added. OpenDP converts between scale and accuracy, so you can price a target error bar before spending the budget.

The chapter also compares noise mechanisms. A pure differential privacy (DP) budget makes the context add Laplace noise; a zero-concentrated differential privacy (zCDP) budget makes it add Gaussian noise. An argument to `dp.loss_of` selects the guarantee and mechanism. The choice matters most when one person contributes to many groups: Gaussian noise uses sensitivity proportional to the square root of the group count rather than the count itself. On the survey, this reduces quarterly-count noise six-fold under zCDP.

The script opendp_utility_and_noise.py runs every step on OpenDP 0.16.0 against the Labour Force Survey (LFS) sample of the earlier chapters. The conversions, scales, accuracies, thresholds and budgets that it prints are the same on every run. The released counts and the means of a small clamping experiment are not, because OpenDP's noise cannot be seeded, so this page shows them in blocks marked as one run.

```
uv run python code/part02-tabular-data/opendp_utility_and_noise.py
```

7.1. What an accuracy statement means

At a given scale, the released value lies within the error bar of the mechanism's input with probability $1 - \alpha$, or confidence $(1 - \alpha) \times 100\%$. This book uses $\alpha = 0.05$, so about one in twenty releases falls outside the bar. Smaller α gives a wider bar, as shown at $\alpha = 0.01$ in Essential statistics.

The `opendp.accuracy` module, exposed as `dp` in the prelude, provides eight conversion functions. Four convert continuous Laplace or Gaussian scales and accuracies; four functions prefixed `discrete_` do the same for integer noise. The documentation's example finds the Gaussian scale needed to stay within 2 of the input with 95% confidence.

```
import opendp.prelude as dp

print(dp.accuracy_to_gaussian_scale(accuracy=2.0, alpha=0.05))  # 1.020426913849308
print(dp.gaussian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 352.7935172172097
print(dp.discrete_gaussian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 354.0
print(dp.laplacian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 539.2318092397184
print(dp.discrete_laplacian_scale_to_accuracy(scale=180.0, alpha=0.05))  #
 ↪  539.7311147961669
```

The continuous conversions have closed forms. Laplace accuracy is scale times $\ln(1/\alpha)$; Gaussian accuracy is scale times the two-sided normal quantile, about 1.96 at the usual confidence level. Discrete conversions sum integer-noise probabilities, so discrete Gaussian accuracy is an integer. Use discrete conversions for counts, which receive integer noise, and continuous conversions for sums and means of floats. At scale 180 the difference is small; `summarize` selects the appropriate conversion.

The error bar covers noise only. It measures the distance from the release to the mechanism's input: the statistic after preparation. Clipping, imputation, row truncation and resizing can change that input. The bar does not cover the resulting bias relative to the intended quantity; repeated releases do not remove bias. The practitioner edition's release workflow in the practitioner edition measures bias from truncation and clipping, which retain about half the revenue.

7.2. A target error bar on the survey

Suppose the survey response count must fall within 100 of its true value with 95% confidence. The count in Essential statistics used $\varepsilon = 1$ split among five queries. The excerpt summarizes that query and calculates the discrete Laplace scale required for the target accuracy.

```
laplace = laplace_context(frame, EPSILON)
count_laplace = laplace.query().select(dp.len(signed=True))
laplace_summary = count_laplace.summarize(alpha=ALPHA)
scale_needed = float(dp.accuracy_to_discrete_laplacian_scale(accuracy=TARGET,
 ↪  alpha=ALPHA))
epsilon_needed = CONTRIBUTIONS / scale_needed  # the Laplace scale is the sensitivity
 ↪  over epsilon
```

```
laplace count: Integer Laplace scale=180 95% accuracy=539.731
budget per query: epsilon=0.2000
error bar relative to the true count: 0.014%
discrete laplace scale for accuracy 100: 33.2152
epsilon that scale needs at sensitivity 36: 1.0838  (5.42 times the share)
```

The current scale, 180, is sensitivity 36 divided by the query share 0.2; see the privacy unit. Accuracy 100 requires scale 33.2, which would consume more than the full budget on this query. The current error bar is 0.014% of the true count, already small for a headline figure.

The conversion also gives the trade-off: Laplace scale is sensitivity divided by ε, and accuracy is proportional to scale. Halving the error bar therefore doubles the budget. At $\varepsilon = 2$, this query's scale is halved.

```
laplace count at epsilon 2: Integer Laplace scale=90 95% accuracy=270.115
```

The discrete conversion makes the bar slightly different from half because integer noise is rounded. The rule of thumb still holds. The full edition's chapter on the Laplace mechanism derives the scale and tail bound.

7.3. Laplace under pure DP, Gaussian under zCDP

The budget type determines the default noise. `dp.loss_of(epsilon=...)` selects pure DP and Laplace noise; `dp.loss_of(rho=...)` selects zCDP and Gaussian noise. Other query details stay the same unless a query specifies a distribution, so the two context functions differ only in their budget declarations.

```
def laplace_context(frame: pl.LazyFrame, epsilon: float) -> dp.Context:
    """Pure differential privacy: the context adds Laplace noise."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=epsilon),
        split_evenly_over=QUERIES,
    )


def gaussian_context(frame: pl.LazyFrame, rho: float, delta: float | None = None) ->
↪  dp.Context:
    """Zero-concentrated differential privacy: the context adds Gaussian noise."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(rho=rho, delta=delta),
        split_evenly_over=QUERIES,
    )
```

Privacy loss is the log ratio of an output's probabilities with and without a person's data. Pure DP bounds it by ε in the worst case. zCDP bounds its moment-generating behaviour: a mechanism is ρ-zCDP when privacy loss has mean at most ρ and tails no heavier than a normal variable with variance 2ρ. The Gaussian mechanism satisfies this bound exactly, and ρ budgets add across queries like ε budgets. The full edition's chapter on concentrated DP derives the guarantee. The documentation's Gaussian example repeats the count with $\rho = 0.1$ split across five queries.

```
gaussian = gaussian_context(frame, RHO)
count_gaussian = gaussian.query().select(dp.len(signed=True))
gaussian_summary = count_gaussian.summarize(alpha=ALPHA)
```

```
privacy loss: (ZeroConcentratedDivergence, 0.1)
gaussian count: Integer Gaussian scale=180 95% accuracy=354
budget per query: rho=0.0200  sqrt(2 rho)=0.2000
continuous gaussian accuracy at scale 180: 352.7935
```

The scale is again 180. Let σ denote the Gaussian standard deviation (OpenDP's scale) and Δ the sensitivity. The mechanism satisfies $\Delta^2/(2\sigma^2)$-zCDP, so the context sets $\sigma = \Delta/\sqrt{2\rho}$. At $\rho = 0.02$, the denominator is 0.2, matching the Laplace budget share. Gaussian noise has a narrower bar at equal scale because its tails decay faster. Equal scales do not imply equal privacy: Section 7.5 converts this ρ share to $\varepsilon \approx 1$, five times the Laplace share.

The Gaussian trade-off differs: scale is sensitivity divided by $\sqrt{2\rho}$, so halving the error bar requires four times the ρ budget. The script confirms this at $\rho = 0.4$.

```
gaussian count at rho 0.4: Integer Gaussian scale=90 95% accuracy=177
```

Gaussian noise under zCDP offers no gain for a single count. Its advantage appears in queries that release many values at once.

7.4. One person in many groups: L1 against L2

A grouped count returns one number per group. Noise is calibrated to the distance one person can move the full vector. Laplace noise uses the L_1 norm, the sum of absolute changes; Gaussian noise uses the L_2 norm, the square root of the sum of squared changes. Opposite corners of a unit square have distances 2 in L_1 and $\sqrt{2}$ in L_2. In d dimensions, these distances are d and $\sqrt{d}$.

Each survey respondent answers at most once per quarter, so their 36 records occupy 36 year-quarter groups, changing each count by one. The count vectors have L_1 distance 36 and L_2 distance $\sqrt{36} = 6$. Without this information, OpenDP must assume all 36 records fall in one group, making both distances 36. A `dp.polars.Bound`, described in Identifier truncation and bounds, supplies the per-group bound.

```
def bounded_context(frame: pl.LazyFrame, privacy_loss: tuple[Measure, Any]) ->
↪  dp.Context:
    """A budget of either kind, with the privacy unit also bounded to one record per
    ↪  year and quarter."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(
            contributions=[
                dp.polars.Bound(by=[], per_group=CONTRIBUTIONS),  # at most 36 records
↪  overall
                dp.polars.Bound(by=["YEAR", "QUARTER"], per_group=1),  # and one in
↪  each year and quarter
            ]
        ),
        privacy_loss=privacy_loss,
        split_evenly_over=QUERIES,
    )
```

The query has no public keys, so its budget includes δ and the context thresholds group counts as described in Grouping. The script summarizes quarterly counts under three settings: zCDP with only the 36-contribution bound, zCDP with a per-group bound, and pure DP with that bound.

```
bounded = bounded_context(frame, dp.loss_of(rho=RHO, delta=DELTA))
count_by_quarter = bounded.query().group_by("YEAR", "QUARTER").agg(dp.len(signed=True))
grouped_summary = count_by_quarter.summarize(alpha=ALPHA)
```

```
privacy loss with a threshold: (Approximate(ZeroConcentratedDivergence), (0.1, 1e-07))
grouped count, 36 contributions: Integer Gaussian scale=180 95% accuracy=354
 ↪  threshold=1133
grouped count, 1 per year and quarter: Integer Gaussian scale=30 95% accuracy=60
 ↪  threshold=184
grouped count, 1 per year and quarter, pure DP: Integer Laplace scale=180 95%
 ↪  accuracy=539.731 threshold=3712
true groups: 36  smallest 82,504  largest 131,656
```

The per-group bound reduces the Gaussian scale from 180 to 30, a factor of six, and lowers the threshold. Under pure DP it changes nothing because the L_1 distance remains 36. Both thresholds are below the smallest quarterly count, so all groups are released; the Gaussian error bar is 60. The full edition's chapter on the two mechanisms derives Gaussian calibration from L_2 sensitivity.

When one person's records are spread across many groups, L_2 sensitivity can be much smaller than L_1 sensitivity, making Gaussian noise under zCDP preferable. This gain requires a per-group bound. If public information cannot justify it, enforce the bound through truncation.

7.5. Stating the guarantee in epsilon

A ρ budget is convenient for accounting but less familiar to release readers. OpenDP converts it to (ε, δ) with two combinators described in Combinators. `make_zCDP_to_approxDP` produces an ε-versus-δ privacy profile; `make_fix_delta` selects a δ from that profile. The script applies them to a Gaussian measurement calibrated so sensitivity 1 costs exactly ρ.

```
def epsilon_of_rho(rho: float, delta: float) -> float:
    """The epsilon that OpenDP's conversion gives for rho-zCDP at delta, read off a
     ↪  Gaussian measurement."""
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    gaussian = dp.m.make_gaussian(*space, scale=1 / math.sqrt(2 * rho))  # sensitivity
 ↪  1 at this scale is rho-zCDP
    approx = dp.c.make_fix_delta(dp.c.make_zCDP_to_approxDP(gaussian), delta)
    epsilon, _ = approx.map(1.0)
    return float(epsilon)
```

```
rho=0.1 at delta=1e-07: epsilon=2.3484
closed form rho + 2 sqrt(rho ln(1/delta)): 2.6391
rho=0.02 (one query) at delta=1e-07: epsilon=0.9951
the whole context converted: epsilon=2.3484 delta=1e-07
```

At $\delta = 10^{-7}$, the full $\rho = 0.1$ budget converts to $\varepsilon = 2.35$, and one query's share converts to about $\varepsilon = 1$. The textbook conversion is

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{7.1}$$

which gives 2.64 here. OpenDP's tighter conversion over Rényi-divergence orders is derived in the full edition's chapter on conversions. To convert a full context, apply the combinators to its

`accountant` attribute, the measurement that composes all queries. It also gives 2.35, the value to report with the release.

You choose δ. Keep it far below the reciprocal of the number of people; otherwise, the guarantee permits a mechanism that publishes one person's record. For 1.7 million people, 10^{-7} is defensible. Conversion δ is separate from the grouping-threshold δ, so a release using both must budget both.

The Gaussian count summary reports accuracy 354; the continuous conversion at the same scale gives 352.79. The context adds discrete Gaussian noise to integer counts, so it reports discrete accuracy, as Measurements explains. Use the discrete conversion to calculate count accuracy from scale.

7.6. Signed and unsigned counts

Every count here uses `signed=True` to avoid upward bias. Noise can make a small count negative; OpenDP previously clamped such results to zero. In version 0.16.0, `signed=True` casts the count to a signed integer before adding noise, so negative results remain. `signed=False` or an omitted argument clamps at zero; OpenDP warns that the default will change.

Clamping can reduce error for one small cell, but it biases sums of counts upward. A clamped count can exceed its true value but cannot fall below zero, so totals over many small cells accumulate positive bias. The OpenDP cookbook demonstrates this on 400 mostly empty cells: clamped totals are too high in every repetition, while signed totals are correct on average.

Clamp signed counts after release for display if needed; this is free post-processing. Release signed counts whenever others may sum, average or model them. The script illustrates the bias by releasing an empty frame 1,000 times with fixed Gaussian scale 30. It uses the framework API without a context and is intended only for toy data.

```
def mean_noisy_count_of_nobody(signed: bool, scale: float, repeats: int) -> float:
    """The mean of `repeats` noisy counts of an empty frame, at a fixed Gaussian scale,
     ↪  signed or clamped at zero."""
    empty = pl.LazyFrame(schema={"A": pl.Int32})
    domain = dp.with_margin(
        dp.lazyframe_domain([dp.series_domain("A", dp.atom_domain(T="i32"))]),
        dp.polars.Margin(by=[], max_length=1000),
    )
    count = dp.m.make_private_lazyframe(
        domain, dp.symmetric_distance(), dp.zero_concentrated_divergence(),
↪   empty.select(dp.len(signed=signed)), scale
    )
    return sum(int(count(empty).collect().item()) for _ in range(repeats)) / repeats
```

```
dp.len() without signed: warns=True clamped=True
expected clamped mean at scale 30: scale/sqrt(2 pi)=11.9683
standard error of the signed mean: scale/sqrt(1000)=0.9487
clamping experiment (one run): 1000 noisy counts of nobody at gaussian scale 30
  unsigned mean 11.48  signed mean 1.02  (true count 0)
```

The true count is zero. The signed mean is near zero and within the standard error of 1,000 draws. The unsigned mean is near 12, the expected positive half of Gaussian noise at scale 30. Repeated releases do not remove this bias; an unsigned table of empty cells carries the same upward bias in every cell.

7.7. Which noise to choose

Choose the mechanism based on the output and the guarantee you must report, as shown in Table 7.1.

Table 7.1.: Which noise to choose, by what is released.

Situation	Noise and budget	Why
One count, sum or mean, and a guarantee in pure ε	Laplace under `dp.loss_of(epsilon=...)`	The guarantee has no δ and is the easiest to state
A grouped table where a person touches many groups	Gaussian under `dp.loss_of(rho=..., delta=...)` with a `Bound` per group	The L_2 sensitivity grows with the root of the number of groups
Many queries composed into one release	Gaussian under zCDP	Budgets in ρ add up tightly and convert once at the end
A release that others will analyse assuming normal errors	Gaussian	The noise is normal, so standard intervals apply
A guarantee that must be reported as (ε, δ)	Either; convert ρ with `make_zCDP_to_approxDP` and `make_fix_delta`	OpenDP's conversion is tighter than the closed form
Counts that anyone might add up	Either, with `signed=True`	Clamping at zero biases totals upward

Gaussian noise cannot satisfy pure DP, so policies requiring $\delta = 0$ rule it out. The L_2 sensitivity gain also requires a known per-group contribution bound; that bound is part of the mechanism choice.

7.8. The releases of one run

The script releases counts under both budgets and a quarterly table under bounded zCDP. Its Laplace count repeats the Essential statistics result with the same error bar.

```
released (one run):
  laplace count 3,811,867  (true 3,811,932, within 539.7 with probability 0.95)
  gaussian count 3,811,994  (true 3,811,932, within 354.0 with probability 0.95)
  2005 Q1 count 87,442  (true 87,372, within 60)
  2005 Q2 count 86,457  (true 86,469, within 60)
```

```
2005 Q3 count 83,536  (true 83,518, within 60)
2005 Q4 count 84,746  (true 84,739, within 60)
groups released 36 of 36; within 60 of the truth: 32
```

Both overall counts fall within their error bars. Of the 36 quarterly counts, 32 are within their bars; 2005 Q1 is one of the misses. At $\alpha = 0.05$, about one cell in twenty falls outside its bar, and the missed cell is not known in advance. For private data, choose from scales and accuracies, which cost nothing to inspect, then release once.

7.9. Summary

Table 7.2.: The calls and concepts of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`summarize(alpha=0.05)`	The scale and the 95% accuracy of each statistic, before release	Read it before `release()`; it costs nothing
`laplacian_scale_to_accuracy`, `gaussian_scale_to_accuracy`	A continuous scale to a 95% error bar	For sums and means of floats
`discrete_laplacian_scale_to_accuracy`, `discrete_gaussian_scale_to_accuracy`	The same for integer noise	For counts, to match `summarize`
`accuracy_to_laplacian_scale` and the three others	A target error bar to the scale it needs	Divide the sensitivity by the scale to see the ε it costs
The error bar	Covers the noise only	Clipping, imputation, truncation and resizing bias the input, and the bar does not see it
`dp.loss_of(epsilon=...)`	Pure DP; the context adds Laplace noise	Halving the bar doubles ε
`dp.loss_of(rho=...)`	zCDP; the context adds Gaussian noise	Halving the bar quadruples ρ
`dp.polars.Bound(by=[...], per_group=1)`	One record per person per group	Cuts the L_2 sensitivity to the root of the number of groups; does nothing for Laplace
`make_zCDP_to_approxDP` then `make_fix_delta`	ρ to (ε, δ), tighter than the closed form	Apply to `context.accountant` for the whole release
`signed=True`	Keeps negative noisy counts	Always, if the counts may be summed; clamp for display afterwards

7.10. References

- OpenDP documentation: "Assessing Utility" in Getting Started, "Gaussian Noise" in the Polars user guide, and "Aggregation" in the Polars expressions guide for `signed`. The OpenDP Cookbook's "Signed Counts: Why Clipping at Zero Biases Totals" is the experiment the signed-counts section summarizes.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP, the Gaussian mechanism under it, and the conversion to (ε, δ).
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the integer Gaussian that the context adds to counts and its tail bounds.
- Kamath, Mouzakis, Regehr, Singhal and Steinke, "A Bias-Variance-Privacy Trilemma for Statistical Estimation" (2023): why the bias of clipping cannot be removed in general, cited by the documentation's note on accuracy.
- The full edition's chapters on the Laplace and Gaussian mechanisms in the full edition, concentrated DP in the full edition and conversions to (ε, δ) in the full edition derive the results this chapter states.

Part III.

The framework

The Context API queries in the previous parts use OpenDP framework components. A private computation chains transformations and measurements. A transformation maps data to data, and its stability bounds how much the output changes when the input changes. A measurement maps data to a random release, and its privacy map bounds changes to the release distribution. These bounds propagate through a chain, while composed measurements accumulate privacy loss. This framework lets you inspect guarantees, build queries unavailable through the Context API and verify claims.

The first chapter defines domains, metrics, measures and the maps that describe stability and privacy. The next three cover transformations, measurements that add noise or select, and combinators for chaining and composition. The final chapter covers supporting utilities and shows how the Context API uses these components.

Table 7.3.: The chapters of this part and the questions they answer.

Chapter	Question it answers
A framework to understand differential privacy	What are domains, metrics, measures, transformations and measurements, and what does each promise?
Transformations	How do I build and chain the data-to-data steps, and how does stability flow through them?
Measurements	Which mechanisms does OpenDP offer, and how do I calibrate each one?
Combinators	How do I compose measurements, convert between privacy measures, and account adaptively?
Utilities and the Context API inside	How do types, parameter search and serialization work, and what does a context do under the hood?

By the end of this part, you can build private queries from framework components, inspect guarantees with `map` and `check`, and describe how the Context API resolves a query.

8. A framework to understand differential privacy

The first two parts used the Context API to choose noise scales, spend budget and reject queries that exceed it. The framework beneath the context has two main objects. A transformation maps data to data and bounds how input distance affects output distance. A measurement maps data to a random release and bounds how input distance affects the output distributions. Each bound is a map from input distance to output distance; OpenDP uses these maps to track differential privacy (DP). This chapter shows how an object's fields and map state its guarantee.

The script opendp_framework.py runs each example on OpenDP 0.16.0. Its sum example uses integer values from the Labour Force Survey (LFS) column `HWUSUAL`; its workflow uses the hundred toy scores from the first chapter. Domains, metrics, measures, maps, scales and error bars are deterministic. Released values vary because OpenDP's noise cannot be seeded, so the page labels them as one run.

```
uv run python code/part03-framework/opendp_framework.py
```

8.1. Two kinds of object

The first chapter built a Laplace measurement with scale 1. The documentation repeats the construction at scale 5 using `make_laplace`, which takes the domain and metric as its first two arguments rather than attaching them with `>>`.

```
def laplace_measurement(scale: float) -> dp.Measurement:
    """Laplace noise of the given scale on one finite float, compared by absolute
     ↪ distance, under pure DP."""
    return dp.m.make_laplace(
        dp.atom_domain(T=float, nan=False),  # input domain: any float that is not NaN
        dp.absolute_distance(T=float),  # input metric: |a - b|, the units of the
 ↪ sensitivity
        scale=scale,
    )
```

The constructor returns a `Measurement`; the script prints its fields.

```
laplace scale 5: type=Measurement
laplace scale 5: input domain=AtomDomain(T=f64)
laplace scale 5: input metric=AbsoluteDistance(f64)
laplace scale 5: output measure=MaxDivergence
laplace scale 5: function=Function
```

Transformations and measurements share four fields. `function` holds the computation, which maps `input_domain` to `output_domain`; `input_metric` measures input distance. Their output fields differ because a **transformation** is deterministic and a **measurement** is random. A transformation returns data, so `output_metric` and the **stability map** describe its output distance as a function of input distance. A measurement returns a random draw, so `output_measure` and the **privacy map** describe the distance between output distributions.

The Laplace measurement accepts a finite float, measures input differences by absolute distance and compares output distributions by max divergence, bounded by ε. Both object guarantees have the same form: a transformation is (`d_in`, `d_out`)-stable if inputs at most `d_in` apart produce outputs at most `d_out` apart; a measurement is (`d_in`, `d_out`)-differentially private if such inputs produce output distributions at most `d_out` apart.

8.2. Functions

Call a transformation or measurement like a function; `invoke` does the same. The clamp below clips each finite float in a vector to the interval [0, 5], under symmetric distance.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = space >> dp.t.then_clamp(bounds=(0.0, 5.0))
print(type(clamp).__name__)  # Transformation
print(clamp([1.3, 7.8, -2.5, 7.0]))  # [1.3, 5.0, 0.0, 5.0]
print(clamp.invoke([10.0]))  # [5.0]
print(type(clamp.function).__name__)  # Function
```

The `function` field stores the computation as an object; the next part's plugin chapter uses it to wrap a Python function. A transformation adds no noise and does not make its output private. It prepares data within the guarantee; only a measurement can release data. Printing a transformation's output is outside that guarantee.

8.3. Domains

A domain is a set of values. A map's proof applies only to inputs in that domain, so the domain states its assumptions. When chaining objects, OpenDP checks that the first object's output domain matches the next object's input domain. The common `atom_domain` contains one value of type `T`; `vector_domain` lifts it to vectors.

```
import opendp.prelude as dp

floats = dp.atom_domain(T=float, nan=False)
print(floats)  # AtomDomain(T=f64)
print(floats.member(1.0), floats.member(float("nan")))  # True False
print(dp.atom_domain(T=float))  # AtomDomain(nan=true, T=f64)
```

```
small = dp.atom_domain(bounds=(-2, 2))
print(small)  # AtomDomain(bounds=[-2, 2], T=i32)
print(small.member(-2), small.member(3))  # True False
pairs = dp.vector_domain(dp.atom_domain(T=bool), size=2)
print(pairs)  # VectorDomain(AtomDomain(T=bool), size=2)
print(pairs.member([True, True]), pairs.member([True, True, True]))  # True False
```

`member` tests whether a value belongs to a domain. Type `T` uses Python names, so `float` prints as `f64` and `int` as `i32`. The `nan` flag controls whether floats may be NaN; version 0.16.0 prints it only when true. Thus `atom_domain(T=float)` allows NaN, while `atom_domain(T=float, nan=False)` rejects it. A noise measurement needs the latter because absolute distance from NaN is undefined.

`bounds` restrict values to an interval. A sum needs bounds to have finite sensitivity. A vector domain can also specify `size`, which a mean needs, as the first chapter's resize demonstrated.

8.4. Metrics

A metric defines distance between members of a domain. The framework uses it to interpret `d_in`; it does not compute the distance. Under **symmetric distance**, two datasets are separated by the number of additions and removals needed to transform one into the other, counting duplicates. The function below computes this distance for two small vectors from the documentation's notebook.

```
from collections import Counter

import opendp.prelude as dp


def symmetric_distance(u: list[int], v: list[int]) -> int:
    """The number of additions and removals that turn the multiset u into v."""
    cu, cv = Counter(u), Counter(v)
    return sum(((cu - cv) + (cv - cu)).values())


print(symmetric_distance([12, 10, 8, 7], [10, 8, 7]))  # 1
print(symmetric_distance([12, 10, 8, 7], [10, 10, 8, 7]))  # 2
print(dp.symmetric_distance(), dp.absolute_distance(T=float))  # SymmetricDistance()
 ↪  AbsoluteDistance(f64)
print(dp.unit_of(contributions=1, ordered=True))  # (InsertDeleteDistance(), 1)
print(dp.unit_of(changes=1))  # (ChangeOneDistance(), 1)
```

Removing one record gives distance 1. Replacing 12 with 10 gives distance 2: one removal and one addition. Under symmetric distance, `d_in` is the maximum records one person can add or remove, which defines the privacy unit. Datasets D and D' are **neighbours** when their input-metric distance is at most `d_in`. This is the metric behind `dp.unit_of(contributions=k)`. In the LFS, one person contributes at most 36 rows (the privacy unit), so `d_in=36`.

Absolute distance is the absolute difference between two numbers. Under this metric, `d_in` is a **sensitivity**: the maximum change in an aggregate when one person's records change. Sensitivity

usually appears in the middle of a chain as an aggregate's output distance, rather than as an input you specify. The full edition's chapter on sensitivity in the full edition derives common bounds.

Other dataset metrics modify symmetric distance. Insert-delete distance also protects record order. Change-one distance counts replacements in fixed-size datasets, the replace-one relation of bounded DP. Hamming distance is the ordered version of change-one.

8.5. Measures

A measure defines distance between probability distributions and gives a measurement's `d_out` its privacy meaning. **Max divergence** is the largest log ratio between the probabilities of an output set under the two distributions. A bound of ε gives pure differential privacy, defined in the full edition's chapter on the definition. OpenDP provides four measures, each with a different form for `d_out`.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.max_divergence(), dp.zero_concentrated_divergence())  # MaxDivergence
 ↪  ZeroConcentratedDivergence
print(dp.smoothed_max_divergence(), dp.fixed_smoothed_max_divergence())  #
 ↪  SmoothedMaxDivergence Approximate(MaxDivergence)
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
gaussian = space >> dp.m.then_gaussian(scale=1.0)
print(gaussian.output_measure, gaussian.map(1.0))  # ZeroConcentratedDivergence 0.5
profile = dp.c.make_zCDP_to_approxDP(gaussian).map(1.0)
print(f"{profile.epsilon(delta=1e-7):.4f}")  # 5.6709
print(dp.c.make_fix_delta(dp.c.make_zCDP_to_approxDP(gaussian), delta=1e-7).map(1.0))
 ↪  # (5.6708588354687715, 1e-07)
```

Under max divergence, `d_out` is ε. Under **zero-concentrated divergence**, it is ρ, the zCDP parameter that bounds a family of Rényi divergences and is the Gaussian mechanism's native measure. At sensitivity 1 and scale 1, a Gaussian mechanism costs $\rho = 0.5$, equal to sensitivity squared divided by twice the scale squared.

The two smoothed measures express approximate DP, whose guarantee allows slack δ. Under smoothed max divergence, `d_out` is a **privacy profile**, a curve giving the smallest ε for each δ; the example evaluates it at 10^{-7}. Fixed smoothed max divergence, printed as `Approximate(MaxDivergence)`, gives one (ε, δ) pair, as in the first chapter's `dp.loss_of(epsilon=1.0, delta=1e-7)`. `make_fix_delta` converts a profile to this pair. Combinators explains the conversions. Use `output_measure` to interpret a measurement's `d_out`.

8.6. Stability maps and privacy maps

A map states the guarantee by converting input distance to output distance. For a measurement, `map(d_in)` returns the smallest provable `d_out`, so the measurement is (`d_in`, `d_out`)-differentially

private. For a transformation, it returns the smallest `d_out` that makes it (`d_in`, `d_out`)-stable. `check` tests whether a proposed `d_out` meets the guarantee. The script evaluates a Laplace map at two sensitivities.

```
epsilon_at_1 = laplace.map(d_in=1.0)  # the sensitivity over the scale: 1 / 5
epsilon_at_10 = laplace.map(d_in=10.0)
fits = laplace.check(d_in=1.0, d_out=0.2)  # is map(1.0) <= 0.2?
too_tight = laplace.check(d_in=1.0, d_out=0.19)
```

```
laplace scale 5: map(1.0)=0.2 map(10.0)=2.0
laplace scale 5: check(1.0, 0.2)=True check(1.0, 0.19)=False
```

The Laplace privacy map divides sensitivity by scale. Sensitivity 1 at scale 5 costs $\varepsilon = 0.2$; ten times the sensitivity costs ten times more. Thus `check` passes at 0.2 and fails at 0.19. The Context API searches for a scale by testing whether `check` passes at the query's budget share. The full edition derives the map in its chapter on the Laplace and Gaussian mechanisms in the full edition.

A map returns a provable upper bound, which may exceed the true value. The sum below illustrates this difference.

```
import opendp.prelude as dp

dp.enable_features("contrib")
float_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0.0, 5.0))),
 ↪  dp.symmetric_distance())
int_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0, 5))),
 ↪  dp.symmetric_distance())
print(float_sum([1.3, 3.8, 0.0, 5.0]))  # 10.1
print(float_sum.map(d_in=2))  # 10.00000004656613
print(int_sum.map(d_in=2))  # 10
print(int_sum.check(d_in=2, d_out=10), float_sum.check(d_in=2, d_out=10.0))  # True
 ↪  False
```

Two records bounded by 5 can change the sum by 10, and the integer sum's map returns exactly 10. Floating-point addition rounds, so its proof includes a small extra margin. Because `check` uses strict comparison, the float sum fails at exactly 10. Use the map's result rather than entering the theoretical value yourself.

8.7. Chaining a transformation and a measurement

When a transformation's output domain and metric match a measurement's input domain and metric, `>>` chains them into a measurement. It applies the transformation first, then the measurement; the combined privacy map composes their maps. The script chains clamping, summation and Laplace noise for `HWUSUAL`, usual weekly hours. After removing nulls and code 99 ("not applicable"), the data contain 1,496,192 integer values from 0 to 80.

```
clamp = dp.t.make_clamp(dp.vector_domain(dp.atom_domain(T=int)),
 ↪  dp.symmetric_distance(), bounds=HOURS_BOUNDS)
bounded_sum = dp.t.make_sum(clamp.output_domain, clamp.output_metric)  # the clamp's
 ↪  output space is its input space
laplace_int = dp.m.make_laplace(bounded_sum.output_domain, bounded_sum.output_metric,
 ↪  scale=LAPLACE_SCALE)
noisy_sum = clamp >> bounded_sum >> laplace_int  # the chain: its input space is the
 ↪  clamp's, its measure Laplace's
sum_sensitivity_1 = bounded_sum.map(d_in=1)  # one row of at most 80 hours
sum_sensitivity_36 = bounded_sum.map(d_in=CONTRIBUTIONS)  # 36 rows
epsilon_at_scale_5 = noisy_sum.map(d_in=CONTRIBUTIONS)
```

Each constructor uses the preceding object's output space as its input, so domains match by construction. The clamp outputs integers from 0 to 80, which the sum accepts. The script prints the fields and maps along the chain.

```
clamp: input domain=VectorDomain(AtomDomain(T=i32))
clamp: output domain=VectorDomain(AtomDomain(bounds=[0, 80], T=i32))  map(36)=36
sum: output domain=AtomDomain(T=i32)  output metric=AbsoluteDistance(i32)
sum: map(1)=80 map(36)=2880
chain scale 5: input domain=VectorDomain(AtomDomain(T=i32))
chain scale 5: input metric=SymmetricDistance()  output measure=MaxDivergence
chain scale 5: map(36)=576.0
```

Read the maps from left to right. Clamping is 1-stable because it changes values without adding or removing records; 36 input records produce 36 output records. The sum changes by at most 80 hours per record, so 36 records give sensitivity 2,880. Laplace noise at scale 5 then gives $\varepsilon = 576$, an inadequate guarantee. The map exposes this problem before release; scale 5 was chosen for one float.

```
sum_scale = dp.binary_search_param(
    lambda scale: clamp >> bounded_sum >> dp.m.then_laplace(scale), d_in=CONTRIBUTIONS,
     ↪  d_out=EPSILON
)
noisy_sum = clamp >> bounded_sum >> dp.m.then_laplace(sum_scale)
sum_accuracy = dp.discrete_laplacian_scale_to_accuracy(scale=sum_scale, alpha=0.05)
released_sum = noisy_sum(hours)
```

```
chain for epsilon 1 at d_in 36: scale=2880.0 map(36)=1.0
chain for epsilon 1 at d_in 36: 95% accuracy=8,628.2 (0.01532% of the sum)
```

`binary_search_param` finds the smallest scale whose chained measurement passes `check`; here it returns 2,880, sensitivity divided by $\varepsilon = 1$. The chain uses `then_laplace`, whose input space comes from the preceding object. The integer sum receives discrete Laplace noise and uses discrete accuracy. An error of about 8,600 hours is small relative to 56 million. Transformations and Measurements describe the other maps.

8.8. The typical workflow in both APIs

The Context API wraps the framework. The script implements the first chapter's five steps in both APIs using the same seed and hundred toy scores from A first release, allowing a direct scale comparison. The framework represents the privacy unit as an input distance and metric, and the budget as an output distance and measure.

```
d_in = 1  # neighbouring datasets differ by at most one row...
input_metric = dp.symmetric_distance()  # ...counted as additions and removals
input_domain = dp.vector_domain(dp.atom_domain(T=float))  # a vector of floats, NaN
 ↪  allowed
d_out = EPSILON  # the distributions of two releases differ by at most epsilon...
privacy_measure = dp.max_divergence()  # ...measured by the max divergence, which is
 ↪  pure DP
```

```
unit: d_in=1 input metric=SymmetricDistance()  (Context API: (SymmetricDistance(), 1))
loss: d_out=1.0 output measure=MaxDivergence  (Context API: (MaxDivergence, 1.0))
input domain: VectorDomain(AtomDomain(nan=true, T=f64))
```

These pairs match the first chapter's `dp.unit_of` and `dp.loss_of` outputs. The framework also requires an input domain. The context inferred a vector of floats from the list, which allows NaN. Both APIs use the same public bounds and imputed value. The fourth step mediates access and exposes the context's components.

```
compositor = dp.c.make_adaptive_composition(
    input_domain=input_domain,
    input_metric=input_metric,
    output_measure=privacy_measure,
    d_in=d_in,
    d_mids=[d_out / QUERIES] * QUERIES,  # the shares of the budget, one per query
)
queryable = compositor(data)  # a measurement; invoking it on the data gives the
 ↪  queryable that answers queries
```

```
compositor: type=Measurement map(1)=1.0
queryable: type=Queryable
context accountant: type=Measurement d_mids=[0.3333, 0.3333, 0.3333]
```

An **adaptive composition** is a measurement whose privacy map sums the shares, because sequential privacy budgets add (the full edition's chapter on properties). Invoking it on data releases nothing; it returns a **queryable** that holds the data and answers submitted measurements, one share each. The composition is adaptive because each measurement can depend on earlier answers. A context's `accountant` field stores the same composition and shares. The fifth step submits manually built queries.

```
count = dp.t.make_count(input_domain, input_metric)
count_sensitivity = count.map(d_in)  # adding or removing one row changes a count by 1
count_scale = dp.binary_search_param(
    lambda scale: count >> dp.m.then_laplace(scale), d_in=d_in, d_out=d_out / QUERIES
)
count_measurement = count >> dp.m.then_laplace(count_scale)
dp_count = queryable(count_measurement)
```

```
count: sensitivity=1  output metric=AbsoluteDistance(i32)
count at scale 3.0: map(1)=0.3333333333333337 check(1, 1/3)=False
count by search: scale=3.0000000000000004 check(1, 1/3)=True
count scale: framework=3.0000 context=3.0000 equal=True
count: 95% accuracy=9.4457
```

The count's stability map gives sensitivity 1, so theory predicts scale $1/(\varepsilon/3) = 3$. However, `check` rejects typed scale 3.0 because floating-point 1/3 is one rounding step above the budget share. The search returns the next representable scale above 3, which passes. It matches `context.query().count().laplace().param()` bit for bit because the context uses the same search. The queryable spends the first share on the count; the mean uses that released count as its size.

```
mean = (
    dp.t.make_impute_constant(input_domain, input_metric, IMPUTED)  # NaN becomes the
 ↪  public midpoint
    >> dp.t.then_clamp(BOUNDS)
    >> dp.t.then_resize(size=dp_count, constant=IMPUTED)  # the released count fixes
     ↪  the size
    >> dp.t.then_mean()
)
mean_scale = dp.binary_search_param(
    lambda scale: mean >> dp.m.then_laplace(scale), d_in=d_in, d_out=d_out / QUERIES
)
dp_mean = queryable(mean >> dp.m.then_laplace(mean_scale))
```

```
mean: output domain=AtomDomain(T=f64)  output metric=AbsoluteDistance(f64)
mean at size 100: sensitivity=1.0000
mean scale at size 100: framework=3.0000 context=3.0000 equal=True
mean at size 100: 95% accuracy=8.9872
mean scale at the released count: equal=True
```

The four transformations form one chain whose stability composes their maps. At the true size of 100, one row changes the mean by at most 1, giving scale 3, as in the context. The page reports this scale because the scale at the noisy released count varies by run. A third query uses the final share; the queryable rejects a fourth.

```
try:
    queryable(count_measurement)
    refusal = "released"
except dp.OpenDPException as err:
    refusal = f"{err.variant}: {err.message}"
```

```
fourth query: FailedFunction: out of queries
```

The queryable raises OpenDP's exception; the context wraps the same refusal in a `ValueError`. It also rejects a query whose map exceeds its share before accessing data. A context therefore consists of three objects: a space, an adaptive composition over it and a queryable. Utilities and the Context API inside shows how a query uses them. The next block reports one run.

```
released (one run):
  laplace scale 5 on 0.0: 12.84  (within 15.0 with probability 0.95)
  hours sum 56,316,146  (true 56,315,594, within 8,628 with probability 0.95)
  count 99  (true 100, within 9.4 with probability 0.95)
  mean scale 3.0303  mean 50.43  (true 50.93)
  third query, a second count 104
```

Each release falls within its error bar. The LFS sample and toy scores are public, so their outputs can be compared with the truth. For private data, inspect error bars, which incur no cost, and release once.

8.9. Summary

Table 8.1.: The objects and elements of the framework, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`Transformation`	A deterministic map from data to data, with an `output_metric` and a stability map	Its output is not private; chain it with a measurement before anything leaves
`Measurement`	A randomized map from data to a release, with an `output_measure` and a privacy map	Its `map(d_in)` is the guarantee; read it before releasing
`trans(x)`, `trans.invoke(x)`	Evaluate the `function` field	Printing a transformation's output steps outside the guarantee
`atom_domain(T, bounds, nan)`	One value of type `T`, optionally bounded, with or without NaN	Noise needs `nan=False`; a sum needs `bounds`; the printed form shows `nan` only when true
`vector_domain(domain, size)`	Vectors of a domain, optionally of fixed length	A mean needs the `size`; `member` tests a value by hand
`symmetric_distance()`	Additions and removals, with multiplicity	`d_in` is the records one person can add or remove

Call or concept	What it does	Rule of thumb
`absolute_distance(T)`	The difference of two numbers	`d_in` is a sensitivity; it appears midway through a chain
`insert_delete`, `change_one`, `hamming`	Ordered add/remove, replace-one, ordered replace-one	Pick the one that matches how your data can change
`max_divergence()`	Pure differential privacy; `d_out` is ε	The measure behind `loss_of(epsilon=...)`
`zero_concentrated_divergence()`	zCDP; `d_out` is ρ	The Gaussian mechanism's native measure
`smoothed_max_divergence()`, fixed	Approximate DP; `d_out` is a privacy profile, or one (ε, δ) pair	`make_fix_delta` turns the curve into the pair
`obj.map(d_in)`, `obj.check(d_in, d_out)`	The smallest provable `d_out`; whether a proposed one fits	Let the map produce the distance; a hand-typed 3.0 fails by one rounding step
`trans >> meas`	A measurement whose map composes the two maps	Domains and metrics must match; `then_` constructors match them for you
`binary_search_param`	The smallest scale whose chain passes `check`	What the Context API runs for `param()`
`make_adaptive_composition`	A measurement whose invocation returns a queryable	A context is this, over a space, plus the queryable

8.10. References

- OpenDP documentation: "A Framework to Understand DP", "Differential Privacy with OpenDP", "Core Structures" and "Supporting Elements" in the Framework API user guide, and "API Comparison".
- Gaboardi, Hay and Vadhan, "A Programming Framework for OpenDP" (2020): the framework of domains, metrics, measures, stability maps and privacy maps that the library implements.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): sensitivity and the Laplace mechanism.
- The full edition's chapters on the definition in the full edition, post-processing and composition in the full edition, the Laplace and Gaussian mechanisms in the full edition and sensitivity in the full edition derive the results this chapter states.

9. Transformations

Previous queries transformed data by casting, imputing, clamping, resizing and aggregating before adding noise. In OpenDP, each data-to-data step is a **transformation**. Its stability map bounds output changes for a given input change (A framework to understand differential privacy). This chapter builds transformations for preprocessing, sums, means and quantile scores, and composes their stability maps.

A chain requires each step's output space to match the next step's input space. Its stability map composes the maps of its steps. You will build sums, means and median scores from framework components, compute sensitivity for a privacy unit and diagnose space-mismatch errors.

The script opendp_transformations.py runs on OpenDP 0.16.0 and the Labour Force Survey (LFS) sample. Its vector contains integer `HWUSUAL` hours, excluding nulls and code 99 ("not applicable"). Sensitivities, scales, accuracies, scores and true values are deterministic. Noisy releases vary because OpenDP's noise cannot be seeded; a float sum also varies because it samples its input. The page labels these outputs as one run.

```
uv run python code/part03-framework/opendp_transformations.py
```

9.1. Constructors and chains

Each constructor has two forms. The `make_` form takes an input space (domain and metric) followed by its arguments, as in `make_clamp(input_domain, input_metric, bounds)`. The `then_` form omits the space and returns a partial constructor. The `>>` operator supplies the space from the object on its left, either a space tuple or transformation. Both forms build the same object.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
clamp = dp.t.make_clamp(*space, bounds=(0, 10))
print(clamp([-5, 3, 50]))  # [0, 3, 10]
print(clamp.output_space)  # (VectorDomain(AtomDomain(bounds=[0, 10], T=i32)),
 ↪  SymmetricDistance())
total = dp.t.make_sum(*clamp.output_space)
chain = space >> dp.t.then_clamp(bounds=(0, 10)) >> dp.t.then_sum()
print(chain([-5, 3, 50]), (clamp >> total)([-5, 3, 50]))  # 13 13
print(chain.output_space)  # (AtomDomain(T=i32), AbsoluteDistance(i32))
print(clamp.map(1), total.map(1), chain.map(1))  # 1 10 10
```

The clamp changes the domain but preserves the metric. Its output is a vector of integers from 0 to 10 under symmetric distance, the number of row additions or removals between vectors. The sum takes the clamp's output space as input, as `make_sum(*clamp.output_space)` shows, and returns an integer under absolute distance. Check `output_space` after each step; the next constructor must accept it (Section 9.6).

`map` returns **stability**, the largest output change for inputs at most `d_in` apart. Clamp stability is 1 because adding one row adds one output row. Sum stability is 10, the larger value bound. The chain composes these maps. When a chain ends with an aggregator, its stability is the query's sensitivity, which determines the measurement's noise. The full edition's chapter on sensitivity derives sensitivities for sums, means and histograms.

9.2. Preprocessing

Preprocessing converts data to an aggregator's input domain. A sum needs bounded values; a mean also needs a known size. This section follows the documentation's constructor order: cast, impute, clamp and resize. All except resize operate row by row, so one added input row produces one output row and stability is 1.

9.2.1. Casting and imputing

File data often arrive as strings. `make_cast` converts them to another type; unparseable strings become missing values, represented as options. Aggregators accept neither options nor silently skipped values: skipping them would understate sensitivity, while retaining them would reveal missingness. Resolve missing values first with `make_impute_constant` or `make_drop_null`. `make_cast_default` instead casts and imputes the type's default, such as 0 for numbers.

```
import opendp.prelude as dp

dp.enable_features("contrib")
strings = dp.vector_domain(dp.atom_domain(T=str)), dp.symmetric_distance()
cast = strings >> dp.t.then_cast(TOA=int)
print(cast.output_domain)  # VectorDomain(OptionDomain(AtomDomain(T=i32)))
imputed = cast >> dp.t.then_impute_constant(0)
print(imputed(["1", "2", "x"]))  # [1, 2, 0]
print(imputed.output_domain)  # VectorDomain(AtomDomain(T=i32))
by_default = strings >> dp.t.then_cast_default(TOA=int)
print(by_default(["1", "2", "x"]))  # [1, 2, 0]
floats = dp.vector_domain(dp.atom_domain(T=float)), dp.symmetric_distance()
print(floats[0])  # VectorDomain(AtomDomain(nan=true, T=f64))
uniform = floats >> dp.t.then_impute_uniform_float(bounds=(0.0, 10.0))
print(uniform.output_domain)  # VectorDomain(AtomDomain(T=f64))
print(cast.map(1), imputed.map(1), uniform.map(1))  # 1 1 1
```

Floats use NaN as a missing value; `atom_domain(T=float)` allows it, as `nan=true` shows. `make_impute_constant` accepts this domain. `make_impute_uniform_float` replaces each NaN with a uniform draw from the supplied bounds, avoiding concentration at one value. Both produce

a NaN-free domain with stability 1. Preparing microdata applies these steps to a table through Polars.

9.2.2. Clamping and resizing

Clamping moves values into public bounds. It preserves values already within bounds and makes sum sensitivity finite. Set bounds from public knowledge, such as a codebook; estimating them from private data would leak information.

Resize has stability 2. It pads short vectors with a constant and reduces long vectors by random sampling. Means and variances require a known row count, which may be private. Resize records a target size in the domain, so OpenDP treats it as public, whether it is a guess, released count or public fact. Adding one row to a short input adds that row and removes one padded row, giving stability 2.

```
import opendp.prelude as dp

dp.enable_features("contrib")
floats = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = floats >> dp.t.then_clamp(bounds=(0.0, 10.0))
print(clamp([-5.0, 3.0, 50.0]), clamp.map(1))  # [0.0, 3.0, 10.0] 1
resize = clamp >> dp.t.then_resize(size=5, constant=5.0)
print(resize([1.0, 2.0, 3.0]))  # [1.0, 2.0, 3.0, 5.0, 5.0]
print(resize.output_domain)  # VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64),
 ↪  size=5)
print(resize.map(1))  # 2
```

A wrong target biases the result, as the documentation's resize page shows. A target below the true size throws rows away, which widens the spread of the mean, and a target above it pulls the mean toward the constant. No error bar shows either error, because the size is treated as true, and the resize cannot warn you without leaking the true size. So spend a little budget on a count first, which lands close to the truth because its sensitivity is 1.

9.2.3. Categories and bins

Three constructors map between categorical values and indices. The first, `make_find`, replaces each value by its position in a list of categories. A value outside the list becomes a missing index, so an imputation follows and gives it the next index. The step `make_find_bin` turns a number into the index of its bin, from the bin edges, and `make_index` turns an index back into a label.

```
import opendp.prelude as dp

dp.enable_features("contrib")
strings = dp.vector_domain(dp.atom_domain(T=str)), dp.symmetric_distance()
find = strings >> dp.t.then_find(categories=["A", "B", "C"]) >>
 ↪  dp.t.then_impute_constant(3)
print(find(["A", "B", "C", "A", "D"]))  # [0, 1, 2, 0, 3]
floats = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
```

```
bins = floats >> dp.t.then_find_bin(edges=[1.0, 2.0, 10.0])
print(bins([0.0, 1.0, 3.0, 15.0]))  # [0, 1, 2, 3]
label = bins >> dp.t.then_index(categories=["low", "mid", "high"], null="other")
print(label([0.0, 1.0, 3.0, 15.0]))  # ['low', 'mid', 'high', 'other']
counts = strings >> dp.t.then_count_by_categories(categories=["A", "B", "C"])
print(counts(["A", "B", "C", "A", "D"]))  # [2, 1, 1, 1]
print(counts.output_metric, counts.map(1))  # L1Distance(i32) 1
```

The aggregator for categories is `make_count_by_categories`, which counts each category and adds a count of all other values unless you set `null_category=False`. Its output is a vector of counts, so its output metric is a vector norm. The default L_1 distance goes with Laplace noise, and the L_2 distance, through the `MO` argument, goes with Gaussian noise. One added row changes one count by one, so the stability is 1 in either norm.

9.3. The sum

The sum is the base of the other aggregators, and its documentation page shows where sensitivity comes from. Start with integers in public bounds and no known size.

```
import opendp.prelude as dp

dp.enable_features("contrib")
bounded = dp.vector_domain(dp.atom_domain(bounds=(0, 10))), dp.symmetric_distance()
total = bounded >> dp.t.then_sum()
print(total([1, 2, 4]), total.map(1))  # 7 10
sized = dp.vector_domain(dp.atom_domain(bounds=(-10, 10)), size=3),
 ↪  dp.symmetric_distance()
sized_total = sized >> dp.t.then_sum()
print([sized_total.map(d) for d in range(5)])  # [0, 0, 20, 20, 40]
```

One added row changes the sum by at most the larger bound in absolute value, here 10, which is `map(1)`. When the size is public, you write it into the domain, and the bound is tighter. Two datasets of the same size differ at least by one replaced row, which is one removal and one addition, a symmetric distance of 2. A replacement changes the sum by at most the width of the bounds, which is `map(2)`. Odd distances round down, so `map(1)` is 0 and the last line is a staircase.

On the survey, the script builds the vector once with Polars, and the chain clamps the hours into the codebook's bounds, 0 to 98, and sums them.

```
def hours_usually_worked(frame: pl.LazyFrame) -> list[int]:
    """The `HWUSUAL` column without nulls and without the code 99, as a list of
     ↪  ints."""
    kept = frame.filter(pl.col("HWUSUAL").is_not_null() & (pl.col("HWUSUAL") != 99))
    return kept.select(pl.col("HWUSUAL").cast(pl.Int64)).collect()["HWUSUAL"].to_list()
```

```
def int_sum(size: int | None = None) -> dp.Transformation:
    """Clamp to the public bounds, then sum; with a size the domain says how many rows
     ↪  there are."""
    space = dp.vector_domain(dp.atom_domain(T=int), size=size), dp.symmetric_distance()
    return space >> dp.t.then_clamp(BOUNDS) >> dp.t.then_sum()
```

```
hours vector: 1,496,192 values
true sum: 56,315,594  true mean: 37.6393  true median: 37
int sum: value 56,315,594  map(1)=98  map(36)=3528
sized int sum: map(36)=1764  map(35)=1666
```

Every map on this page is read at an input distance of 36, the most rows one person has (the privacy unit). Without a size, the sensitivity is 36 rows of 98 hours, which is 3,528 hours. With the size, it is half that, 18 replacements, and a distance of 35 rounds down to 17. But the Polars filter runs outside OpenDP, so the guarantee does not cover it. Preparing microdata runs it inside the guarantee, and Essential statistics releases this sum through the Context API.

Floats are different, for two reasons from the documentation's sum page. The first is rounding: each partial sum of floats rounds to the nearest float. So the sums of two datasets at distance 2 can differ by a little more than the width of the bounds, and the stability map adds a term for that.

```
import opendp.prelude as dp

dp.enable_features("contrib")
metric = dp.symmetric_distance()
sized = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 10.0)), size=1000), metric
print((sized >> dp.t.then_sum()).map(2))  # 20.00000000004426
unsized = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 10.0))), metric
print((unsized >> dp.t.then_sum()).map(1))  # 20.00000009313226
negative = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 0.0))), metric
print((negative >> dp.t.then_sum()).map(1))  # 10.00000009313226
print(dp.t.make_bounded_float_checked_sum(size_limit=100, bounds=(-10.0, 0.0)).map(1))
 ↪  # 10.00000000000295
```

With a known size of a thousand rows, the term is in the eleventh decimal place. The second reason applies when the size is unknown. Then the rounding error has no bound, so the float sum first keeps at most 2^{20} rows, a little over a million, by a random sample. A neighbour with one more row may then drop a different row, so the sensitivity becomes the largest of the magnitudes of the bounds and their width. That doubles it for bounds symmetric about zero (the second line) but not for bounds of one sign (the third). With `make_bounded_float_checked_sum` you set the size limit, and a limit of 100 shrinks the rounding term to the twelfth decimal place.

This limit matters on the survey, because the hours vector has 1,496,192 values, more than 2^{20}. So a plain `then_sum` on them as floats sums a random million of them. The result is about 70% of the true sum, as the one-run block at the end shows, and nothing warns you. The guarantee holds for the truncated sum, but the number answers a different question. So set a size limit from public knowledge above any size the data can have, here four million rows. Or put the exact size in the domain when it is public.

```
def float_sum(size_limit: int | None = None) -> dp.Transformation:
    """The float sum: `then_sum` truncates to 2^20 rows; a size limit of your own lifts
     ↪ that."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.symmetric_distance()
    clamped = space >> dp.t.then_clamp(FLOAT_BOUNDS)
    if size_limit is None:
        return clamped >> dp.t.then_sum()
    return clamped >> dp.t.make_bounded_float_checked_sum(size_limit=size_limit,
     ↪  bounds=FLOAT_BOUNDS)
```

```
float sum, size limit 2^20=1,048,576: map(36)=3528.0000009127
float sum, size limit 4,000,000: value 56,315,594  map(36)=3528.0000038179
```

Both sensitivities are 3,528 hours plus a rounding term, and the larger limit gives a larger term, in the sixth decimal place instead of the seventh. The term also depends on the summation algorithm, which the specialized constructors take as the `S` argument.

```
import opendp.prelude as dp

dp.enable_features("contrib")
pairwise = dp.t.make_sized_bounded_float_checked_sum(1_000, (0.0, 10.0),
 ↪  S="Pairwise<f64>")
sequential = dp.t.make_sized_bounded_float_checked_sum(1_000, (0.0, 10.0),
 ↪  S="Sequential<f64>")
print(f"{pairwise.map(0):.2e} {sequential.map(0):.2e}")  # 4.43e-11 4.44e-09
pairwise = dp.t.make_sized_bounded_float_checked_sum(1_000_000, (0.0, 10.0),
 ↪  S="Pairwise<f64>")
sequential = dp.t.make_sized_bounded_float_checked_sum(1_000_000, (0.0, 10.0),
 ↪  S="Sequential<f64>")
print(f"{pairwise.map(0):.2e} {sequential.map(0):.2e}")  # 8.85e-08 4.44e-03
```

The map at distance 0 isolates the rounding term, because the same rows in a different order can sum to different floats. The default pairwise sum adds the values as a balanced tree, so its term grows a little faster than the number of rows. The sequential sum adds them one after another, and its term grows with the square of the number of rows. Use the sequential analysis for a sum computed elsewhere, because database engines add floats in sequence. Integer sums have no rounding term, which is one reason to cast hours to integers.

To release the integer sum, add a Laplace measurement with a scale equal to the sensitivity over ε (the full edition's chapter on the Laplace and Gaussian mechanisms). Each of the three measurements on this page uses $\varepsilon = 0.5$ on its own, and Combinators composes them. Here the scale is 7,056 hours, and the error bar is 0.04% of the total.

```
sum release: scale=7056  map(36)=0.5000  95% accuracy=21,138.4
```

9.4. The mean and the variance

The mean is a sum divided by a size, and `make_mean` divides by the size in its input domain, not by the length of the data. So the domain must have a size, or the constructor refuses.

```
mean without a size: dataset size must be known. Either specify size in the input
 ↪  domain or use make_resize
```

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(bounds=(0.0, 10.0)), size=10),
 ↪  dp.symmetric_distance()
mean = space >> dp.t.then_mean()
print(mean([5.0] * 10), mean.map(2))  # 5.0 1.0000000000000169
print(mean([5.0]))  # 0.5
print(mean.input_domain.member([5.0]))  # False
```

With ten values between 0 and 10, one replaced value moves the mean by at most the width over the size. So `map(2)` is 1, plus a rounding term. The second print shows the documentation's warning: the mean of a one-element list is 0.5, because the division uses the declared size and nothing checks the data. Each transformation assumes that its input is in its input domain, and `member` checks that.

On the survey the size is not public, so the chain resizes first, with the true median of 37 hours as the padding constant. A real release would resize to a released count, as in A first release, or to a public total. The script resizes to the true size, which it can do because the data are public, so nothing is padded.

```
def mean_chain(size: int, constant: float) -> dp.Transformation:
    """Clamp, resize to a known size (padding with the constant), then the mean over
     ↪  that size."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.symmetric_distance()
    return space >> dp.t.then_clamp(FLOAT_BOUNDS) >> dp.t.then_resize(size=size,
     ↪  constant=constant) >> dp.t.then_mean()
```

```
mean chain: value 37.6393  map(36)=0.0023580
mean release: scale=0.0047160  map(36)=0.5000  95% accuracy=0.0141
```

Compare this map with the sized sum's. That map was 18 replacements of 98 hours, and divided by the size it is 0.0012. The mean's map is twice that, because the resize in front of it is 2-stable and turns the input distance of 36 into 72. That is the cost of an unknown size, paid once per chain. At $\varepsilon = 0.5$, the Laplace scale is 0.0047 hours and the error bar 0.014 hours.

The documentation's mean page also shows the plug-in approach, which releases a sum and a count and then divides. The division is post-processing, so it costs no more budget (the full edition's chapter on properties). But at the same budget the resized mean is tighter, because its numerator

and denominator use the same number of rows. The variance uses the same preprocessing. On the resized chain, `then_variance` gives 137.63 hours squared, and its stability at distance 36 is 0.2311.

```
variance chain: value 137.6290  map(36)=0.2311
```

9.5. Quantile scores

Noise alone cannot release a quantile, and the documentation's quantile page shows why. Take a dataset with half its values at the lower bound and half at the upper. One added value moves its median from the middle of the range to the upper bound, so the sensitivity is half the range. Laplace noise of that size hides the median. So OpenDP scores every candidate by how far it is from the quantile and lets a selection mechanism pick one. This is the exponential mechanism of the full edition's chapter on selection, and `make_quantile_score_candidates` computes the scores.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
median = space >> dp.t.then_quantile_score_candidates(candidates=[0, 25, 50, 75, 100],
 ↪  alpha=0.5)
print(median([10, 20, 30, 40, 50, 60, 70, 80, 90]))  # [9, 5, 0, 5, 9]
print(median.output_metric, median.map(1))  # LInfDistance(T=u64) 1
quartile = space >> dp.t.then_quantile_score_candidates(candidates=[0, 25, 50, 75,
 ↪  100], alpha=0.25)
print(quartile([10, 20, 30, 40, 50, 60, 70, 80, 90]), quartile.map(1))  # [9, 1, 8, 19,
 ↪  27] 3
```

A candidate's score for the median is the absolute difference between the number of values below it and the number above it. In the snippet, the middle candidate has four values on each side and scores zero, while the lowest has all nine above it. So the lowest score is the best, and the selection picks the minimum. For another quantile, the two counts get weights. The quarter quantile weights the values below by three, so the score is zero where a quarter of the data are below, and its stability is 3. The output metric is the L_∞ distance, the largest change of any score, because one row moves every score at once.

On the survey, the candidates are the whole numbers of hours from 0 to 98.

```
def median_scores() -> dp.Transformation:
    """One score per candidate: how far it is from being the median; the lowest score
     ↪  is the best candidate."""
    space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
    return space >> dp.t.then_quantile_score_candidates(candidates=CANDIDATES,
     ↪  alpha=0.5)


def median_measurement(scale: float) -> dp.Measurement:
```

```
    """Noisy max over the scores releases an index; `negate` makes the lowest score the
     ↪  one most likely to win."""
    return median_scores() >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale,
     ↪  negate=True)
```

```
median scores: lowest 15,190 at 37 hours, next 89,573 at 36  map(36)=36
median release: scale=144  map(36)=0.5000  score gap=74,383  gap/scale=517
```

The lowest score is at 37 hours, the true median, and the next candidate scores worse by 74,383. To release it, the noisy max of Measurements adds noise to every score and reports the index of the best one. Its privacy map under the max divergence is twice the stability over the scale. So the scores' stability of 36 at a scale of 144 gives $\varepsilon = 0.5$. The gap between the two best scores is over 500 times the scale, so the released median is 37 on almost every run. The `make_private_quantile` of the next chapter chains the same steps but doubles its scale for the median, so its scale 72 is this chain's 144.

9.6. Reading a chain

Two methods read the promise of a chain. The `map` method gives the stability, or the privacy loss when a measurement ends the chain. And the `check` method asks whether the promise at an input distance is within a bound. The sum chain's map at 36 is 3,528, so `check(36, 3528)` is true and `check(36, 3527)` is false. Neither method touches the data, and the context runs `check` in a loop to find a scale (Utilities and the Context API inside).

```
int sum chain: check(36, 3528)=True  check(36, 3527)=False
```

You must also read refusals. The `>>` operator checks, when it builds the chain, that the output space of each step is the input space of the next. Below, a sum built on vectors bounded by 0 and 100 follows a clamp to 0 and 98.

```
try:
    _ = clamped >> dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0, 100))),
 ↪  dp.symmetric_distance())
    mismatch = "built"
except dp.OpenDPException as err:
    mismatch = f"{err.variant}: {err.message}"
```

```
mismatched chain: DomainMismatch: Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
    output_domain: VectorDomain(AtomDomain(bounds=[0, 98], T=i32))
    input_domain:  VectorDomain(AtomDomain(bounds=[0, 100], T=i32))
```

The message names both spaces. To fix the chain, build the sum from the clamp's output space, as `make_sum(*clamped.output_space)`. The `then_` form does that for you. A metric mismatch is similar. An ordered sum needs the insert-delete distance, so it refuses to follow a clamp under

the symmetric distance, and `make_ordered_random` between them converts the metric. The third refusal is a domain that lacks what the constructor needs, such as a size for the mean or bounds for the sum.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
clamp = space >> dp.t.then_clamp(bounds=(0, 10))
try:
    clamp >> dp.t.make_bounded_int_ordered_sum(bounds=(0, 10))
except dp.OpenDPException as err:
    print(err.variant)  # MetricMismatch
try:
    space >> dp.t.then_sum()
except dp.OpenDPException as err:
    print(err.message)  # `input_domain` must be bounded. Use `make_clamp` to bound
     ↪ data.
```

Every refusal happens when the chain is built, before it touches any data. Here are the releases of one run, to read against their error bars.

```
released (one run):
  sum 56,307,689  (true 56,315,594, within 21,138 with probability 0.95)
  float sum truncated to 2^20 rows 39,467,741  (true 56,315,594; about 70% of it)
  mean 37.6367  (true 37.6393, within 0.0141 with probability 0.95)
  median 37  (true 37)
```

The sum and the mean landed within their bars, as they do about 19 times in 20. The truncated float sum is not noise but a wrong computation, 30% short, and no error bar covers it. The median is exact, as the gap between its scores predicted.

9.7. Summary

Table 9.1.: The transformations of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`make_x(domain, metric, ...)`, `space >> then_x(...)`	Two forms of every constructor	Read `output_space` after each step
`transformation.map(d_in)`	The stability at input distance `d_in`	Read it at the privacy unit's distance, 36 here
`make_cast`, `make_cast_default`	Parse strings into a type; failures become options or the default	Impute or drop options before any aggregator

Call or concept	What it does	Rule of thumb
`make_impute_constant`, `make_impute_uniform_float`	Replace missing values by a constant or a uniform draw from bounds	Stability 1; the constant or bounds are public
`make_clamp(bounds)`	Pin values into public bounds	Bounds come from a codebook, never from the data
`make_resize(size, constant)`	Pad or sample to a known size, written into the domain	Stability 2; a wrong size biases silently
`make_find`, `make_find_bin`, `make_index`, `make_count_by_categories`	Values to indices, numbers to bins, indices to labels, counts per category	Counts carry an L_1 or L_2 metric
`make_sum`	Sum of bounded values; with a size, per replacement	Unsized float sums truncate to 2^{20} rows: set `size_limit`
`make_mean`, `make_variance`	Divide by the size in the domain	A resize before them doubles the input distance
`make_quantile_score_candidates`	A score per candidate, lowest best	Feed the scores to the next chapter's noisy max

9.8. References

- OpenDP documentation: "Transformations", "Preprocessing: Resize", "Aggregation: Sum", "Aggregation: Mean" and "Aggregation: Quantile" in the API user guide.
- Casacuberta, Shoemate, Vadhan and Wagaman, "Widespread Underestimation of Sensitivity in Differentially Private Libraries and How to Fix It" (2022): the rounding term and the truncation of the float sum.
- Smith, "Privacy-Preserving Statistical Estimation with Optimal Convergence Rates" (2011): the quantile scores and their sensitivity.
- McKenna and Sheldon, "Permute-and-Flip: A New Mechanism for Differentially Private Selection" (2020): the selection behind the noisy max.
- The full edition's chapters on sensitivity in the full edition, the Laplace and Gaussian mechanisms in the full edition, post-processing and composition in the full edition and selection in the full edition derive the results this chapter states.

10. Measurements

A **measurement** ends a chain by mapping data to a random release. Its privacy map bounds how much the release distribution can change when one person's data change. Transformations described the deterministic part of a query; this chapter covers the randomized part and its guarantee.

Every OpenDP measurement has four parts: an input domain, an input metric, an output measure and a privacy map. The domain specifies valid inputs; the metric compares them. The output measure defines the privacy-cost units, and the map converts input distance into that cost. These components let you calibrate and compare mechanisms.

The script opendp_measurements.py builds the mechanisms on OpenDP 0.16.0 and prints their metrics, measures and maps. It applies thresholded counts of `ILOSTAT` and a private median of `HWUSUAL` to the Labour Force Survey (LFS); other mechanisms use the documentation's toy inputs. Maps, checks, thresholds, scores and errors are deterministic. Releases vary because OpenDP's noise cannot be seeded, so the page labels them as one run.

```
uv run python code/part03-framework/opendp_measurements.py
```

10.1. Additive noise

The simplest measurements add noise to a number. `make_laplace` adds Laplace noise and measures cost as ε under pure differential privacy (DP) and `MaxDivergence`. `make_gaussian` adds Gaussian noise and measures cost as ρ under zero-concentrated differential privacy (zCDP) and `ZeroConcentratedDivergence`. Both use the same input space.

```
def scalar_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """Laplace and Gaussian noise on one float; the input space is the same, the output
     ↪  measures differ."""
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    laplace = space >> dp.m.then_laplace(scale=scale)  # output measure: MaxDivergence,
↪   read as epsilon
    gaussian = space >> dp.m.then_gaussian(scale=scale)  # output measure:
↪   ZeroConcentratedDivergence, read as rho
    return laplace, gaussian
```

The Laplace map is sensitivity divided by scale, so unit sensitivity and scale cost $\varepsilon = 1$. The Gaussian map is squared sensitivity divided by twice the squared scale, giving $\rho = 0.5$ for the same values. `check` compares this cost with a budget: sensitivity 2 exceeds the budget that accepts sensitivity 1.

```
laplace float: map(1.0)=1.0 measure=MaxDivergence
gaussian float: map(1.0)=0.5 measure=ZeroConcentratedDivergence
laplace float: check(1.0, 1.0)=True check(2.0, 1.0)=False
```

These privacy measures provide different guarantees. Pure DP with ε limits the probability ratio for any release to e^{ε}. zCDP with ρ bounds privacy loss on average and in its tails, matching Gaussian noise. Combinators converts ρ to ε with slack δ; Assessing utility and choosing the noise compares the guarantees. The full edition's chapter on the Laplace and Gaussian mechanisms derives both maps.

10.1.1. Integers, floats and vectors

The same constructors accept integers. On an integer domain, `make_laplace` draws from the discrete Laplace distribution and `make_gaussian` from the discrete Gaussian, so the release is an integer. The maps do not change.

```
laplace int: map(1)=1.0 output type=int
gaussian int: map(1)=0.5
```

Vector inputs require different metrics. Laplace noise uses an L1 bound, the sum of absolute coordinate differences, because its map adds coordinate costs. Gaussian noise uses an L2 bound, the square root of summed squared differences, because its cost adds in squares. OpenDP rejects a mismatched metric.

```
def vector_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """The same two mechanisms on a vector of floats: Laplace wants an L1 bound,
     ↪  Gaussian an L2 bound."""
    domain = dp.vector_domain(dp.atom_domain(T=float, nan=False))
    laplace = (domain, dp.l1_distance(T=float)) >> dp.m.then_laplace(scale=scale)
    gaussian = (domain, dp.l2_distance(T=float)) >> dp.m.then_gaussian(scale=scale)
    return laplace, gaussian
```

Doubling L2 sensitivity quadruples ρ, while the Laplace cost doubles. Passing a scalar to a vector mechanism raises a type error that states the expected input.

```
laplace vector: metric=L1Distance(f64) map(1.0)=1.0
gaussian vector: metric=L2Distance(f64) map(1.0)=0.5
gaussian vector: map(2.0)=2.0 map(4.0)=8.0
laplace vector on a scalar: Expected type is Vec<f64> but input data is not a list.
gaussian on an L1 space: No match for concrete type L1Distance<f64>. See
 ↪  https://github.com/opendp/opendp/discussions/451.
```

10.1.2. Why OpenDP rounds, and the parameter `k`

Computers cannot sample a continuous distribution. Floating-point noise can also leak inputs through gaps between representable values. OpenDP converts a float to an exact rational, adds discrete noise to its numerator, then rounds back to a float. This is equivalent to continuous noise followed by rounding, a post-processing step, so the analysis remains exact.

The rational grid is 2^k, where `k` is a constructor parameter. The default is the smallest exponent of a 64-bit float, giving an exact grid with no extra cost. Coarser grids sample faster but may increase the distance between neighbouring inputs by one grid step. The map adds that step to sensitivity: a grid of 0.5 adds 0.5 even at zero input sensitivity.

```
laplace k=-1074: map(0.0)=0.0 map(1.0)=1.0
laplace k=-1: map(0.0)=0.5 map(1.0)=1.5
laplace k=0: map(0.0)=1.0 map(1.0)=2.0
laplace k=1: map(0.0)=2.0 map(1.0)=3.0
laplace k on integers: k is only valid for domains over floats
```

Integer domains need no grid, so OpenDP rejects `k` for them. Change `k` only to speed up a vector release, then check the map again. The full edition's chapter on discrete mechanisms explains the discrete distributions' guarantees.

10.1.3. The geometric mechanism and `make_noise`

`make_geometric` implements the integer Laplace mechanism with an additional option. Supplying `bounds` makes sampling constant-time, closing a timing side channel, and clamps the release. It rejects floats because the geometric distribution is discrete.

```
geometric = int_space >> dp.m.then_geometric(scale=1.0)
bounded = int_space >> dp.m.then_geometric(scale=1.0, bounds=(0, 10))  # runs in
 ↪  constant time; clamps the output
```

```
geometric: map(1)=1.0 measure=MaxDivergence
geometric bounds (0, 10): map(1)=1.0
geometric on floats: No match for concrete type f64. See
 ↪  https://github.com/opendp/opendp/discussions/451.
```

`make_noise` generalizes both constructors. Specify the output measure, and OpenDP selects Laplace noise for max divergence or Gaussian noise for zero-concentrated divergence. Use it when callers choose the privacy measure.

```
noise_pure = space >> dp.m.then_noise(dp.max_divergence(), scale=1.0)  # the Laplace
 ↪  mechanism
noise_zcdp = space >> dp.m.then_noise(dp.zero_concentrated_divergence(), scale=1.0)  #
 ↪  the Gaussian mechanism
```

```
make_noise MaxDivergence: map(1.0)=1.0 measure=MaxDivergence
make_noise ZeroConcentratedDivergence: map(1.0)=0.5 measure=ZeroConcentratedDivergence
```

10.2. Canonical noise

The mechanisms above take a scale and give its cost. The canonical noise mechanism takes the sensitivity and the budget instead, and draws from the distribution that spends exactly that budget. The budget is approximate DP, which relaxes the pure guarantee with a slack δ, the probability that the factor e^{ε} fails. Its canonical distribution is the Tulap distribution, a discrete Laplace draw plus a uniform one, which leaves the most powerful hypothesis test possible at the budget. So use it when a release feeds a statistical test or a confidence interval.

```
def canonical_mechanism(d_in: float, epsilon: float, delta: float) -> dp.Measurement:
    """Canonical noise on one float, calibrated so that inputs at most d_in apart cost
     ↪  exactly (epsilon, delta)."""
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    return space >> dp.m.then_canonical_noise(d_in=d_in, d_out=(epsilon, delta))
```

The map is a step and not a line, because the calibration is fixed at construction. Up to the declared sensitivity, the map returns the declared pair, and above it the map refuses, and so does `check`.

```
canonical noise: measure=Approximate(MaxDivergence)
canonical noise: map(1.0)=(1.0, 1e-07) map(0.5)=(1.0, 1e-07)
canonical noise: check(1.0, (1.0, 1e-07))=True check(1.0, (0.9, 1e-07))=False
canonical noise map(2.0): d_in from the map (2) must be in [0, 1]
```

The installed version supports only a scalar float and an (ε, δ) budget. In the Context API, `query.canonical_noise()` with `binomial_size` gives confidence intervals and p-values for a binomial count, as the documentation's notebook shows.

10.3. Thresholded noise

When the keys of a histogram come from the data, noise alone does not protect them: if a category appears in the release, someone in the data belongs to it. So the thresholded noise mechanisms release only the keys whose noisy count passes a threshold. A key with many contributors passes on both of two neighbouring datasets, so its presence says nothing about one person. A key with one contributor passes only if its noise is very large, and the slack δ pays for that chance. Grouping used this mechanism through Polars.

The input domain is a map from keys to counts, and the input metric is a triple of bounds. They bound how many keys one person can touch, the total change across all keys, and the change to any one key. The total is in L1 for Laplace and in L2 for Gaussian.

```
def threshold_mechanisms(scale: float, threshold: float) -> tuple[dp.Measurement,
 ↪  dp.Measurement]:
    """Laplace and Gaussian noise on a dictionary of float counts, releasing only the
     ↪  keys whose noisy count passes."""
    domain = dp.map_domain(dp.atom_domain(T=str), dp.atom_domain(T=float, nan=False))
    laplace = dp.m.make_laplace_threshold(
        domain, dp.l01inf_distance(dp.absolute_distance(T=float)), scale=scale,
 ↪  threshold=threshold
    )
    gaussian = dp.m.make_gaussian_threshold(
        domain, dp.l02inf_distance(dp.absolute_distance(T=float)), scale=scale,
 ↪  threshold=threshold
    )
    return laplace, gaussian
```

In the documentation's example, a person touches one key by at most one. At scale 1, the Laplace version prices that at $\varepsilon = 1$ plus a δ, the chance that a count one person can make passes the threshold by noise alone. The Gaussian version prices it in ρ with a far smaller δ, because its tails fall faster.

```
laplace threshold: metric=L01InfDistance(AbsoluteDistance(f64))
 ↪  measure=Approximate(MaxDivergence)
laplace threshold scale 1 threshold 20: map((1, 1.0, 1.0))=(1.0, 2.8014e-09)
gaussian threshold scale 1 threshold 20: map((1, 1.0, 1.0))=(0.5, 1.1102e-16)
gaussian threshold: measure=Approximate(ZeroConcentratedDivergence)
```

But a smaller δ is not a smaller cost, because the currencies differ. At the Laplace δ, the Gaussian guarantee converts to an ε about six times larger (Combinators). The Gaussian wins only when a person touches many keys by a little, as in the second line. Then the L2 bound is much smaller than the L1 bound.

```
gaussian threshold as approximate DP at delta=2.8014e-09: epsilon=6.3036
spread (100, 10.0, 0.001): laplace (0.1000, 1.0316e-07) gaussian epsilon=0.0500
```

In that line, the Laplace map charged a tenth, not ten. A hundred keys changed by at most a thousandth change the total by at most a tenth, so the library tightens the L1 bound to that product. Integer counts use the discrete distributions. The general `make_noise_threshold` takes the measure as an argument, and the installed version needs it wrapped as approximate, which its docstring does not say.

```
discrete laplace threshold scale 1 threshold 10: map((1, 1, 1))=(1.0, 3.3190e-05)
make_noise_threshold Approximate(MaxDivergence): map((1, 1, 1))=(1.0, 3.3190e-05)
make_noise_threshold MaxDivergence: output_measure must be MaxDivergence or
 ↪  ZeroConcentratedDivergence
```

10.3.1. Counts of ILOSTAT

The LFS codes the labour status of each response in ILOSTAT, with four codes in this sample. Suppose that you do not know which codes occur, so the keys must come from the data. The script counts the responses per code with Polars and gives the dictionary to a thresholded Laplace mechanism.

```
def ilostat_counts(frame: pl.LazyFrame) -> dict[int, int]:
    """The number of responses in each ILOSTAT category, keyed by the category: the
     ↪  input of the thresholded count."""
    counts = frame.group_by("ILOSTAT").len().sort("ILOSTAT").collect()
    return {int(key): int(count) for key, count in zip(counts["ILOSTAT"],
     ↪  counts["len"])}


def ilostat_mechanism(scale: float, threshold: int) -> dp.Measurement:
    """Discrete Laplace noise on integer counts keyed by an integer category, with a
     ↪  threshold on the noisy count."""
    domain = dp.map_domain(dp.atom_domain(T=int), dp.atom_domain(T=int))
    metric = dp.l01inf_distance(dp.absolute_distance(T=int))
    return dp.m.make_laplace_threshold(domain, metric, scale=scale,
     ↪  threshold=threshold)
```

One person has at most 36 rows (the privacy unit). Without more public knowledge, each entry of the triple is 36, because a person could touch 36 keys once each, or one key 36 times. At a budget of 1, the scale is 36, with an error bar of about 108 responses. So the threshold is the only free choice, and it sets δ. The script prices three thresholds and finds the smallest that keeps δ under one in ten million.

```
ILOSTAT counts: {1: 1517011, 2: 155661, 3: 1450283, 9: 688977}
ILOSTAT d_in: (36, 36, 36) scale: 36 95% accuracy: 108.3
ILOSTAT threshold 500: map=1.0, delta=4.4835e-05
ILOSTAT threshold 1000: map=1.0, delta=4.1667e-11
ILOSTAT threshold 1500: map=1.0, delta=3.9968e-15
ILOSTAT smallest threshold with delta <= 1e-07: 720
ILOSTAT threshold 10: threshold (10) must not be smaller than l-infinity sensitivity 36
```

Each δ is the chance that the noise lifts a count that one person can make alone, at most 36, over the threshold. It falls fast as the threshold rises. The mechanism refuses a threshold below the per-key sensitivity, because one person could then make a key with no noise. Here the smallest count is over 150,000, so a threshold of 720 drops nothing. The threshold removes only rare categories, which is where it must act. The full edition's chapter on sensitivity shows, under partition selection, why it costs a δ.

10.4. The approximate Laplace projection

The approximate Laplace projection is another answer to unknown keys. It releases a hashed sketch of the whole key space, which you then query for any key, including keys that never

occurred. It lists no keys, so no key leaks, and queries after the release are post-processing. It has two costs: answers are rounded to multiples of `alpha`, four by default, and two frequent keys can collide in the hash unless the sketch is large enough. The size of the sketch grows with two public bounds. These are `total_limit` on the sum of all counts and `value_limit` on any one count.

```
def projection_mechanism(scale: float) -> dp.Measurement:
    """A private projection of counts keyed by strings; releases a queryable that
     ↪  answers for any key, known or not."""
    domain = dp.map_domain(dp.atom_domain(T=str), dp.atom_domain(T=int))
    metric = dp.l01inf_distance(dp.absolute_distance(T=int))
    return dp.m.make_alp_queryable(domain, metric, scale=scale, total_limit=50_000,
     ↪  value_limit=1_000)
```

Be careful: here `scale` is not a noise scale but the privacy loss per unit of sensitivity, so a larger value spends more, not less. The map at unit sensitivity equals the scale, and it grows with the L1 entry of the triple.

```
alp scale 1: map((1, 1, 1))=1.0 measure=MaxDivergence
alp scale 2: map((1, 1, 1))=2.0
alp scale 1: map((2, 2, 1))=2.0
```

The documentation finds the scale for a budget with `binary_search_chain` (Utilities and the Context API inside). On the toy table, the one-run answers are within a few units of the true counts. The name that is not in the table comes back near zero.

10.5. Noisy max and selection

Some releases are a choice: which category is largest, which candidate is the median. The noisy max mechanisms add noise to each score and release the index of the largest. Their input metric is the L-infinity distance, a bound on the change of any one score. That is the right bound, because the release depends on how the scores compare. The output measure chooses the noise: exponential under the max divergence, and Gumbel under the zero-concentrated divergence.

```
def noisy_max_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """Select the index of the largest score: exponential noise under pure DP, Gumbel
     ↪  noise under zCDP."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.linf_distance(T=float)
    exponential = space >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale)
    gumbel = space >> dp.m.then_noisy_max(dp.zero_concentrated_divergence(),
 ↪  scale=scale)
    return exponential, gumbel
```

The exponential version's map is twice the sensitivity over the scale, because one person can raise one score and lower another at the same time. So at a scale of 2, a unit sensitivity costs $\varepsilon = 1$. If the scores can move only in one direction, declare the metric monotonic, which removes the factor of two. The Gumbel version's map is the square of the same ratio, divided by eight. Negated

scores select the smallest at no extra cost, and `make_noisy_top_k` releases the best k indices for k times the cost.

```
noisy max: metric=LInfDistance(T=f64) measure=MaxDivergence
noisy max exponential scale 2: map(1.0)=1.0
noisy max gumbel scale 2: map(1.0)=0.125 measure=ZeroConcentratedDivergence
noisy max monotonic scale 2: exponential map(1.0)=0.5 gumbel map(1.0)=0.03125
noisy min scale 2: map(1.0)=1.0
noisy top 2 scale 2: exponential map(1.0)=2.0 gumbel map(1.0)=0.25
```

The older `make_report_noisy_max_gumbel` still exists, but the installed version marks it deprecated. The full edition's chapter on selection shows why selection costs twice what a count costs, and why exponential noise beats Gumbel under pure DP.

```
report_noisy_max_gumbel scale 2: map(1.0)=1.0 measure=MaxDivergence
report_noisy_max_gumbel: Call to deprecated function (or staticmethod)
 ↪  make_report_noisy_max_gumbel
```

10.5.1. A private median of `HWUSUAL`

OpenDP releases a quantile by selection. The transformation `make_quantile_score_candidates` of the previous chapter scores each candidate, here the hours from 1 to 98, by how far it is from splitting the data in half. The best candidate has the smallest score, so a noisy max over negated scores selects it. A lookup in the candidate list then turns the index into an answer, as post-processing. The constructor `make_private_quantile` does the three steps in one call.

```
def hours_of_the_working(frame: pl.LazyFrame) -> list[float]:
    """HWUSUAL for the people at work (ILOSTAT 1), keeping 1 to 98 hours: 0 means
     ↪  varying, 99 not applicable."""
    at_work = (pl.col("ILOSTAT") == 1) & pl.col("HWUSUAL").is_between(1, 98)
    return frame.filter(at_work).select("HWUSUAL").collect()["HWUSUAL"].to_list()


def median_by_hand(scale: float) -> tuple[dp.Transformation, dp.Measurement]:
    """The private median from its pieces: score each candidate, then pick an index by
     ↪  noisy max."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.symmetric_distance()
    scores = space >> dp.t.then_quantile_score_candidates(candidates=HOURS, alpha=0.5)
    select = scores >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale,
 ↪  negate=True)  # the smallest score wins
    return scores, select  # the released index is looked up in HOURS afterwards, which
     ↪  is post-processing


def median_mechanism(scale: float) -> dp.Measurement:
    """The same median in one constructor."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)),
 ↪  dp.symmetric_distance()
    return space >> dp.m.then_private_quantile(dp.max_divergence(), candidates=HOURS,
     ↪  alpha=0.5, scale=scale)
```

The data are the usual hours of the people at work, without the codes for “varies” and “not applicable”. A row filter adds no rows, so the input distance is still 36. Each integer score moves by at most the input distance, so the scores’ L-infinity sensitivity is 36.

```
HWUSUAL at work, 1 to 98 hours: n=1,490,113 true median=37
quantile scores: metric=LInfDistance(T=u64) map(36)=36
best candidate: 37 hours, score 9,111
second candidate: 36 hours, score 95,652
gap in noise scales: 1,202
scores >> noisy max scale 72: map(36)=1.0
make_private_quantile scale 36: map(36)=1.0
make_private_quantile zCDP scale 36: map(36)=0.125
```

Two scales appear because `make_private_quantile` doubles the scale you give it. The scores of a median are integers multiplied by two, the denominator of one half. So the constructor multiplies the scale by the same two, and the two maps agree at $\varepsilon = 1$. Read the scores before you trust the release. The best candidate beats the next by over a thousand noise scales, so the release is 37 hours on almost every run. On a smaller dataset the gaps are smaller, and the release moves among nearby candidates, which is the error of a private quantile.

10.6. Randomized response

All the mechanisms so far run on a trusted server that holds the data. Randomized response runs on the respondent’s device before the answer is sent, so no server sees the truth. This is the local model of DP, where the privacy unit is one answer. Its metric is the discrete distance, which is 1 between two different inputs and 0 otherwise. OpenDP gives only the randomizers, not the transport or the aggregation.

```
def randomized_response_mechanisms(prob: float) -> tuple[dp.Measurement,
 ↪  dp.Measurement]:
    """Randomized response on one yes/no answer and on one ILOSTAT category, each kept
     ↪  truthful with probability prob."""
    boolean = dp.m.make_randomized_response_bool(prob=prob)
    categorical = dp.m.make_randomized_response(categories=[1, 2, 3, 9], prob=prob)
    return boolean, categorical
```

The boolean version keeps the true answer with the probability you give and flips it otherwise. Its map is the logarithm of the odds of the truth, so a probability of three quarters costs about 1.1. It refuses a probability below one half. The categorical version keeps the truth with the same probability and otherwise picks one of the other categories at random. Its map grows with the logarithm of the number of alternatives, so four `ILOSTAT` codes cost twice what a yes/no answer costs.

```
randomized response bool: domain=AtomDomain(T=bool) metric=DiscreteDistance()
 ↪  measure=MaxDivergence
randomized response bool prob 0.75: map(1)=1.0986 ln(0.75/0.25)=1.0986
randomized response bool prob 0.9: map(1)=2.1972
randomized response bool prob 0.4: probability must be within [0.5, 1]
randomized response 4 categories prob 0.75: map(1)=2.1972 ln(3*0.75/0.25)=2.1972
```

One randomized answer is almost useless by design. The answers become useful in aggregate, after a correction for the known flip rate. The third randomizer, for a bit vector, is the primitive behind RAPPOR, the first large deployment of local DP. Its domain bounds how many bits can be set, and it rerandomizes each bit with probability `f`. Its map is twice that weight times the log odds that a bit survives.

```
def bitvector_mechanism(f: float, max_weight: int) -> dp.Measurement:
    """Randomized response on a bit vector with at most max_weight ones, each bit
     ↪  rerandomized with probability f."""
    return dp.m.make_randomized_response_bitvec(dp.bitvector_domain(max_weight=
     ↪  max_weight), dp.discrete_distance(), f=f)
```

```
bit vector f=0.8 max_weight=3: domain=BitVectorDomain(weight=3) map(1)=2.4328
 ↪  formula=2.4328
```

The script sends the documentation's 24-bit vector through the randomizer ten thousand times and runs `debias_randomized_response_bitvec` on the answers. The three set bits come back near ten thousand and the others near zero, with a spread of a few hundred. So local DP costs much accuracy (the full edition's chapter on trust models).

Here are the releases of one run. The additive releases are within a few units of 100, the thresholded count kept all four codes, the projection answered the unknown name with zero, and both medians are 37.

```
released (one run):
  laplace on 100.0: 104.05
  gaussian on 100.0: 100.26
  discrete laplace on 100: 100
  laplace vector on [0.0, 2.0, 2.0]: [-1.4, 1.17, 4.28]
  laplace k=-1 on 100.0, five draws: [99.5, 99.5, 102.5, 101.5, 100.0]
  bounded geometric on 10, five draws: [10, 10, 10, 9, 7]
  canonical noise on 100.0: 99.74
  laplace threshold on {a: 0, b: 20, c: 40}: {'c': 37.74439496773442}
  ILOSTAT counts: {1: 1517058, 2: 155697, 3: 1450247, 9: 689013}
  alp answers for Michael, James, Sharon, Lancelot: [748.0, 500.0, 48.0, 0.0]
  noisy max on [0, 2, 4, 6]: 3
  noisy min on [0, 2, 4, 6]: 0
  noisy top 2 on [0, 2, 4, 6]: [3, 1]
  median by hand: 37 hours
  median by make_private_quantile: 37 hours
  randomized response on True, ten draws: [True, True, False, True, True, False, False,
 ↪  False, True, True]
```

```
randomized response on category 1, ten draws: [1, 1, 1, 1, 1, 1, 1, 2, 1, 3]
bit vector release: 10011011100000010001110
bit vector frequencies over 10,000 copies, bits 12, 20, 21: [10050, 10235, 10115]
bit vector frequencies, largest other bit: 565
```

10.7. Summary

Table 10.1.: The measurements of the chapter: their input metric, their output measure, the map the script printed at scale 1 for inputs at distance 1, and what each is for.

Constructor	Input metric	Output measure	Map at scale 1 for inputs at distance 1	What it is for
`make_laplace`	absolute or L1 distance	`MaxDivergence`	$\varepsilon = 1$	One number or a vector under pure DP; integers get the discrete Laplace
`make_gaussian`	absolute or L2 distance	`ZeroConcentratedDivergence`	$\rho = 0.5$	The same under zCDP; vectors cost by the L2 bound
`make_geometric`	absolute distance on integers	`MaxDivergence`	$\varepsilon = 1$	Integer Laplace with `bounds` for constant time
`make_noise`	as above	the one you pass	as above	Lets the caller choose the currency
`make_canonical_noise`	absolute distance	`Approximate(MaxDivergence)`	the one you declared, up to your `d_in`	Exact (ε, δ) calibration; inference on counts
`make_laplace_threshold`	L0, L1, L-infinity triple	`Approximate(MaxDivergence)`	$(1.2801, 10^{-9})$ at threshold 20	Counts over keys from the data; δ pays for the threshold
`make_gaussian_threshold`	L0, L2, L-infinity triple	`Approximate(ZeroConcentratedDivergence)`	$(0.5, 1.1102 \times 10^{-16})$ at threshold 20	The same when a person touches many keys by a little
`make_alp_queryable`	L0, L1, L-infinity triple	`MaxDivergence`	$\varepsilon = 1$ (the scale is a loss, not a noise)	Queries for any key after the release

Constructor	Input metric	Output measure	Map at scale 1 for inputs at distance 1	What it is for
`make_noisy_max`	L-infinity distance	`MaxDivergence` or `ZeroConcentratedDivergence`	$\varepsilon = 2$ or $\rho = 0.5$	Select an index by score; monotonic halves the cost
`make_noisy_top_k`	L-infinity distance	either	$\varepsilon = 4$ for $k = 2$	The best k indices, at k times the cost
`make_private_quantile`	symmetric distance	either	$\varepsilon = 1$	Scores plus noisy max plus lookup in one call
`make_randomized_response_bool`	discrete distance	`MaxDivergence`	1.099 at probability 0.75	One yes/no answer on the respondent's device
`make_randomized_response`	discrete distance	`MaxDivergence`	2.197 for four categories at 0.75	One category; the cost grows with the alternatives
`make_randomized_response_bitvec`	discrete distance	`MaxDivergence`	2.433 for weight 3 at `f` = 0.8	The RAPPOR primitive; debias the aggregate

10.8. References

- OpenDP documentation: "Measurements", "Additive Noise Mechanisms", "Canonical Noise Mechanism", "Thresholded Noise Mechanisms", "Approximate Laplace Projection", "Report Noisy Max Mechanisms" and "Randomized Response" in the API user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism.
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the discrete Gaussian and the discrete Laplace samplers OpenDP uses.
- Awan and Vadhan, "Canonical Noise Distributions and Private Hypothesis Tests" (2023): the Tulap distribution.
- Rogers, "A Unifying Privacy Analysis Framework for Unknown Domain Algorithms in Differential Privacy" (2023): the thresholded mechanisms' maps.
- Aumüller, Lebeda and Pagh, "Differentially Private Sparse Vectors with Low Error, Optimal Space, and Fast Access" (2021): the approximate Laplace projection.
- McKenna and Sheldon, "Permute-and-Flip: A New Mechanism for Differentially Private Selection" (2020): the exponential-noise noisy max.
- Smith, "Privacy-Preserving Statistical Estimation with Optimal Convergence Rates" (2011): the quantile by selection.

- Warner, “Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias” (1965), and Erlingsson, Pihur and Korolova, “RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response” (2014).
- The full edition’s chapters on the Laplace and Gaussian mechanisms in the full edition, selection in the full edition, sensitivity and partition selection in the full edition, discrete mechanisms in the full edition and trust models in the full edition derive the results this chapter states.

11. Combinators

Real releases chain data preparation, aggregation and noise; later queries may depend on earlier answers. The **combinators** compose framework objects and return a combined object with a privacy map. They account for the total differential privacy (DP) cost and reject components whose guarantees do not compose. This chapter explains chain maps, query refusals and conversions between privacy measures.

The script opendp_combinators.py runs on OpenDP 0.16.0. It follows the documentation with 20 seeded integers from 0 to 10, releasing a Laplace count and a Laplace sum of clamped values. One example uses the Labour Force Survey (LFS) sample. Maps, profiles, scales and errors are deterministic; releases vary because OpenDP's noise cannot be seeded and are labelled as one run.

```
uv run python code/part03-framework/opendp_combinators.py
```

11.1. Chaining

Chain objects by applying them in sequence. `make_chain_tt` joins two transformations, `make_chain_mt` appends a measurement to a transformation, and `make_chain_pm` attaches post-processing to a measurement. Two measurements do not need a chainer: computation after a measurement is post-processing and requires no additional accounting. The script first builds a clamped sum and Laplace release with these constructors.

```
def chained_by_hand() -> tuple[dp.Transformation, dp.Measurement]:
    """The clamped sum and its Laplace release, built with the long-form chainers."""
    clamp = INT_SPACE >> dp.t.then_clamp(BOUNDS)
    total = dp.t.make_sum(clamp.output_domain, clamp.output_metric)
    clamped_sum = dp.c.make_chain_tt(total, clamp)  # a transformation: clamp, then sum
    laplace = dp.m.make_laplace(clamped_sum.output_domain, clamped_sum.output_metric,
↪   scale=SUM_SCALE)
    noisy_sum = dp.c.make_chain_mt(laplace, clamped_sum)  # a measurement: the chain,
↪   then noise
    return clamped_sum, noisy_sum
```

Each constructor takes the outer object before the inner one, as in function composition. The chained transformation has a stability map (Transformations): adding or removing one record changes the clamped sum by at most the bound width, 10. The chained measurement's privacy map (Measurements) divides that stability by the noise scale. Laplace noise of scale 5 therefore costs $\varepsilon = 2$.

```
clamp >> sum: map(1)=10
clamp >> sum >> laplace: map(1)=2.0 check(1, 2.0)=True
with >>: map(1)=2.0
```

The `>>` operator selects the chainer. A `then_` constructor also infers the input space, so the same measurement fits in one line.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
noisy_sum = space >> dp.t.then_clamp((0, 10)) >> dp.t.then_sum() >>
 ↪  dp.m.then_laplace(scale=5.0)
print(noisy_sum.map(1))  # 2.0
print(noisy_sum.input_domain)  # VectorDomain(AtomDomain(T=i32))
print(noisy_sum.output_measure)  # MaxDivergence
```

Post-processing is any computation on a release that does not touch the data again. It costs nothing, because what an adversary could learn from the processed value, they could learn from the release itself (the full edition's chapter on properties). In OpenDP you attach a Python function to a measurement with `>>`, and the result has the same privacy map.

```
def divide_by_size(total: int) -> float:
    return total / N  # the released sum over the public size: a mean, at no further
     ↪  cost

mean_of_sum = shortcut >> divide_by_size
```

```
post-processed: map(1)=2.0
```

In version 0.16.0, this requires the `honest-but-curious` feature because OpenDP cannot inspect a Python function. The script enables the feature only around the chain. A post-processor must not read global state, such as the clock, or raise data-dependent exceptions; either behaviour creates a channel absent from the privacy map.

A chain requires matching spaces: the inner object's output domain and metric must equal the outer object's input domain and metric. The script passes a float sum to a Laplace mechanism built for integers, so OpenDP rejects it.

```
try:
    chain = float_sum >> int_laplace  # a float sum feeding a Laplace mechanism on
 ↪  integers
    domain_error = f"chained: {chain}"
except dp.OpenDPException as err:
    domain_error = str(err.message)
```

```
domain mismatch: Intermediate domains don't match. See
 ↪ https://github.com/opendp/opendp/discussions/297
    output_domain: AtomDomain(T=f64)
    input_domain:  AtomDomain(T=i32)
metric mismatch: Intermediate metrics don't match. See
 ↪ https://github.com/opendp/opendp/discussions/297
    output_metric: AbsoluteDistance(i32)
    input_metric:  AbsoluteDistance(f64)
```

The second error comes from connecting an integer count to a Laplace mechanism that expects float distances. Read both spaces in the error and insert a cast, clamp or imputation step to make them match.

Here are the error bars of the sum and the count that the rest of the chapter composes (Assessing utility and choosing the noise explains them).

```
sum: scale=5 95% accuracy=15.4537
count: scale=1 95% accuracy=3.3756
```

11.2. Non-adaptive composition

When you run the sum and the count on the same data, you spend both budgets. **Basic composition** says that the total is the sum of the parts. So an ε_1-DP release and an ε_2-DP release on the same data are together $(\varepsilon_1 + \varepsilon_2)$-DP (the full edition's chapter on properties). The combinator `make_composition` builds that rule into one measurement, which runs every part and returns the releases as a list.

```
both = dp.c.make_composition([shortcut, count])  # one measurement that releases the
 ↪ sum and the count
print(f"composition: map(1)={both.map(1):.1f}  parts: {shortcut.map(1):.1f} +
 ↪ {count.map(1):.1f}")
```

```
composition: map(1)=3.0  parts: 2.0 + 1.0
composition output measure: MaxDivergence
```

The parts must agree on their input domain, input metric and output measure. A composition is a statement about one dataset, one notion of neighbours and one measure of privacy. It is non-adaptive, because every part is fixed before the data are seen.

The same combinator composes measurements under other measures, with each measure's own rule. **Zero-concentrated differential privacy (zCDP)** measures privacy by one parameter ρ, which bounds the Rényi divergence between the output distributions on neighbours at every order. Under zCDP the parameters add like ε (the full edition's chapter on concentrated DP). The Gaussian sum at scale 5 has $\rho = 2$, the Gaussian count at scale 1 has $\rho = 0.5$, and the composition reports their sum.

```
zCDP composition: map(1)=2.50  parts: 2.00 + 0.50
mixed measures: All output measures must be the same
```

The second line shows that you cannot mix measures, because the maps of a Laplace count and a Gaussian count return numbers that mean different things. Section 11.7 shows how to bring them to one measure first.

11.3. Adaptive composition

Often you choose the second query only after you see the first answer. Composition still holds when each query depends on the earlier answers, if the budget of each is fixed in advance (the full edition's chapter on composition). The combinator for this is `make_adaptive_composition`, which needs more arguments, because it cannot read the supporting elements from parts it has not seen.

```
def adaptive_compositor(d_mids: list[float]) -> dp.Measurement:
    """An adaptive compositor over the toy space: one share of pure-DP budget per
     ↪ query, fixed in advance."""
    return dp.c.make_adaptive_composition(
        input_domain=INT_SPACE[0],
        input_metric=INT_SPACE[1],
        output_measure=dp.max_divergence(),
        d_in=1,  # neighbours add or remove one record
        d_mids=d_mids,  # the epsilon each query may spend, in the order the queries
 ↪  arrive
    )
```

The argument `d_in` is the distance between neighbouring datasets, here 1 for one added or removed record. The argument `d_mids` lists the privacy loss that each query may spend, in order, and the compositor's privacy map is their sum, here 3.

```
adaptive: map(1)=3.0
```

When you invoke the compositor on the data, it releases nothing. It returns a **queryable**, a small state machine that holds the data and the remaining shares. It accepts a measurement as a query, checks that the measurement's map at `d_in` fits within the next share, and answers with the release. The script submits the sum, then the sum again, then the count.

```
queryable = compositor(data)  # the data go in once; what comes back answers
 ↪  measurements
adaptive_sum = queryable(shortcut)  # spends the first share, 2.0
try:
    queryable(shortcut)  # asks 2.0 of the second share, 1.0
    over_share = "answered"
except dp.OpenDPException as err:
    over_share = str(err.message)
adaptive_count = queryable(count)  # spends the second share, 1.0
```

```
second query over its share: insufficient budget for query: 2.0 > 1.0
third query: out of queries
```

The shares are spent in order, so the queryable refuses a query larger than the next share, without spending it, and after the second share it refuses every query. A context wraps this object. With `split_evenly_over=3` in A first release, the library built an adaptive compositor with three equal shares, the context's `accountant`. Each `release()` submits to the queryable, and the context's refusal is the "out of queries" above.

On real data, the number of shares sets the noise of every query. The script builds two contexts over the LFS sample, one with the whole budget of $\varepsilon = 1$ on one query and one split evenly over three. Both ask for the sum of `HWUSUAL`, the hours usually worked per week, without the code 99 for "not applicable" and bounded to 0 to 80 hours. One person has at most 36 rows (the privacy unit), so the sum moves by up to 36 times the upper bound.

```
LFS context over 1: accountant map(36)=1.0 d_mids=[1.0]
LFS HWUSUAL sum over 1: scale=2,881.04 95% accuracy=8,630.8
LFS context over 3: accountant map(36)=1.0 d_mids=[0.3333, 0.3333, 0.3333]
LFS HWUSUAL sum over 3: scale=8,643.11 95% accuracy=25,892.4
true HWUSUAL sum: 56,315,594
```

Both accountants map the privacy unit's distance to 1, because the shares add up to the declared budget. But the scale is the sensitivity over the share, so three shares triple the scale and the error bar. An error bar of 26 thousand is small on 56 million hours, but not on a small group. The scale is slightly above the sensitivity over the share because of the float sum's rounding term (Transformations).

11.4. Odometers

Adaptive composition fixes the budget of each query before the first answer, which is sometimes too early. **Fully adaptive composition** lets you choose the privacy loss of each query as you go. An **odometer** tracks the running total of the loss spent, valid at whatever point you stop. The full edition's chapter on adaptive composition shows why the ordinary theorems fail with adaptively chosen parameters, and which totals are valid. For pure DP the sum of the losses spent is valid, and OpenDP's odometer reports it.

```
odometer = dp.c.make_fully_adaptive_composition(
    input_domain=INT_SPACE[0], input_metric=INT_SPACE[1],
 ↪  output_measure=dp.max_divergence()
)
odometer_queryable = odometer(data)
loss_before = odometer_queryable.privacy_loss(1)  # nothing spent yet
odometer_sum = odometer_queryable(shortcut)
loss_after_sum = odometer_queryable.privacy_loss(1)
odometer_count = odometer_queryable(count)
loss_after_count = odometer_queryable.privacy_loss(1)
```

The constructor takes the three supporting elements of the adaptive compositor, but no `d_in` and no `d_mids`. It returns an `Odometer`, not a `Measurement`, with no privacy map, because it promises nothing about the total. On the data it gives an **odometer queryable**, which answers measurements and reports `privacy_loss(d_in)`, the total spent so far at that input distance.

```
odometer loss: before=0.0 after sum=2.0 after count=3.0 after another count=4.0
odometer loss at d_in=2 after three queries: 8.0
```

The total starts at zero, grows by the map of each query, and never refuses anything. At distance 2, each Laplace query costs twice as much, so the total doubles. So the odometer says how much you spent, and the decision to spend more is yours. That suits open exploration, but nothing stops the total from passing what you may claim, and the next combinator adds that stop.

11.5. Privacy filters

A **privacy filter** is an odometer with a cap: it refuses any query that would take the running total past a bound fixed in advance. Since the total can never pass the bound, the filter is a measurement with an ordinary privacy map. For pure DP, a cap on the sum of the realized losses is a valid guarantee (the full edition's chapter on adaptive composition). The combinator `make_privacy_filter` takes the odometer, the input distance and the cap.

```
privacy_filter = dp.c.make_privacy_filter(odometer, d_in=1, d_out=2.0)
filter_queryable = privacy_filter(data)
filter_count_1 = filter_queryable(count)
filter_count_2 = filter_queryable(count)
try:
    filter_queryable(count)  # would take the loss to 3.0
    refusal = "answered"
except dp.OpenDPException as err:
    refusal = str(err.message)
```

```
filter: map(1)=2.0
filter third query: filter is now exhausted: pending privacy loss (3.0) would exceed
 ↪  privacy budget (2.0)
filter loss after refusal: filter is exhausted: no more queries can be answered
```

The filter's map at distance 1 is the cap, whatever queries arrive later, so the filter is a measurement. Two counts at a cost of 1 each fit under a cap of 2. The filter refuses the third with a message that names the pending total and the budget. After one refusal, the installed version treats the filter as finished, and even `privacy_loss` raises an error. So read the running total before the query that can exhaust it.

A context wraps a filter when you give it a budget but no split. The script builds a context on the toy data with `dp.loss_of(epsilon=2.0)` and no split. Its queryable is an odometer queryable behind a filter, and `current_privacy_loss` reads the odometer. The count query needs an explicit scale, because there is no share to fit.

```
context without a split: queryable=OdometerQueryable spent=1.0 remaining=1.0
```

11.6. Chaining and nesting compositors

Compositors and filters are measurements, so you can chain and compose them too. In the first common pattern, a transformation before a compositor does the preprocessing once for all queries. The script chains the clamped sum before an adaptive compositor, so that the queries are measurements on the sum, one integer.

```
sum_compositor = clamped_sum_only >> dp.c.then_adaptive_composition(
    output_measure=dp.max_divergence(),
    d_in=clamped_sum_only.map(1),  # the sum moves by at most 10 when one record
 ↪  changes
    d_mids=[2.0, 1.0],
)
sum_space = clamped_sum_only.output_domain, clamped_sum_only.output_metric
laplace_on_sum = sum_space >> dp.m.then_laplace(scale=SUM_SCALE)  # a query on the sum,
 ↪  not on the vector
```

```
transformation >> compositor: d_in=10 map(1)=3.0
laplace on the sum: map(10)=2.0
```

Set `d_in` with care. The compositor comes after the sum, so its input distance is the sum's stability map at the privacy unit's distance, here 10. The `then_` form takes the domain and metric from the transformation but cannot know the distance, so compute it with `map`. A Laplace mechanism of scale 5 on the sum then costs 2, which fits the first share.

In the second pattern, a compositor is inside a compositor. The inner compositor is a measurement whose map is the sum of its shares. So you can submit it to an outer one, which answers with the inner queryable.

```
outer = adaptive_compositor(d_mids=[3.0, 1.0])
inner = adaptive_compositor(d_mids=[2.0, 1.0])  # a measurement costing 3.0: it fits
 ↪  the outer's first share
outer_queryable = outer(data)
inner_queryable = outer_queryable(inner)  # the answer to a compositor query is a
 ↪  queryable
inner_sum = inner_queryable(shortcut)
outer_count = outer_queryable(count)  # the outer's second share, while the inner
 ↪  queryable stays open
inner_count = inner_queryable(count)  # pure DP composes concurrently, so the inner one
 ↪  still answers
```

```
nested: outer map(1)=4.0 inner map(1)=3.0
```

The outer compositor charges the inner map, 3, whatever the inner one answers later. The script asks the inner queryable for the sum, the outer one for the count, and the inner one again for the

count. This works because pure DP supports concurrent composition, where several queryables from one compositor stay open at once. For approximate zCDP and approximate Rényi differential privacy (RDP), post-processing is not proved to hold while an analyst can still interact. So there the library freezes an earlier queryable when a new query arrives. The documentation nests a zCDP compositor inside an approximate-DP one, after it casts the inner measure.

11.7. Measure casting

A composition needs all of its parts to have the same output measure. But a Laplace release is measured in pure DP, a Gaussian release in zCDP, and a report usually needs a privacy loss ε with a slack δ, the (ε, δ) of approximate DP. The casting combinators move a measurement from one measure to another, each by a conversion that the full edition derives.

The first conversion goes from pure DP to zCDP. An ε-DP mechanism is ρ-zCDP with $\rho = \varepsilon^2/2$ (the full edition's chapter on concentrated DP), and `make_pureDP_to_zCDP` applies that to the map. The combinator `make_approximate` turns a pure-DP measurement into an approximate-DP one with a slack of zero, so that it can compose with measurements that have a real δ.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print(dp.c.make_pureDP_to_zCDP(laplace).map(1.0))  # 0.5
print(dp.c.make_approximate(laplace).map(1.0))  # (1.0, 0.0)
```

You will use the conversion from zCDP to (ε, δ) most, because that is how a Gaussian release is reported. A ρ-zCDP mechanism satisfies approximate DP at every slack at once, with a loss that grows as the slack shrinks. So the conversion gives the **privacy profile**, a curve that gives the smallest privacy loss for each slack. The combinator `make_zCDP_to_approxDP` returns a measurement whose map is that curve, with the methods `epsilon(delta)` and `delta(epsilon)`. The script reads it at a slack of one in ten million, for a Gaussian with $\rho = 0.1$, which is the scale $1/\sqrt{2\rho}$ at unit sensitivity.

```
gaussian = FLOAT_SPACE >> dp.m.then_gaussian(scale=math.sqrt(1 / (2 * rho)))  # rho = 1
 ↪  / (2 scale^2)
profile_measurement = dp.c.make_zCDP_to_approxDP(gaussian)
profile = profile_measurement.map(1.0)  # a curve, not a number
epsilon_at_delta = profile.epsilon(delta=delta)
delta_at_one = profile.delta(epsilon=1.0)
fixed = dp.c.make_fix_delta(profile_measurement, delta=delta)  # the curve at one
 ↪  delta: an (epsilon, delta) pair
```

```
gaussian: rho=0.1000
profile at rho=0.1: epsilon(delta=1e-07)=2.3484 delta(epsilon=1)=0.008933
simple conversion: 2.6391  improved conversion: 2.3484
fix_delta: map(1.0)=(2.3484, 1e-07)  measure=Approximate(MaxDivergence)
```

So a budget of $\rho = 0.1$ is $(2.35, 10^{-7})$-DP. The third line compares that with the two conversions of the full edition. The simple one is

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{11.1}$$

which gives 2.64 here. The improved conversion of Canonne, Kamath and Steinke minimizes a tighter expression over the Rényi order, and the script's numerical minimum agrees with OpenDP to four decimals. So OpenDP uses the improved conversion, which matters when you compare its numbers with a paper that quotes the simple one. The full edition's chapter on RDP conversions shows where the simple formula loses.

A curve cannot compose with a pair, so `make_fix_delta` evaluates the curve at one δ and returns a measurement whose map is the pair. Its output measure is the approximate max divergence, as from `make_approximate`, so the two can compose. The last casting combinator, `make_fixed_approxDP_to_approxDP`, turns a fixed pair back into a curve. For a pure-DP release cast with `make_approximate`, that curve is flat, because ε-DP at zero slack holds at every slack.

```
approximate: map(1.0)=(1.0, 0.0)  measure=Approximate(MaxDivergence)
fixed to profile: epsilon(delta=1e-07)=1.0 epsilon(delta=0)=1.0
laplace count + gaussian count: map(1)=(6.6709, 1e-07)
```

The last line shows the result. The Laplace count, which Section 11.2 could not compose with the Gaussian count, is cast with `make_approximate`. The Gaussian count is cast to a curve and fixed at the same slack, and then `make_composition` accepts the two. The total is large, because a Gaussian count with $\rho = 0.5$ is expensive at so small a slack: casting makes a composition legal, not cheap. When the Gaussian parts dominate, account the whole release in zCDP and convert once at the end, as the practitioner edition's release workflow does.

11.8. Amplification by subsampling

The last combinator changes what a privacy map promises without changing the computation. Suppose that your dataset is a simple random sample of n people from a population of N, and nobody knows who is in the sample. Then a person may not be in the sample, which makes the release more private with respect to the population. The full edition's chapter on subsampling derives this amplification and shows why the sampling scheme must match the neighbouring relation. Under pure DP the amplified loss is

$$\varepsilon' = \ln\Big(1 + \frac{n}{N}\,(e^{\varepsilon} - 1)\Big). \tag{11.2}$$

The script prints it next to the library's map. The combinator `make_population_amplification` needs a measurement whose input domain has the sample size, because the ratio of the two sizes is part of the guarantee.

```
def sample_mean() -> dp.Measurement:
    """The Laplace mean of a sample of exactly SAMPLE values in [0, 10]: the domain
     ↪  carries the size."""
    space = dp.vector_domain(dp.atom_domain(bounds=(0.0, 10.0), nan=False),
 ↪  size=SAMPLE), dp.symmetric_distance()
```

```
    return space >> dp.t.then_mean() >> dp.m.then_laplace(scale=0.5)


def amplified_mean(population_size: int) -> dp.Measurement:
    """The same mean, with its guarantee read for the population the sample was drawn
     ↪ from."""
    dp.enable_features("honest-but-curious")  # the library cannot check that the data
 ↪ are such a sample
    return dp.c.make_population_amplification(sample_mean(),
     ↪ population_size=population_size)
```

```
sample mean: map(2)=2.0000
amplified, population 100: map(2)=0.4940  formula=0.4940
amplified, population 1000: map(2)=0.0619  formula=0.0619
amplifying an unsized domain: elements of the vector domain have unknown size
```

The maps are read at distance 2, because two datasets of the same size differ by a swap of one record. A swap moves the mean of ten values in a range of 10 by 1, so Laplace noise of half that costs $\varepsilon = 2$. For a population of a hundred, the release costs about 0.49, and for a thousand, about 0.06, as the formula also gives. The library refuses a vector domain without a size.

The combinator needs the `honest-but-curious` flag, for the same reason as the Python post-processor: the library cannot verify the claim. The guarantee holds only if the data are a uniform sample of a population of the stated size. It also needs that the readers of the release do not know who is in the sample. So use the map without amplification when the sample depends on the people in it, or when the sampled identities are public.

11.9. Summary

Table 11.1.: The combinators of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`make_chain_tt`, `make_chain_mt`, `make_chain_pm`	Join two transformations, a transformation and a measurement, or a measurement and a post-processor	Write `>>`; it picks the right one
`measurement >> python_function`	Post-processing, at no cost	Needs `honest-but-curious` in 0.16.0; no global state, no data-dependent exceptions
`DomainMismatch`, `MetricMismatch`	A chain whose intermediate domain or metric differs	Read the two printed lines and insert the step that makes them equal

Call or concept	What it does	Rule of thumb
`make_composition([m1, m2])`	One measurement that releases every part; its map is the sum of the parts' maps	Parts share domain, metric and measure; ρ adds like ε
`make_adaptive_composition(..., d_in, d_mids)`	A queryable that answers measurements one at a time, each within the next share	What a context with a split wraps; a query over its share is refused
`make_fully_adaptive_composition`	An odometer whose queryable reports `privacy_loss(d_in)` so far	Never refuses; for exploration, not for a release
`make_privacy_filter(odometer, d_in, d_out)`	An odometer with a cap, which makes it a measurement	What a context without a split wraps; after one refusal it answers nothing
`transformation >> then_adaptive_composition(...)`	Preprocessing shared by every query of a compositor	Set `d_in` to the transformation's map at the privacy unit's distance
A compositor submitted to a compositor	A sub-budget, answered by a queryable	Costs the inner compositor's map; pure DP lets both queryables stay open
`make_pureDP_to_zCDP`, `make_approximate`	ε to $\rho = \varepsilon^2/2$; ε to $(\varepsilon, 0)$	Cast to make a composition legal, then read the total once
`make_zCDP_to_approxDP`	ρ to a privacy profile with `epsilon(delta)` and `delta(epsilon)`	OpenDP uses the improved conversion; $\rho = 0.1$ gives 2.35 at $\delta = 10^{-7}$
`make_fix_delta`, `make_fixed_approxDP_to_approxDP`	A profile at one δ, and a pair to a flat profile	Fix δ last, after composing in zCDP
`make_population_amplification`	A smaller map for a secret simple random sample of a larger population	Sized input domain, `honest-but-curious`, and a sampling you can vouch for

11.10. References

- OpenDP documentation: "Combinators", "Chaining", "(Non-Adaptive) Composition", "Adaptive Composition", "Fully Adaptive Composition and Odometers", "Privacy Filters", "Compositor Chaining and Nesting", "Measure Casting" and "Amplification" in the API user guide.
- Rogers, Roth, Ullman and Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (2016): the odometer and the filter behind `make_fully_adaptive_composition` and `make_privacy_filter`.
- Whitehouse, Ramdas, Rogers and Wu, "Fully Adaptive Composition in Differential Privacy" (2023): the fully adaptive composition theorem the odometer implements.

- Haney, Shoemate, Tian, Vadhan, Vyrros, Xu and Zhang, "Concurrent Composition for Interactive Differential Privacy with Adaptive Privacy-Loss Parameters" (2023): why several queryables may stay open at once.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP, its composition and the cast from pure DP.
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the improved conversion from zCDP to (ε, δ) that the privacy profile implements.
- The full edition's chapters on post-processing and basic composition in the full edition, composition in the full edition, concentrated DP in the full edition, RDP conversions in the full edition, subsampling in the full edition and adaptive composition in the full edition derive the results this chapter states.

12. Utilities and the Context API inside

So far, each query ended with `.laplace()` and let the context choose a scale. The context uses framework utilities that are also available directly: type descriptors specify value types, binary search converts budgets to scales, and serializers convert measurements to text. The Context API wraps these utilities and the combinators in the previous chapter. Understanding this layer helps predict query behaviour and reproduce queries manually when the API rejects them.

The chapter dissects the count query from the first chapter, which returns the same scale of 3. It also shows how to interpret type errors, run the context's parameter search, serialize a measurement and identify the framework object created by each Context API call.

The script opendp_utilities.py runs on OpenDP 0.16.0 and Polars 1.36.1. Plain-data queries use the first chapter's hundred seeded integers; the Polars query counts rows in the France Labour Force Survey (LFS) sample. Type descriptors, errors, scales and privacy maps are deterministic. Released counts vary because OpenDP's noise cannot be seeded, so the page labels them as one run.

```
uv run python code/part03-framework/opendp_utilities.py
```

12.1. Types

OpenDP's Rust core supports signed and unsigned integers from 8 to 128 bits and two float widths. It preserves this precision because proof constants depend on the type; a 32-bit integer sum can overflow where a 64-bit sum cannot. Every domain, metric and measure therefore carries a **type descriptor**, such as the `T=` arguments in the framework chapter. OpenDP does not cast between types automatically.

Type descriptors have three equivalent forms. Canonical strings include `"i32"` and `"Vec<f64>"`. Python types such as `int`, `float`, `str` and `bool` are aliases; `list[int]` aliases a vector. Objects from `opendp.typing`, including `dp.i32`, `dp.f64` and `dp.Vec[dp.i32]`, provide the same types with editor completion.

```
import opendp.prelude as dp

print(dp.atom_domain(T=float))  # AtomDomain(nan=true, T=f64)
print(dp.atom_domain((0, 1), T=dp.i32))  # AtomDomain(bounds=[0, 1], T=i32)
print(dp.RuntimeType.parse(list[int]))  # Vec<i32>
print(dp.Vec[dp.i32] == dp.RuntimeType.parse(list[int]))  # True
```

Python aliases have fixed widths: `int` means 32-bit and `float` means 64-bit. `dp.RuntimeType.infer` uses the same mapping. NumPy's default integer is 64-bit, so an array value has type `i64` and does not match a domain built with `T=int`. If your integers are 64-bit, use `set_default_int_type` to change the alias.

```
import numpy as np
import opendp.prelude as dp

print(dp.RuntimeType.parse(int))  # i32
print(dp.RuntimeType.infer(np.int64(1)))  # i64
dp.set_default_int_type(dp.i64)
print(dp.RuntimeType.parse(int))  # i64
print(dp.vector_domain(dp.atom_domain(T=int)))  # VectorDomain(AtomDomain(T=i64))
```

When the types do not meet, the error reads `inferred type is XXX, expected YYY`, which the documentation's troubleshooting page lists. The script builds a Laplace mechanism on an integer domain with an integer metric, and then asks for its privacy map at a sensitivity of 1.5.

```
int_space = dp.atom_domain(T=int), dp.absolute_distance(T=int)
laplace_int = int_space >> dp.m.then_laplace(scale=1.0)  # Laplace noise on a 32-bit
 ↪  integer
try:
    laplace_int.map(1.5)  # a float sensitivity, on a metric whose distances are
 ↪  integers
    message = "no error"
except TypeError as err:
    message = first_line(err)
```

```
map(1.5) on an integer metric: TypeError: inferred type is f64, expected i32
map(2) on an integer metric: 2.0
float d_in on the count: TypeError: inferred type is f64, expected u32
integer d_in on the count: 0.3333
```

The error reports the inferred and expected types; here, pass integer 2. The next lines show a common mismatch: symmetric distance uses unsigned integer distances, so `map(1.0)` fails and `map(1)` succeeds. Another mismatch occurs when a scalar ε is passed to a measure that expects an (ε, δ) pair. Version 0.16.0 reports this differently from the documentation, as shown below.

```
scalar epsilon on an approximate measure: TypeError: Cannot coerce a non-tuple type to
 ↪  a tuple
pair on an approximate measure: True
```

The chapter on limitations and troubleshooting covers other errors. Read the expected type and adjust the argument, bounds or data. Do not change the domain to fit the argument; OpenDP chose that type for the proof.

12.2. Parameter search

A measurement has three parameter types: input distance `d_in` (the privacy-unit bound), output distance `d_out` (the budget) and constructor arguments such as noise scale. With these fixed, `check(d_in, d_out)` tests whether the measurement meets the budget, as explained in the measurements chapter. The result is monotone: any larger scale or budget also passes. Binary search locates the boundary and solves for one parameter from the others.

OpenDP provides three searches. `dp.binary_search` takes a predicate and returns the value nearest the boundary on the passing side. `dp.binary_search_param` takes a function that builds a chain from one parameter and uses `check` with `d_in` and `d_out`. `dp.binary_search_chain` performs the same search and returns the chain at the selected value.

```
import opendp.prelude as dp

print(dp.binary_search(lambda x: x >= 5.0))  # 5.0
print(dp.binary_search(lambda x: x > 5, T=int))  # 6
print(dp.binary_search(lambda x: x <= 5, bounds=(0, None)))  # 5
```

The three lines show the arguments of a search. Without bounds, the search first finds bounds by steps outwards from zero. The type `T` says whether the parameter is a float or an integer, and bounds with `None` on one side give a one-sided search. The script then runs the search of a context on the first chapter's Laplace mechanism, with a share of $\varepsilon/3$.

```
scale = dp.binary_search_param(make_laplace, d_in=1.0, d_out=share)  # the smallest
 ↪  scale within the share
chain = dp.binary_search_chain(make_laplace, d_in=1.0, d_out=share)  # the measurement
 ↪  at that scale
largest_d_in = dp.binary_search(lambda d_in: make_laplace(1.0).check(d_in, EPSILON))  #
 ↪  the other way round
```

```
search: scale=3.0000000000000004 chain.map(1.0)=0.3333 chain.check(1.0, share)=True
search: largest d_in at scale 1 within epsilon 1: 1.0
search: gaussian scale for rho=1 at d_in=1: 0.7071
```

The scale is the smallest one whose map at a sensitivity of 1 is within a third of $\varepsilon = 1$. It is a little above 3, because the search stops on the passing side of the boundary, and it is the scale that `param()` reported in the first chapter. The third call asks for the largest sensitivity that a scale of 1 hides within $\varepsilon = 1$, which is 1. The Gaussian mechanism is measured in zero-concentrated differential privacy (zCDP), with a loss ρ of half the squared sensitivity over the squared scale. So the search for $\rho = 1$ gives a scale of 0.7071 (the full edition's chapter on the two mechanisms).

To show what the search does, the script writes it by hand. The predicate is `check`, the interval starts at 0 to 100, and each step keeps the half that contains the boundary.

```
def search_by_hand(passes: Callable[[float], bool], low: float, high: float, tolerance:
 ↪  float) -> tuple[float, int]:
    """Bisection on a predicate that fails at `low` and passes at `high`: the smallest
     ↪  passing value, and the steps."""
    steps = 0
    while high - low > tolerance:
        middle = (low + high) / 2
        if passes(middle):
            high = middle  # the boundary is at or below the middle
        else:
            low = middle  # the boundary is above it
        steps += 1
    return high, steps
```

```
search by hand: scale=3.000000 after 27 steps
search with bounds (0, 100): 3.0000000000000004
```

After twenty-seven halvings, the interval of 100 is below a millionth, and its passing end is the scale. The library's search on the same bounds lands on the float just above 3, so it bisects further than that tolerance. A search can fail in two ways, and the script shows both messages.

```
search with bounds (0, 1): ValueError: the decision boundary of the predicate is
 ↪  outside the bounds
search on a constant predicate: ValueError: unable to infer bounds
```

Bounds that do not contain the boundary give the first error, so make them wider. A predicate that never changes gives the second, here one that always passes. With a real chain, it means that no value of the parameter changes the answer, so look at the chain before the bounds.

12.3. Serialization

A measurement is a recipe that names its constructors and their arguments. The function `dp.serialize` writes that recipe as a JSON string, and `dp.deserialize` turns it back into a measurement. So an analyst can build and inspect a query on a machine without the data, and send the text to the machine that holds them. The script serializes the first chapter's count measurement.

```
measurement = count_laplace(3.0)
text = dp.serialize(measurement)  # JSON that names each constructor and its arguments
restored = dp.deserialize(text)
same_space = restored.input_space == measurement.input_space
```

```
serialized: 2152 characters, starting {"__function__": "make_chain_mt",
restored: map(1)=0.3333 same input space: True
```

The text starts with the outermost constructor, the chain of a transformation and a measurement, and nests the rest inside it. The restored measurement has the same input space and privacy map, so a reviewer can check the guarantee before anyone touches the data. The documentation does not promise a stable format between versions, and it asks you not to write it by hand.

Two things do not survive a round trip. The first is the `param` attribute that a context attaches to a measurement after a search (a later section). It is a Python attribute and not part of the recipe. The second is anything with Python state that the constructors did not set, such as a context, a query or a plugin built from a Python function.

```
param before a round trip: 3.0000000000000004
param after a round trip: None
serialize a context: OpenDP JSON Encoder does not handle instances of <class
 ↪  'opendp.context.Context'>
```

A Polars query has its own route, because Polars can serialize the plan of a `LazyFrame` to bytes. The method `Context.deserialize_polars_plan` wraps those bytes in a query on a context of its own. So a client can write the plan against the survey's schema, and the server can run it on the rows.

```
client = lfs_context(pl.LazyFrame(schema=frame.collect_schema()))  # the schema, but no
 ↪  rows
plan = client.query().select(dp.len(signed=True)).polars_plan.serialize()  # bytes
server = lfs_context(frame)  # the real data
query = server.deserialize_polars_plan(plan)
summary = query.summarize(alpha=0.05)
```

```
plan: 1792 bytes
plan on the server: scale=36.0 95% accuracy=108.3429
```

The client's context holds an empty frame with the survey's schema, and the server's holds the file. Both must state the budget and the privacy unit of 36 rows per person (the chapter on the data and the context). The plan carries the query but not the guarantee. The scale of 36 is the unit's distance over the budget, and the error bar is about 108 rows on a count of millions.

12.4. The compositor behind a context

The rest of the chapter rebuilds the Context API from the combinators of the previous chapter. The constructor `dp.Context.compositor` builds an **adaptive composition** over the data's space and calls it on the data. That composition takes a list of budgets, the `d_mids`, and accepts one measurement per share, each of which may depend on the earlier answers. Together they stay within the sum (the full edition's chapter on composition). The script reads the context's accountant and builds the same one by hand.

```
by_hand_accountant = dp.c.make_adaptive_composition(
    input_domain=SPACE[0],
    input_metric=SPACE[1],
    output_measure=dp.max_divergence(),
    d_in=1,  # the privacy unit's distance: one row added or removed
    d_mids=[share] * QUERIES,  # the shares, which add up to the budget
)
queryable = by_hand_accountant(data)  # answers measurement queries on the data, and
 ↪  keeps count
```

```
context accountant: VectorDomain(AtomDomain(nan=true, T=f64)) SymmetricDistance()
 ↪  MaxDivergence
context d_in: 1  d_mids: [0.3333, 0.3333, 0.3333]
context accountant map(1): 1.0000
by hand accountant map(1): 1.0000  queryable: Queryable(Q=AnyMeasurement)
d_mids by weights [1, 2, 1]: [0.25, 0.5, 0.25]
```

The context found the domain from the data, a list of floats, and the metric and the measure come from `unit_of` and `loss_of`. Both compositions map the unit's distance to the whole budget, 1. On the data, the accountant returns a **queryable**, which answers one measurement at a time and refuses when the shares run out. The context keeps that queryable, and the data are only inside it, which is why the first chapter told you to drop your own reference.

The shares come from `split_evenly_over` or `split_by_weights`. The context gives the weights to `binary_search_param` and scales them until the composition's map at `d_in` meets the budget. For pure differential privacy, that is the expected proportional split: weights of 1, 2 and 1 give a quarter, a half and a quarter of ε.

12.5. The query behind a release

The call `context.query()` returns a **Query**, which holds a chain and the share it must fit, and on a LazyFrame it is the `LazyFrameQuery` of the Polars chapters. The chain starts as the data's space, and each method you call, such as `count` or `laplace`, chains on the constructor of that name. So `query.count()` is `space >> dp.t.then_count()`. The script shows the type of the chain at each step.

```
query = plain_query(context)  # the chain is the data's space: a domain and a metric
counted = query.count()  # the chain is a Transformation
noisy = counted.laplace()  # no scale: the chain is a PartialChain, waiting for one
 ↪  number
is_partial = isinstance(noisy._chain, dp.PartialChain)
```

```
query chain: tuple -> Transformation -> PartialChain
partial chain: True
resolve a transformation: ValueError: Query is not yet a measurement or odometer.
```

The important step is `laplace()` without a scale. The query finds one argument missing from the constructor's signature, so it wraps the constructor in a **partial chain**, a function from the missing number to the finished chain. Only one number may be missing at a time, so a second noise step without a scale fails. A query that is still a transformation cannot resolve into a measurement, as the third line shows.

The method `resolve` finds the number. It calls the partial chain's `fix`, which runs `binary_search` with the predicate "the chain at this value checks within the query's share". That is the search of Section 12.2, and `fix` stores the value on the measurement as `param`. So `param()` reports the scale of the search, or `None` when there was no search.

```
param of .laplace(): 3.0000000000000004
param of .laplace(scale=3.0): None
two missing parameters: ValueError: gaussian is missing 1 parameter(s).
```

The script then runs the same search by hand on the same chain, and releases through the queryable built by hand.

```
by_hand_param = dp.binary_search_param(count_laplace, d_in=1, d_out=share)  # what
 ↪  resolve runs on the chain
by_hand_measurement = count_laplace(by_hand_param)
dp_count_by_hand = queryable(by_hand_measurement)  # what release does: the queryable
 ↪  runs the measurement
```

```
resolve: context param=3.0000000000000004 by hand=3.0000000000000004 equal=True
d_mids after context(measurement): [0.3333, 0.3333]
```

The two scales are equal, because they come from the same search on the same predicate. The method `release` gives the resolved measurement to the queryable and records the share as spent, so the first share leaves `d_mids`. The version by hand does the same with `queryable(measurement)`, but nothing records the spend. That record is what the context adds.

12.6. Nested compositors and other measures

Two features of the Context API are easier to understand when you can see the chain. The first is `Query.compositor`, which nests an adaptive composition inside one share. The query's share becomes the inner budget, split again, and the release of the query returns a new `Context` and not a number.

```
inner = plain_query(context).compositor(split_evenly_over=2).release()  # a Context
 ↪  inside one share
inner_scale = plain_query(inner).count().laplace().param()
```

```
inner context: d_in=1 d_out=0.3333 d_mids=[0.1667, 0.1667]
inner count scale: 6.0000
outer d_mids after the inner compositor: [0.3333]
```

The inner context has the outer share, a third of ε, as its whole budget and splits it in two, so a count inside it gets a scale of 6. The outer context counts the nested compositor as one query, whatever happens inside it, because the composition's guarantee covers everything its queryable answers.

The second feature is a budget in one measure for a mechanism proved in another. The Laplace mechanism's measure is the max divergence of pure differential privacy, and the Gaussian mechanism's is zCDP. When the context's measure is different, `resolve` casts the measurement with the converters of Combinators before it checks the budget.

```
approx = dp.Context.compositor(
    data=data,
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),  # approximate differential
 ↪  privacy
    split_evenly_over=QUERIES,
)
laplace_cast = plain_query(approx).count().laplace().resolve()  # pure DP, cast to
 ↪  approximate DP
gaussian_cast = plain_query(approx).count().gaussian().resolve()  # zCDP, converted and
 ↪  its delta fixed
```

```
approx share: (0.3333, 3.3e-08)
laplace under (epsilon, delta): Approximate(MaxDivergence) map(1)=(0.3333, 0.0e+00)
gaussian under (epsilon, delta): Approximate(MaxDivergence) map(1)=(0.3333, 3.3e-08)
param after a cast: None
```

Under an (ε, δ) budget, `make_approximate` wraps the Laplace mechanism and reports $(\varepsilon, 0)$, so the search fits only ε. The Gaussian mechanism goes through `make_zCDP_to_approxDP` and `make_fix_delta`, which spend the share's slack to convert ρ into ε. But the cast wraps the old measurement in a new one, so the `param` attribute is on the inner measurement, and `param()` reads `None`. To see the scale, run `binary_search_param` on the chain yourself.

```
laplace under rho: ZeroConcentratedDivergence map(1)=0.0333
gaussian under rho: scale=3.8730
gaussian under epsilon: ValueError: Unable to cast measure from
 ↪  ZeroConcentratedDivergence to MaxDivergence
```

Under a ρ budget, the Laplace mechanism is cast with `make_pureDP_to_zCDP`, and the Gaussian mechanism needs no cast, so its `param()` reports a scale of 3.873. A Gaussian mechanism under a pure ε budget cannot be cast, because zCDP never implies pure differential privacy, and the context says so. The full edition's chapter on concentrated differential privacy derives the conversions behind each cast.

Here are the releases of one run, to read against their error bars.

```
released (one run):
  survey rows 3811875  (true 3811932, within 108.3 with probability 0.95)
  count by hand 94  (true 100, within 9.4 with probability 0.95)
  count through the context 101  (true 100)
```

All three are within their bars. The two counts of the hundred integers came from the same measurement, through the queryable built by hand and through the context. So they differ only by their noise.

12.7. Summary

Table 12.1.: The utilities and the pieces of the Context API, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`"i32"`, `int`, `dp.i32`	Three spellings of one type descriptor	`int` is `i32` and `float` is `f64`; NumPy's integers are `i64`
`dp.set_default_int_type(dp.i64)`	Changes what `int` means for the rest of the program	The clean fix when all your integers have 64 bits
`inferred type is XXX, expected YYY`	Your argument's type does not match the metric's or the domain's	Change the argument to the expected type, not the domain to the argument
`dp.binary_search(predicate, bounds, T)`	The value nearest the boundary of a monotone predicate	Pass bounds when the chain is only valid in a range, and `T=int` for an integer
`dp.binary_search_param(make_chain, d_in, d_out)`	The smallest scale whose chain checks within the budget	What a context runs for `.laplace()` without a scale
`dp.binary_search_chain(...)`	The chain built at the value the search found	It carries no `param`; use `binary_search_param` for the number
`dp.serialize`, `dp.deserialize`	A measurement as JSON, and back	The recipe survives; `param`, contexts and Python plugins do not
`Context.deserialize_polars_plan(bytes)`	A Polars plan from a client, run on the server's context	The plan carries the query; each context states the guarantee
`dp.Context.compositor(...)`	An adaptive composition over the data's space, called on the data	Its queryable holds the data; `d_mids` are the shares
`context.query()`	A `Query` whose chain grows with each constructor	A missing number makes a partial chain; one at a time
`query.param()`	The scale that `resolve` found	`None` when the scale was given or the measure was cast
`Query.compositor(...)`	A context nested inside one share	The outer context counts it as one query
A Gaussian query under a pure budget	`ValueError: Unable to cast measure ...`	zCDP never implies pure differential privacy; state the budget with a δ or as ρ

12.8. References

- OpenDP documentation: “Typing”, “Parameter Search” and “Serialization” under “Utilities” in the API user guide; “Context” and “API Comparison” in the same guide; and the entry `Inferred type is XXX, expected YYY` of “Troubleshooting” in Getting Started.
- Gaboardi, Hay and Vadhan, “A Programming Framework for OpenDP” (2020): the domains, metrics, measures and maps that the type descriptors and the searches work on.
- Dwork, Rothblum and Vadhan, “Boosting and Differential Privacy” (2010): composition of adaptively chosen mechanisms, which the compositor behind a context implements.
- Bun and Steinke, “Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds” (2016): zCDP and its conversions to approximate differential privacy, which the measure casts use.
- The full edition’s chapters on the Laplace and Gaussian mechanisms in the full edition, composition in the full edition and concentrated differential privacy in the full edition derive the results this chapter states.

Part IV.

Extending the library

OpenDP's framework accepts built-in and user-defined transformations and measurements with stability or privacy maps. A plugin preserves privacy only if its map is valid; you must prove that validity. The framework then supports chaining and accounting.

Plugins implements a transformation and measurement in Python, registers Context API constructors, and reproduces the documentation's examples: Theil-Sen regression and private selection of grouping columns. Statistical modelling fits differentially private linear regression and principal component analysis models and interprets their guarantees. Limitations and troubleshooting covers limitations such as floating-point arithmetic and side channels, feature flags and common errors.

Table 12.2.: The chapters of this part and the questions they answer.

Chapter	Question it answers
Plugins	How do I add my own transformation or measurement, and what must I prove about it?
Statistical modelling	How do I fit a private regression or a private principal component analysis with the library's estimators?
Limitations and troubleshooting	What does the library not guarantee, and what does each common error mean?

By the end of this part, you can implement a mechanism, fit the built-in models and identify your remaining deployment responsibilities.

13. Plugins

OpenDP may not include the mechanism you need, such as a new estimator from a paper. The framework allows user-defined transformations with stability maps and measurements with privacy maps. You can implement them in Python as **plugins** and chain, compose or run them through a context like built-in components.

A plugin's privacy depends entirely on its map, which OpenDP cannot verify. Built-in constructors include written proofs; plugin authors must provide them. The examples explain each map or identify what remains unproved. They cover measurements, transformations, custom spaces, Context API registration, Theil-Sen regression and private selection of grouping columns.

The script opendp_plugins.py runs on OpenDP 0.16.0. The regression uses the documentation's toy data: 100 points on a line with seeded normal noise. Grouping-column selection uses the Labour Force Survey (LFS) sample. Maps, checks, errors, scores and candidate ranges are deterministic. OpenDP's noise cannot be seeded, so the page labels the released values as one run.

```
uv run python code/part04-extending/opendp_plugins.py
```

13.1. Trust and the honest-but-curious flag

Plugins let the library support mechanisms it cannot ship and let users prototype in Python. Because Python functions are central to their guarantees, OpenDP cannot inspect them. It cannot verify the privacy map, detect side effects or identify errors that occur only for some data.

Plugin constructors therefore require the `honest-but-curious` feature flag in addition to `contrib`. The trust model assumes the author has proved the map and protects against downstream curiosity. Without the flag, a plugin constructor refuses to build.

```
without the flag: Attempted to use function that requires "contrib",
 ↪  "honest-but-curious", but not enabled
```

Each constructor's docstring states its guarantee: inputs at most `d_in` apart must produce output distributions at most `privacy_map(d_in)` apart. The function must have no side effects, and the map must be pure. An incorrect map silently invalidates the guarantee because the context charges the reported cost. The snippet demonstrates the extreme case: a measurement returns its input but claims zero privacy loss.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
leak = dp.m.make_user_measurement(*space, dp.max_divergence(), function=lambda x: x,
 ↪  privacy_map=lambda d: 0.0, TO=list[int])
print(leak.map(1))  # 0.0
print(leak([1, 2, 3]))  # [1, 2, 3]
```

OpenDP accepts this plugin, so a context would release raw data at no cost. The flag records that the guarantee depends on the author's claim; each map below needs a justification.

13.2. A measurement plugin

The documentation first demonstrates a measurement that ignores its input and returns a constant. It is not useful as a release, but its map is easy to prove. `make_user_measurement` takes the five components described in A framework to understand differential privacy: input domain, input metric, output measure, function and privacy map.

```
def make_base_constant(constant: str) -> dp.Measurement:
    """The documentation's example: a measurement that ignores its input and returns a
     ↪  constant."""

    def function(_arg: int) -> str:
        return constant  # the output does not depend on the data

    def privacy_map(_d_in: int) -> float:
        return 0.0  # so two neighbouring inputs give the same distribution: a privacy
         ↪  loss of zero

    return dp.m.make_user_measurement(
        input_domain=dp.atom_domain(T=int),
        input_metric=dp.absolute_distance(T=int),
        output_measure=dp.max_divergence(),
        function=function,
        privacy_map=privacy_map,
        TO=str,
    )
```

The input is one integer under absolute distance. Max divergence is the output measure, so the map reports ε. It returns zero at every distance because the output is independent of the input. The `TO` argument names the function's return type. The script chains this measurement after a bounded integer sum.

```
constant: input=AtomDomain(T=i32) metric=AbsoluteDistance(i32) measure=MaxDivergence
sum >> constant: map(1)=0.0 release=denied
```

The sum outputs an integer under absolute distance, matching the plugin's input space. OpenDP checks this match because it owns the domain and metric objects; it does not check the map.

The next constructor adds Laplace noise with NumPy and supplies a privacy map. Its first two arguments are the input domain and metric, as required for Context API registration later.

```
def make_numpy_laplace(input_domain: dp.Domain, input_metric: dp.Metric, scale: float)
 ↪  -> dp.Measurement:
    """Laplace noise drawn by NumPy instead of OpenDP: the same map as the built-in,
     ↪  but on your word."""
    rng = np.random.default_rng()

    def function(arg: float) -> float:
        return float(arg + rng.laplace(scale=scale))

    def privacy_map(d_in: float) -> float:
        return d_in / scale  # the Laplace mechanism's loss: the sensitivity over the
         ↪  scale

    return dp.m.make_user_measurement(
        input_domain, input_metric, dp.max_divergence(), function=function,
 ↪  privacy_map=privacy_map, TO=float
    )
```

The map divides sensitivity by scale, the Laplace cost in ε (the full edition's chapter on the Laplace and Gaussian mechanisms derives this result). At the same scale, built-in `then_laplace` reports the same map. Compare plugin maps with built-in equivalents when available.

```
python laplace scale 2: map(1.0)=0.5 built-in map(1.0)=0.5
halved map, same noise: map(1.0)=0.25 check(1.0, 0.25)=True
laplace scale 2: 95% accuracy=5.9915
```

The second line keeps the noise but claims half its true cost. OpenDP accepts the map, and `check` incorrectly approves a budget of 0.25; a context would allocate a quarter while spending half. The third line gives the 95% error bar for scale 2. In production, prefer the Rust core: NumPy's floating-point sampler leaks slightly through rounding, while OpenDP's samplers prevent this.

13.3. A transformation plugin

A transformation plugin takes one more argument than a measurement, an output domain. Its map is a stability map, which bounds the distance between two outputs when the inputs are a given distance apart (Transformations). The documentation's example repeats each record a fixed number of times.

```
def make_repeat(multiplicity: int) -> dp.Transformation:
    """The documentation's example: a transformation that repeats each record
     ↪  `multiplicity` times."""
```

```
    def function(arg: list[int]) -> list[int]:
        return arg * multiplicity

    def stability_map(d_in: int) -> int:
        return d_in * multiplicity  # one added record of the input is `multiplicity`
         ↪  added records of the output

    return dp.t.make_user_transformation(
        input_domain=dp.vector_domain(dp.atom_domain(T=int)),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.vector_domain(dp.atom_domain(T=int)),
        output_metric=dp.symmetric_distance(),
        function=function,
        stability_map=stability_map,
    )
```

Both metrics are the symmetric distance, the number of records added or removed. The map multiplies that distance by the multiplicity, because each input record becomes `multiplicity` output records. The output domain is a second claim. Every output must belong to it, or the function must fail in a way that does not depend on the data. Here it holds, because the output is a vector of integers whenever the input is.

The documentation chains the plugin between built-in steps, a cast from strings and a clamp before a sum, and ends with Laplace noise.

```
repeat 2: [1, 2] -> [1, 2, 1, 2] map(1)=2 map(3)=6
cast >> repeat 2 >> clamp (1, 2) >> sum: map(1)=4 on ['0', '1', '2', '3'] = 12
... >> laplace scale 1: map(1)=4.0
domain mismatch: DomainMismatch("Intermediate domains don't match
```

The stability of a chain is the product of the stabilities of its links (Combinators). The cast and the clamp have stability 1 and the repeat has 2. The clamp bounds every value by 2, so one added input record, repeated twice, moves the sum by at most 4. Laplace noise of scale 1 then costs an ε of 4. In the last line the cast gives floats, and the library refuses the chain, because it can compare its own spaces.

The library cannot compare the function with the output domain, and that is the second way a transformation plugin goes wrong. The snippet declares that its output lies between 0 and 1 and then returns its input unchanged.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
ints = dp.vector_domain(dp.atom_domain(T=int))
bounded = dp.vector_domain(dp.atom_domain(bounds=(0, 1)))
claim = dp.t.make_user_transformation(ints, dp.symmetric_distance(), bounded,
 ↪  dp.symmetric_distance(), function=lambda x: x, stability_map=lambda d: d)
print(claim([5, 6]))  # [5, 6]
print((claim >> dp.t.then_sum()).map(1))  # 1
```

The sum trusts the bounds and reports a sensitivity of 1, but the real sum moves by the whole value of a record. A wrong output domain becomes a wrong map one step later, so a transformation plugin needs a justification for each.

13.4. Your own domains, metrics and measures

A plugin may work on data that the library has no domain for, such as a data frame, or compare neighbours by a metric that the library does not define. Three constructors let you define your own, and each carries a promise.

A **user domain** has an identifier and a membership function. The library treats two user domains as equal when their identifiers are equal, and uses that to decide whether two steps chain. So the identifier must name one set only. Otherwise two different domains chain, and the chain's stability no longer holds. The membership function must return true only for members. A **user distance** is a metric that is only a name. It promises that your distance is a pseudo-metric, which is zero from a point to itself, never negative, symmetric, and obeys the triangle inequality. The snippet declares a domain of even integers and an edit distance on strings, and measures the length of a string.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
even = dp.user_domain("EvenInts", member=lambda x: x % 2 == 0)
print(even.member(4), even.member(3))  # True False
edits = dp.user_distance("EditDistance")
length = dp.t.make_user_transformation(dp.atom_domain(T=str), edits,
 ↪  dp.atom_domain(T=int), dp.absolute_distance(T=int), function=len,
 ↪  stability_map=lambda d: d)
print(length("hello"), length.map(1))  # 5 1
print((length >> dp.m.then_laplace(scale=1.0)).map(1))  # 1.0
```

The stability map of the length is the identity, because one edit changes the length by at most one. So the chain into Laplace noise closes, and a scale of 1 costs an ε of 1 per edit.

A **user divergence** is a privacy measure of your own. Post-processing must not increase it, because a release must stay private under further computation. The full edition's chapter on properties proves this for the max divergence. The function `new_privacy_profile` turns a curve from ε to the slack δ into a privacy profile that the library can invert. And `new_function` wraps a Python post-processor, as Combinators did for a bare function after `>>`. The post-processor must not read global state or raise an error that depends on the data, because either opens a channel from the data that no map accounts for.

```
import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
renyi = dp.user_divergence("RenyiDivergence(alpha=2)")
rng = np.random.default_rng()
```

```
gaussian = dp.m.make_user_measurement(*space, renyi, function=lambda x: float(x +
 ↪  rng.normal()), privacy_map=lambda d: d**2, TO=float)
print(gaussian.map(1.0), gaussian.output_measure)  # 1.0 RenyiDivergence(alpha=2)
try:
    dp.c.make_composition([gaussian, gaussian])
except dp.OpenDPException as err:
    print(str(err).split(". See")[0].strip())  # Type("No match for concrete type
     ↪  ExtrinsicDivergence
profile = dp.new_privacy_profile(lambda eps: 1.0 if eps < 0.5 else 1e-8)
print(profile.delta(epsilon=0.499), profile.epsilon(delta=1e-8))  # 1.0 0.5
double = dp.new_function(lambda x: 2 * x, TO=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print((laplace >> double).map(1.0))  # 1.0
```

The measurement adds standard normal noise. Under the Rényi divergence of order 2 its map is the squared sensitivity. The reason is that this divergence between two unit Gaussians is the squared distance between their centres (the full edition's chapter on Rényi differential privacy). The composition fails, because the combinators add only the measures that the library defines, so a plugin measure gets `map` and `check` and nothing more. The privacy profile answers in both directions, the slack at a given ε and the ε at a given slack, and its curve must decrease. The post-processor leaves the map of the Laplace measurement unchanged.

13.5. Registering a constructor for the Context API

A plugin built with the framework API is a loose measurement, and A first release showed why every query should go through a context. The function `dp.register` adds a constructor to the Context API as a method of `context.query()`. The constructor's name must start with `make_`, and the method has the name without that prefix, unless you pass a `name`. If its first two parameters are `input_domain` and `input_metric`, the context fills them in with the output space of the previous step. The documentation calls that partial application, and it gives every built-in `make_` constructor its `then_` form.

```
def make_anything_constant(input_domain: dp.Domain, input_metric: dp.Metric, constant:
 ↪  str) -> dp.Measurement:
    """The documentation's constructor for the Context API: it accepts whatever space
     ↪  the previous step outputs."""

    def function(_arg: Any) -> str:
        return constant

    def privacy_map(_d_in: Any) -> float:
        return 0.0

    return dp.m.make_user_measurement(
        input_domain, input_metric, dp.max_divergence(), function=function,
 ↪  privacy_map=privacy_map, TO=str
    )
```

```
dp.register(make_anything_constant)  # from now on,
 ↪  context.query().anything_constant(...)
```

The script builds the documentation's context over three integers, with a budget of $\varepsilon = 1$ split over two queries. Its privacy unit is the book's 36 contributions per person (the data and the context). Then the script calls the plugin through it.

```
anything_constant: map(36)=0.0 release=denied
budget spent by the plugin query: [0.5]
clamp >> sum >> int_constant: release=denied
register twice: 'anything_constant' is already registered in the Context API. Please
 ↪  choose a different name.
register without make_: constructor.__name__ must start with 'make_', found select_cols
```

On the second line the map of the plugin is zero, but the context charged the query its full share. The reason is that a context splits the budget before it sees a query (Utilities and the Context API inside). So a cheap plugin does not give the difference back.

The third line uses the documentation's other style, `make_int_constant`, without the two leading parameters. Its input space is fixed to integers, so it works only after a clamp and a sum. The documentation recommends the first style, but a constructor that accepts any space should check that the space suits its function. The last two lines are the errors of `register` for a name already taken and for a name without the prefix.

13.6. A Theil-Sen regression

The documentation's first larger example fits a line through noisy points. Least squares is a poor private estimator, because one far-away point can move the line by any amount. The **Theil-Sen** estimator takes the median of the slopes through pairs of points instead, and OpenDP already releases medians privately. The documentation's variant pairs the points at random and fits a line through each pair. It records where each line crosses two vertical cuts, at a quarter and three quarters of the range of x, and the medians at the cuts are two points on the fitted line. The library ships a vetted version of this estimator (Statistical modelling), so this one is for learning how plugins, your own transformations and measurements, combine.

The first plugin is the pairing, a transformation from a two-column array to a two-column array of predictions.

```
def pairwise_predict(data: NDArray[np.float64], x_cuts: NDArray[np.float64]) ->
 ↪  NDArray[np.float64]:
    """Pair the rows at random, fit a line through each pair, and predict y at the two
     ↪  cut points."""
    data = np.array(data, copy=True)[: len(data) // 2 * 2]
    np.random.shuffle(data)
    p1, p2 = np.array_split(data, 2)
    dx, dy = (p2 - p1).T
    x_bar, y_bar = (p1 + p2).T / 2
```

```
    points = dy / dx * (x_cuts[None].T - x_bar) + y_bar
    return points.T[dx != 0]
```

The transformation wraps that function.

```
def make_pairwise_predict(x_cuts: NDArray[np.float64], runs: int = 1) ->
↪  dp.Transformation:
    """A transformation from the (x, y) rows to the predictions: one row per pair and
    ↪  run."""

    def function(arg: NDArray[np.float64]) -> NDArray[np.float64]:
        return np.vstack([pairwise_predict(arg, x_cuts) for _ in range(runs)])

    def stability_map(d_in: int) -> int:
        return d_in * runs  # each record sits in one pair, so it changes one
        ↪  prediction per run

    return dp.t.make_user_transformation(
        input_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        output_metric=dp.symmetric_distance(),
        function=function,
        stability_map=stability_map,
    )
```

The domains, from the library's NumPy extension, are arrays with two columns. The documentation states the map, the input distance times the number of runs, without its argument: each record sits in one pair, so it changes at most one prediction per run. That claim is weaker than a built-in map in two ways. The pairing is random, so the claim is about a coupling of two runs, not about a function. And the function drops the last record when the count is odd, so removing one record can change the pair of another. That is the kind of gap the flag asks you to notice.

The second plugin selects one column of the predictions, and the library's quantile does the rest.

```
def make_select_column(j: int) -> dp.Transformation:
    """Column j of the predictions as a vector; one row in is one row out, so the
    ↪  stability is the input distance."""

    def function(arg: NDArray[np.float64]) -> NDArray[np.float64]:
        return arg[:, j]

    def stability_map(d_in: int) -> int:
        return d_in

    return dp.t.make_user_transformation(
        input_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.vector_domain(dp.atom_domain(T=float)),
        output_metric=dp.symmetric_distance(),
        function=function,
```

```
        stability_map=stability_map,
    )
```

The library's quantile then runs once for each cut.

```
def make_private_medians(output_measure: dp.Measure, y_bounds: tuple[float, float],
 ↪  scale: float) -> dp.Measurement:
    """The private median of the predictions at each cut, by the library's quantile,
     ↪  composed."""
    median = dp.m.then_private_quantile(output_measure,
 ↪  candidates=np.linspace(*y_bounds, 100), alpha=0.5, scale=scale)
    return dp.c.make_composition(
        [
            make_select_column(0) >> dp.t.then_drop_null() >> median,
            make_select_column(1) >> dp.t.then_drop_null() >> median,
        ]
    )
```

Selecting a column keeps one row per row, so its stability is the input distance. The private quantile scores a hundred candidates between the public bounds of y and picks one by noisy max (Measurements), at a cost of the input distance over the scale. The composition adds the two costs (the full edition's chapter on composition). Solving for the line is post-processing of two released numbers and costs nothing.

```
def make_private_theil_sen(
    output_measure: dp.Measure, x_bounds: tuple[float, float], y_bounds: tuple[float,
 ↪  float], scale: float
) -> dp.Measurement:
    """Pair and predict, release the two medians, then solve for the slope and the
     ↪  intercept (post-processing)."""
    x_cuts = x_bounds[0] + (x_bounds[1] - x_bounds[0]) * np.array([0.25, 0.75])
    p_inv = np.linalg.inv(np.vstack([x_cuts, np.ones_like(x_cuts)]).T)  # the line
 ↪  through two points

    def to_coefficients(medians: list[float]) -> tuple[float, float]:
        slope, intercept = p_inv @ np.array(medians)
        return float(slope), float(intercept)

    return make_pairwise_predict(x_cuts) >> make_private_medians(output_measure,
     ↪  y_bounds, scale) >> to_coefficients
```

The script runs it with the documentation's bounds and a scale of 1, on the toy points. Their true line has slope 2 and intercept 1.

```
theil-sen: x cuts -1.5 and 1.5, 100 candidates from -10.0 to 10.0 step 0.2020
pairwise predict: map(1)=1 medians: map(1)=2.0 theil-sen: map(1)=2.0 map(2)=4.0
toy data: n=100 least-squares slope=2.0573 intercept=1.0376
pairs: 50 medians at the cuts -2.2424 and 4.2857
selection bound: 26.2 score units at noise scale 2.0
```

```
cut -1.5: best candidate -2.3232 score 2; within the bound: -2.9293 to -1.5152 (8
 ↪  candidates)
cut 1.5: best candidate 4.3434 score 8; within the bound: 3.3333 to 6.3636 (16
 ↪  candidates)
```

The pairing has stability 1 and the two medians cost 2 together, so the estimator costs an ε of 2 per record. The exact medians of the fifty pairwise predictions are the values that the private medians aim at.

The last three lines give the bound for the released medians. The private quantile adds exponential noise of scale 2 to each score and picks the best. The difference of two such draws is a Laplace draw, so the chance that a worse candidate wins falls exponentially in its gap to the best. A union bound over the other 99 candidates gives the printed bound, which a losing candidate beats at most once in ten thousand runs.

13.7. Choosing grouping columns privately

The documentation's second larger example solves a problem that comes before any grouped release. You want to choose grouping columns without looking at the data. Some columns are too uniform and others too diverse, and too many columns leave most groups too small to pass the thresholds of Grouping. So the example scores each candidate set by how many of its groups reach a public minimum of rows. Then it picks a set with the **exponential mechanism**, which chooses a candidate with a probability that grows exponentially in its score (the full edition's chapter on selection).

The score is a transformation plugin on a user domain, because the library has no domain for a whole data frame. The documentation scores a pandas frame, and the same score runs here on a Polars frame.

```
def score(frame: pl.DataFrame, columns: tuple[str, ...], min_rows: int) -> float:
    """How many groups of these columns have at least `min_rows` rows."""
    return float(frame.group_by(list(columns)).len().filter(pl.col("len") >=
     ↪  min_rows).height)
```

The transformation applies it to every candidate set.

```
def make_grouping_cols_score(
    input_domain: dp.Domain, input_metric: dp.Metric, candidates: list[tuple[str,
 ↪  ...]], min_rows: int
) -> dp.Transformation:
    """A transformation from the frame to one score per candidate set of grouping
     ↪  columns."""
    dp.assert_features("contrib")

    def function(frame: pl.DataFrame) -> list[float]:
        return [score(frame, columns, min_rows) for columns in candidates]

    def stability_map(d_in: int) -> float:
```

```
        return float(d_in)  # one row changes one group's size, so it moves each score
         ↪ by at most 1

    return dp.t.make_user_transformation(
        input_domain=input_domain,
        input_metric=input_metric,
        output_domain=dp.vector_domain(dp.atom_domain(T=float, nan=False)),
        output_metric=dp.linf_distance(T=float),  # not monotonic: a person's rows may
↪  be added and removed
        function=function,
        stability_map=stability_map,
    )
```

The output metric is the L-infinity distance, the largest change in any one score, because noisy max compares scores. The map is the identity, because one row belongs to one group of each candidate, so it moves each score by at most one.

The documentation declares the metric monotonic, which halves the cost of noisy max. That is right when neighbours differ by one row added or one row removed, because every score then moves the same way. Here a person has up to 36 rows, and a neighbouring dataset can add some and remove others, so the scores can move both ways. So the script keeps the metric plain and pays the factor of two (the full edition's chapter on sensitivity).

The measurement chains the scores into noisy max and names the chosen candidate, and it is registered so that a context can run it.

```
def make_select_grouping_cols(
    input_domain: dp.Domain, input_metric: dp.Metric, candidates: list[tuple[str,
↪  ...]], min_rows: int, scale: float
) -> dp.Measurement:
    """Score the candidates, pick one by the exponential mechanism, and name its
     ↪ columns."""

    def name(index: int) -> tuple[str, ...]:
        return candidates[index]  # post-processing: the index is already private

    scores = make_grouping_cols_score(input_domain, input_metric, candidates, min_rows)
    return scores >> dp.m.then_noisy_max(dp.max_divergence(), scale) >> name


dp.register(make_select_grouping_cols)


def frame_domain() -> dp.Domain:
    """A domain for a Polars DataFrame: the library has none, so membership is a Python
     ↪ check."""

    def member(value: object) -> bool:
        return isinstance(value, pl.DataFrame)

    return dp.user_domain("PolarsDataFrameDomain", member=member)
```

On the survey the candidates are every non-empty subset of six columns. Five are `SEX`, `ILOSTAT`, `YEAR`, `QUARTER` and `AGE`, in seven bands, and the sixth is the person identifier `PIDENT`. A group counts in a score when it has at least 100 rows, a public choice. The context is built over the collected frame, with the user domain passed as `domain`, the privacy unit of 36 contributions and a budget of $\varepsilon = 1$. The script leaves out the scale, so that the context searches for one against the map of the plugin.

```
LFS: 3,811,932 rows; candidate columns SEX, ILOSTAT, YEAR, QUARTER, AGE, PIDENT; 63
 ↪  candidate sets
distinct values: SEX 2 ILOSTAT 4 YEAR 9 QUARTER 4 AGE 7 PIDENT 1,682,306
score: groups of at least 100 rows
rank 1: SEX, ILOSTAT, YEAR, QUARTER, AGE score 996 of 1,105 groups
rank 2: ILOSTAT, YEAR, QUARTER, AGE score 504 of 570 groups
rank 3: SEX, YEAR, QUARTER, AGE score 432 of 470 groups
sets with PIDENT: 32, largest score 0
select scale 72: map(36)=1.0 monotonic would be 0.5
context query: param scale=72.0
within 910 of the best: 10 candidates
```

The scores, computed at development time on this public survey, have the shape the documentation describes. The five columns without the identifier score best, and the next set scores about half of that. Every set with `PIDENT` scores zero, because no person has a hundred rows. The map is twice the sensitivity over the scale, so the search finds a scale of 72 for an ε of 1.

By the argument used for the medians, a set that scores more than 910 below the best wins at most once in ten thousand runs. Ten sets are within that bound. The best beats the runner-up by 492, under seven noise scales, so it wins most runs, but any of the ten can appear. That spread is the price of the person-level unit, because a person can move each score by 36 while the scores are only in the hundreds. A higher minimum group size lowers every score and makes the choice more random. A larger budget is the only other lever.

```
released (one run):
  python laplace of 10.0: 9.6979  (within 6.0 with probability 0.95)
  repeated sum of ['0', '1', '2', '3'] with laplace scale 1: 13  (true 12)
  theil-sen: slope 2.1549 intercept 1.1111  (medians at the cuts -2.1212 and 4.3434)
  grouping columns: SEX, ILOSTAT, YEAR, QUARTER, AGE
```

The Python Laplace landed within its error bar, and the repeated sum within one of the truth. Both private medians fell inside their candidate ranges, so the slope and the intercept are near 2 and 1, and the chosen columns are the five that score best. Your run can differ in each of these.

13.8. Summary

Table 13.1.: The plugin constructors, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.enable_features("honest-but-curious")`	Unlocks the plugin constructors	The library cannot check a Python map; the proof is yours
`make_user_measurement(...)`	A measurement from your function and privacy map	Compare the map with a built-in cousin where one exists
`make_user_transformation(...)`	A transformation from your function and stability map	Justify the map and the output domain
`user_domain`, `user_distance`, `user_divergence`	Your own domain, metric and measure	Identifiers name one thing; a user measure cannot be composed
`new_privacy_profile`, `new_function`	A profile from a curve; a post-processor	No global state, no data-dependent exceptions
`dp.register(make_thing)`	`context.query().thing(...)`	Take `input_domain` and `input_metric` first
A plugin in a context	Charged its share of the budget, not its map	A cheap plugin does not give its share back
Theil-Sen from plugins	Pair, predict at two cuts, two private medians, solve	Use the vetted estimator for real work
Grouping columns by noisy max	Score each set by its groups above a minimum, pick one	Monotonic only for one-row units

13.9. References

- OpenDP documentation: “Plugins”, “Measurement example”, “Transformation example”, “Context API Plugins”, “Thiel-Sen Regression” and “Privately Selecting Grouping Columns” in the API user guide.
- Theil (1950) and Sen, “Estimates of the regression coefficient based on Kendall’s tau” (1968): the estimator behind the regression example.
- McSherry and Talwar, “Mechanism Design via Differential Privacy” (2007): the exponential mechanism behind noisy max.
- Gaboardi, Hay and Vadhan, “A Programming Framework for OpenDP” (2020): the maps a plugin must satisfy.
- The full edition’s chapters on the Laplace and Gaussian mechanisms in the full edition, composition in the full edition, selection in the full edition and sensitivity in the full edition derive the results this chapter states.

14. Statistical modelling

Earlier releases reported statistics such as counts, sums, means and quantiles. Some questions concern relationships, such as actual hours worked versus usual hours or the directions that explain a table's variance. OpenDP provides differentially private linear regression and principal component analysis (PCA), with interfaces similar to scikit-learn. A fitted model is a release: its cost is the privacy map behind `fit`, not necessarily the argument named `epsilon`.

This chapter fits regression to the documentation's generated points and the Labour Force Survey (LFS), then reads its privacy loss and sensitivity bound. It also fits PCA to the documentation's generated table and compares it with NumPy's exact decomposition. The final sections cover the NumPy domain and review questions for private models.

The script opendp_statistical_modelling.py runs on OpenDP 0.16.0 and scikit-learn 1.8.0. A seeded NumPy generator creates the toy data and fixes regression pairings, making exact fits, bounds, cut points and privacy maps reproducible. Released fits vary because OpenDP's generator cannot be seeded; the page labels them as one run.

```
uv run python code/part04-extending/opendp_statistical_modelling.py
```

14.1. A model or a table?

The documentation recommends a model when the target relationship is known. A purpose-built model spends the budget on the few parameters needed: a slope costs two private medians, while a table detailed enough to estimate it would require noise on every cell. PCA similarly releases a few directions rather than a full covariance matrix.

Synthetic data are more flexible: they support later analyses and can be shared as microdata. That flexibility reduces accuracy for each question compared with a direct release under the same budget (AIM and MST). Release a table for counts, a model for a known relationship and synthetic data when future questions are unknown.

Both models have narrow scope. Regression accepts one explanatory variable and one response; PCA accepts a numeric table with bounded row lengths. For multiple explanatory variables, use the methods in the full edition's chapter on private empirical risk minimization, which OpenDP does not implement.

14.2. Linear regression

Directly releasing a least-squares line is unsafe: its slope is a ratio of sums with unbounded sensitivity unless both columns are clipped, which changes the line. OpenDP instead provides **Theil-Sen regression**. It randomly pairs rows, fits a line to each pair, and evaluates each line at two fixed x-axis cut points. It privately releases the median reading at each point, then computes the slope and intercept by post-processing at no additional cost.

The private median has bounded sensitivity. It scores candidates on a fixed grid by the number of readings on either side, then uses the exponential mechanism to favour high scores. One person's row enters one pair, changes one reading and changes each candidate's score by at most one. The reading's magnitude does not matter, so the estimator needs no clipping and tolerates extreme reports. The full edition's chapter on selection derives the exponential mechanism and private quantiles; its chapter on sensitivity in the full edition explains why the score change is the relevant bound.

```
def make_regression(
    x_bounds: tuple[float, float], y_bounds: tuple[float, float], scale: float, runs:
 ↪  int = 1
) -> dp.sklearn.linear_model.LinearRegression:
    """The private Theil-Sen regression under pure differential privacy, with public
     ↪  bounds on x and y."""
    return dp.sklearn.linear_model.LinearRegression(
        dp.max_divergence(),  # the output measure: the privacy loss is read in epsilon
        x_bounds=[x_bounds],  # fixes the two cut points, a quarter and three quarters
 ↪  of the way across
        y_bounds=y_bounds,  # the range over which the candidate medians are spread
        scale=scale,  # the noise scale of each of the two private medians
        runs=runs,  # how many random pairings to pool; each multiplies the privacy
 ↪  loss
    )
```

The first argument selects the output measure; `dp.max_divergence()` gives pure differential privacy (DP) with privacy parameter ε. Bounds are public and do not clip data. The x bounds place cut points at the `fraction_bounds` fractions of the range, one quarter and three quarters by default. The y bounds define `candidates_count` evenly spaced median candidates, 100 by default. `scale` sets noise per median; `runs` repeats random pairings and pools readings. Plugins builds the same estimator from plugins.

```
regression space: NPArray2Domain(num_columns=2, nan=True, T=f64) SymmetricDistance()
 ↪  MaxDivergence
regression map: scale=1 runs=1 -> epsilon=2.0
regression map: scale=2 runs=1 -> epsilon=1.0
regression map: scale=2 runs=3 -> epsilon=3.0
```

The estimator's `measurement` attribute exposes its privacy map. Its input is a two-column NumPy array under symmetric distance, so map distance 1 gives the cost of adding or removing one row. Two medians at scale 1 cost $\varepsilon = 2$; doubling the scale halves the cost. Three runs at scale 2 cost

$\varepsilon = 3$ because one row can affect three pairwise readings. More runs define the median better, but require a larger scale at fixed budget and rarely help on large datasets.

The documentation's toy data is a hundred points with a true slope of 2 and an intercept of 1, here from a seeded generator. The script prints the exact values to read a fit against.

```
toy: n=100 cut points=[-1.5, 1.5] candidate step=0.2020
toy least squares: slope=2.0275 intercept=0.9725
toy median line, no noise: slope=2.0379 intercept=1.0786 readings=50
```

The candidate step is the y range divided into 99 intervals, about 0.2. The cut points are 3 apart, so one grid step at one median changes the slope by about 0.07, before any noise. The median line is what the estimator would return with exact medians and the seeded pairing, from the 50 readings of 100 points. It is close to least squares because the noise in the toy data is symmetric.

14.2.1. Actual hours against usual hours on the survey

The survey question is how many hours people actually worked in the reference week for each hour they usually work. The script takes the people at work in the fourth quarter of 2013 with both hours recorded. The survey codes hours from 0 to 98, so the bounds are public, and it drops the code 99, "not applicable". It also keeps the first row of each person, so that the privacy unit of the fit is one person (Identifier truncation). These steps run in Polars before the array reaches the measurement, so they are your claim and not the library's.

```
survey: working people in 2013 Q4 with hours recorded: 36,914 (one row each)
survey: of them with 0 actual hours that week: 4,109
survey: bounds=(0.0, 98.0) cut points=[24.5, 73.5] candidate step=0.9899
survey least squares: slope=0.8582 intercept=-0.1935
survey median line, no noise: slope=1.0000 intercept=0.0000 medians=[24.50, 73.50]
survey readings: 16,463 pairs; within one candidate step of the medians: 5,606 and
 ↪  5,252
```

The two exact fits disagree, and that is the lesson. Of the 36,914 people, 4,109 worked no hours that week, through leave or illness, and they pull least squares down to 0.86 actual hours per usual hour. The median line ignores them, because most pairs of people lie on the line where actual equals usual. So its medians are the cut points themselves, and its slope is exactly 1. A private Theil-Sen fit therefore estimates the typical relation, not the average one, and the median line is the exact value to compare it with. The readings are fewer than half the people, because a pair with equal usual hours has no slope, and thousands of them lie within one grid step of each median.

```
model = make_regression(HOURS_BOUNDS, HOURS_BOUNDS, scale=REGRESSION_SCALE)
epsilon = model.measurement.map(1)  # the privacy loss for adding or removing one
 ↪  person's row
survey_slope, survey_intercept = fit_regression(model, hours, seed)
```

```
survey private fit: scale=2 epsilon=1.0
```

The fit at scale 2 costs $\varepsilon = 1$ for one person. Like scikit-learn, `fit` takes the explanatory column as a two-dimensional array and the response as a vector. It returns a fitted scikit-learn `LinearRegression`, so `coef_`, `intercept_` and `predict` work as usual.

14.3. Principal component analysis

A PCA finds the directions along which a table varies most. OpenDP's version releases three things under one budget. They are the mean of the rows, the singular values, which say how much variance each direction carries, and the components, which are the directions.

The algorithm, from Amin and others (2019), scales every row into a ball of radius `row_norm` and centres the rows on a private mean. Then it forms the matrix of sums of squares and cross-products. It releases the eigenvalues of that matrix with Laplace noise, because replacing one row moves their sum by at most twice the squared row norm. Then it draws the eigenvectors one at a time with the exponential mechanism on the sphere, which favours directions of high variance, and removes each drawn direction from the matrix.

So two facts must be public. The row norm is the sensitivity of the whole algorithm, and every map below is in units of it. The number of rows is part of the domain, which gives the mean a known divisor. So neighbouring tables have the same size with one row replaced, which is distance 2 under the symmetric distance. The library declares both facts and does not check them, as the reviewer's section shows. The budget is split in three equal parts for the mean, the eigenvalues and the eigenvectors, and the third for the eigenvectors is split again over the draws.

The table is the documentation's, ten thousand centred rows of four columns from a normal distribution with a random covariance. A seeded generator makes the exact decomposition reproducible.

```
pca table: 10,000 rows x 4 columns, row length mean=8.53 max=36.56
pca table: rows longer than 1: 99.9%  longer than 40: 0.0%
pca exact, raw rows: singular values=[901.9, 278.6, 222.2, 156.9]
pca exact, rows clamped to 1: singular values=[80.48, 40.67, 34.29, 26.29]
pca exact, rows clamped to 1: first component=[-0.614, -0.534, -0.526, -0.247]
```

The documentation fits with `row_norm=1.0`, and the first two lines show what that means here. Almost every row is longer than 1, so the clamp scales almost every row onto the unit sphere. The algorithm can then only estimate the singular values of the clamped table, about 80 instead of about 900 for the first. The directions survive the clamp, because scaling a row does not turn it, so the release is compared with the first component of the clamped table. The script also fits with a row norm of 40, which clamps nothing, to show the other side of the trade.

```
def make_pca(epsilon: float, row_norm: float, n_components: int | None = None) ->
 ↪  dp.sklearn.decomposition.PCA:
    """The private PCA of the generated table, with the two public facts the algorithm
     ↪  needs."""
```

```
    return dp.sklearn.decomposition.PCA(
        epsilon=epsilon,
        row_norm=row_norm,  # public: every row is scaled into a ball of this radius
↪   before anything else
        n_samples=PCA_ROWS,  # public: the exact number of rows, which fixes the domain
        n_features=PCA_COLUMNS,
        n_components=n_components,  # None releases every component
    )
```

```
pca = make_pca(PCA_EPSILON, ROW_NORM)
measurement = pca.measurement()  # the measurement behind fit(), for its map and for
↪   compositors
pca.fit(table)  # the private release: the mean, the singular values and the components
```

```
pca model: PCA(epsilon=1.0, n_components=4, n_features=4, n_samples=10000,
↪   row_norm=1.0)
pca space: NPArray2Domain(size=10000, num_columns=4, nan=True, T=f64)
↪   SymmetricDistance() MaxDivergence
pca map: map(2)=2.0 map(1)=2.0 check(2, 1.0)=False check(2, 2.0)=True
make_private_pca unit_epsilon=1: map(2)=1.0 map(1)=1.0
pca n_components=2: map(2)=2.0
pca composition of epsilon 1 and 0.5: map(2)=3.0
```

Read the third line twice. The model was built with `epsilon=1.0`, but the map of its measurement at distance 2, one replaced row, is 2.0. And `check` confirms that this fit does not stay within $\varepsilon = 1$.

The reason is in the constructor. The `PCA` class calls the framework's `make_private_pca` with twice its `epsilon`, divided by `n_changes`, the number of rows one person may change. The fourth line shows that `make_private_pca` with a unit budget of 1 has a map of 1.0 at distance 2. So in version 0.16.0 a `PCA(epsilon=1.0)` fit costs $\varepsilon = 2$ per changed row by its own map, and the map is what you should quote. The map is flat below distance 2, because the algorithm is an adaptive composition declared at that distance (Combinators).

The fifth line shows that two components leave the total unchanged, and only the split inside changes. The library draws one eigenvector fewer than there are columns, because the others fix the last direction. So the third for the eigenvectors is split over three draws for four components and over two draws for two, and each draw gets more. The `measurement()` method is there for the compositors. The last line composes a fit at `epsilon=1.0` with one at `epsilon=0.5`, and the maps add to 3.0 at distance 2.

14.4. The NumPy domain underneath

Both estimators use `dp.numpy.array2_domain`, the domain of two-dimensional NumPy arrays, described by a number of columns, an optional number of rows and the element type. It can also have a norm, its order `p` and an origin, which say that every row lies within that norm of the origin. A domain with a norm is a promise about the data, and `make_np_clamp` is the transformation that makes the promise true.

```
import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
rows = dp.numpy.array2_domain(num_columns=2, T=float)
clamp = dp.numpy.make_np_clamp(rows, dp.symmetric_distance(), norm=1.0, p=2,
 ↪  origin=np.array([1.0, 2.0]))
print(clamp(np.array([[1.0, 2.0], [4.0, 6.0], [1.5, 2.0]])).tolist())  # [[1.0, 2.0],
 ↪  [1.6, 2.8], [1.5, 2.0]]
print(clamp.map(1))  # 1
print(clamp.output_domain)  # NPArray2Domain(origin=[1. 2.], norm=1.0, p=2,
 ↪  num_columns=2, nan=False, T=f64)
```

The clamp leaves the first and third rows alone, because they lie within distance 1 of the origin, and pulls the second row from distance 5 back to the sphere. Its stability map is the identity, because clamping each row on its own neither adds nor removes rows (Transformations). Its output domain records the norm, the order, the origin, and that no value is missing.

That output domain is what `make_private_pca` prefers to receive. Such a domain states that the rows are already bounded around a known centre. So the algorithm skips the private mean and its third of the budget.

```
bounded domain: NPArray2Domain(origin=[0. 0. 0. 0.], norm=1.0, p=2, size=10000,
 ↪  num_columns=4, nan=True, T=f64)
pca on the bounded domain: map(1)=0.50 map(2)=1.00 map(4)=2.00
```

The script passes an explicit split as a `PCAEpsilons`, with half for the eigenvalues, a sixth for each of three eigenvectors and nothing for the mean. The map is then linear in the distance, not flat, because no compositor is declared in between. Use this route when the centre is public, for example for scores on a known scale, and the class when it is not.

14.5. What a reviewer should ask

The first question is the privacy unit. Both estimators count rows, so the claim “one person” holds only if each person has one row, which on the survey was your own Polars step. The second is the bounds. The bounds of the regression and the row norm and row count of the PCA must come from the code book or the domain. A bound read from the data leaks it. The third is which features are on, because a feature is a statement about what you accept on trust.

```
regression without honest-but-curious: Attempted to use function that requires
 ↪  "contrib", "honest-but-curious", but not enabled
pca without honest-but-curious: Attempted to use function that requires "contrib",
 ↪  "honest-but-curious", but not enabled
pca without idealized-numerics: Attempted to use function that requires "contrib",
 ↪  "idealized-numerics", but not enabled
regression without idealized-numerics: constructed
pca with n_samples one short of the table: fitted without complaint
```

Both estimators need `honest-but-curious`, because they use user-defined transformations whose functions and stability maps the library takes as written. For example, the pairing step of the regression claims that one row moves `runs` readings, and the library records that claim without proof. The PCA also needs `idealized-numerics`, which concedes that its proofs hold for real arithmetic and not for floating point. Every guarantee of the PCA rests on that concession, and the library's own source notes that it knows no floating-point-safe sampler for the eigenvector draw. The regression constructs without the flag, because its private medians run in the Rust core and only the pairing is Python.

The last line is the trap for a reviewer. The number of rows is part of the domain of the PCA, but the library does not check that the array has that many rows. So a wrong `n_samples` is a wrong guarantee that nothing reports. The clamp enforces the row norm, and the bounds of the regression need no enforcement because its sensitivity does not depend on them. Limitations and troubleshooting collects the other things the library does not check.

14.6. The releases of one run

```
released (one run):
  toy private fit: slope=1.9529 intercept=1.0101
  survey private fit: slope=1.0101 intercept=-0.9899
  survey private fit at 35 usual hours: 34.36 actual
  pca row_norm=1: singular values=[80.48, 40.66, 34.28, 26.37]
  pca row_norm=1: |cos| with exact components=[1.000, 0.998, 0.996, 0.996]
  pca row_norm=1: mean=[0.001, 0.002, 0.001, 0.001]  explained variance ratio=[0.648,
↪  0.165, 0.117, 0.070]
  pca row_norm=40: singular values=[906.8, 268.6, 212.6, 153.2] (exact [901.9, 278.6,
↪  222.2, 156.9])
  pca row_norm=40: |cos| with exact components=[0.996, 0.659, 0.426, 0.086]
  pca n_components=2: returned the model: True  singular values=[80.41, 40.64]
  pca n_components=2: |cos| with exact components=[0.999, 0.999]
```

The toy fit is about one grid step off the least-squares line at one cut point, which is what a private median on 50 readings at $\varepsilon = 2$ gives. The test allows the slope within 0.3 of the least-squares slope and the intercept within 0.5, about three grid steps at either cut point. A candidate three steps from the best scores lower by every reading in between, and the exponential mechanism makes such a candidate rare at this budget.

The survey fit is the same on every run. The medians sit exactly at the cut points, and the nearest candidates on the grid give a slope of 1.0101 and an intercept of about -1. Thousands of readings lie within one grid step of each median, so the next candidate scores thousands worse than the best, and noise at scale 2 never overturns that gap. The error of the fit is therefore the grid, not the noise, and a finer `candidates_count` removes it at no cost in privacy. The test allows the slope within 0.1 of the median line and the intercept within 4 hours, two grid steps at the lower cut point. The fit predicts about 34.4 actual hours for someone who usually works 35.

At row norm 1 the PCA recovers the singular values of the clamped table to within a tenth, and all four directions to a cosine above 0.99. Its mean is within a few thousandths of the true zero. The test allows each singular value within 1 and each cosine above 0.9. Both bounds leave a wide

margin, because the Laplace noise on the eigenvalues is a few units against eigenvalues in the thousands, and the four directions are well separated.

At row norm 40 the singular values are those of the raw table, within a few percent, but only the first direction is right. The concentration of the eigenvector sampler falls with the square of the row norm, so the last three directions are close to random. The test checks only the first singular value, within 30, and the first cosine, above 0.9. The two-component fit returns the model itself, and its two directions are as good as the first two of the four-component fit.

14.7. Summary

Table 14.1.: The calls and concepts of private modelling, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.sklearn.linear_model.LinearRegression(measure, x_bounds, y_bounds, scale, runs)`	A Theil-Sen regression from two private medians	Bounds fix the cut points and the grid; they never clip the data
`model.measurement.map(1)`	The privacy loss of the fit per added or removed row	Two medians at scale s cost $2\,\text{runs}/s$; quote this, not the arguments
`candidates_count`	The grid the medians are chosen from	On a large dataset the grid, not the noise, is the error; refine it for free
The median line against least squares	What the estimator targets	Compare a Theil-Sen fit with the median line; outliers pull least squares only
`dp.sklearn.decomposition.PCA(epsilon, row_norm, n_samples, n_features, n_components)`	A release of the mean, the singular values and the components	`row_norm` and `n_samples` are public facts the library does not check
`pca.measurement()`	The measurement behind `fit`, for maps and compositors	In 0.16.0 its map at distance 2 is twice `epsilon`; read the map
`n_components`	How many directions to draw	Fewer draws give each more budget at the same total
`row_norm`	The sensitivity scale of the whole algorithm	Tight: magnitudes are the clamped table's, directions good. Loose: magnitudes right, directions poor
`dp.numpy.array2_domain(norm, p, origin)` and `make_np_clamp`	The bounded domain and the transformation that enforces it	A known origin skips the private mean and its third of the budget
`honest-but-curious`, `idealized-numerics`	Trust the plugins' maps; accept real-arithmetic proofs	Both estimators need the first; the whole PCA rests on the second

14.8. References

- OpenDP documentation: “Statistical Modeling”, “Linear Regression” with the API reference of `opendp.extras.sklearn.linear_model`, “Differentially Private PCA”, and “Theil-Sen Regression” in the plugins user guide.
- Theil, “A Rank-Invariant Method of Linear and Polynomial Regression Analysis” (1950), and Sen, “Estimates of the Regression Coefficient Based on Kendall’s Tau” (1968): the median-of-slopes estimator.
- Dwork and Lei, “Differential Privacy and Robust Statistics” (2009): why robust estimators such as medians are the natural private ones.
- Amin, Dick, Kulesza, Munoz and Vassilvitskii, “Differentially Private Covariance Estimation” (2019): the eigenvalue and eigenvector mechanisms behind the PCA.
- The full edition’s chapters on selection in the full edition, sensitivity in the full edition and private empirical risk minimization in the full edition derive the results this chapter states.

15. Limitations and troubleshooting

OpenDP's guarantee covers only computations through measurements with valid proofs. It does not cover assumptions about arithmetic, code before the context or channels other than the output. Most library refusals enforce this boundary; error messages often indicate that the guarantee is working as intended.

This chapter follows the documentation's pages on limitations, features and troubleshooting. It describes unsupported guarantees, the role of each feature flag and behaviour outside the guarantee. It also explains recurring error messages and the checks reviewers should make before trusting a release.

The script opendp_limitations.py reproduces these messages with OpenDP 0.16.0 and Polars 1.36.1. Framework examples use small lists; Polars examples use the France Labour Force Survey (LFS) sample behind a context. Messages, sensitivities, scales and error bars are deterministic. OpenDP's noise cannot be seeded, so the released count varies and is labelled as one run.

```
uv run python code/part04-extending/opendp_limitations.py
```

15.1. What OpenDP does not promise

The documentation identifies five deployment limitations. First, privacy proofs use real arithmetic, while computers round floating-point sums. Casacuberta, Shoemate, Vadhan and Wagaman (2022) found that several libraries treated arithmetic as exact when computing sum sensitivity. Rounding increased the true sensitivity, making the added noise too small and the stated ε incorrect.

OpenDP accounts for rounding in its proof. The script builds float and integer sums of values from 0 to 80 (Transformations) and computes their sensitivity for 36 added or removed rows.

```
def sum_sensitivities(d_in: int) -> tuple[float, int]:
    """The sensitivity of a float sum and of an integer sum of values in [0, 80], at
     ↪  `d_in` added or removed rows."""
    float_sum = (dp.vector_domain(dp.atom_domain(bounds=HOURS_BOUNDS)),
 ↪  dp.symmetric_distance()) >> dp.t.then_sum()
    int_sum = (dp.vector_domain(dp.atom_domain(bounds=(0, 80))),
 ↪  dp.symmetric_distance()) >> dp.t.then_sum()
    return float_sum.map(d_in), int_sum.map(d_in)
```

```
float sum of 36 rows in [0, 80]: sensitivity 2880.0000007451
int sum of 36 rows in [0, 80]: sensitivity 2880
```

With exact arithmetic, both sensitivities would be 36×80. The float sum's map adds a rounding term; integer addition needs none. This term grows with row count, so float sums on the LFS need a public row-count bound (Essential statistics). Components that still assume exact arithmetic require the `idealized-numerics` flag. Prefer integer statistics when possible; otherwise use OpenDP's reported scale rather than a formula that ignores rounding.

Second, OpenDP is not hardened against side channels. Query time, memory use and cache-line access may depend on the data, and proofs do not cover information learned through these channels. Deployment must therefore limit who can interact with the system. An analyst submitting queries through a queue poses a different risk from one with shell access. Restrict the latter or account for the weaker guarantee.

Third, components require vetting. Each should have a written proof and review; until review is complete, it requires the `contrib` flag. Review is intended to identify errors in code or proofs. Releases using `contrib` components therefore rely on proofs not yet checked by a second reviewer. Record the components used so you can reassess a release after corrections.

Fourth, interfaces may change between minor releases before version 1.0.0. Pin the version, as this book does at 0.16.0, and test release code so changed defaults are detected before they alter results. Fifth, unknown software bugs remain possible. Keep a reproducible record of each run so you can recompute its guarantee, as described in the practitioner edition's Release and governance in the practitioner edition.

15.2. The feature flags

Feature flags record acceptance of these limitations. OpenDP has four flags and one deprecated alias; all are off until `dp.enable_features` enables them. A first release enabled `contrib` without showing the disabled behaviour, so the script disables it, tests the Laplace mechanism and enables it again.

```
dp.disable_features("contrib")
without_contrib = refusal(lambda: space >> dp.m.then_laplace(scale=1.0))
dp.enable_features("contrib")
```

```
laplace without contrib: OpenDPException: Attempted to use function that requires
 ↪  "contrib", but not enabled. See
 ↪  https://docs.opendp.org/en/stable/api/user-guide/#feature-listing, then call
 ↪  enable_features("contrib")
```

The helper `refusal` returns an exception's type and message. Each flag's error names the flag and the call that enables it. To find which calls require a flag, run the script with that flag disabled.

The `honest-but-curious` flag gates constructors whose guarantees rely on user-supplied arguments, mainly the plugins in Plugins. For example, `make_user_measurement` accepts a Python privacy map that OpenDP cannot verify. The name describes the assumed adversary: honest when constructing the measurement, curious when inspecting its outputs. The script builds a plugin that returns its input unchanged and claims privacy loss equal to input distance.

```
def plugin_measurement() -> dp.Measurement:
    """A plugin measurement whose privacy map is only a claim: the library cannot check
     ↪ `identity_map`."""
    return dp.m.make_user_measurement(
        dp.atom_domain(T=float, nan=False),
        dp.absolute_distance(T=float),
        dp.max_divergence(),
        function=identity,
        privacy_map=identity_map,
    )
```

```
plugin without honest-but-curious: OpenDPException: Attempted to use function that
 ↪ requires "contrib", "honest-but-curious", but not enabled. See
 ↪ https://docs.opendp.org/en/stable/api/user-guide/#feature-listing, then call
 ↪ enable_features("contrib", "honest-but-curious")
plugin with honest-but-curious: map(1.0)=1.0
```

With the flag enabled, the plugin reports its claimed map: loss 1 at distance 1, with no verification. Enable it when developing and proving plugins, not where untrusted analysts submit queries; they could register a map claiming any cost. The flag also gates `OnceFrame.lazy()`, which exposes a released Polars plan that could run twice.

The third flag is `idealized-numerics`, which marks components whose proof assumes exact arithmetic. In 0.16.0 none of the constructors in this book needs it. The float Laplace mechanism and the float sums above build with the flag off, because their proofs account for rounding. The older name of the flag, `floating-point`, still works and gives a warning.

```
float laplace without idealized-numerics: no error
enable_features("floating-point"): DeprecationWarning: "floating-point" is deprecated.
 ↪ Use "idealized-numerics" instead.
assert_features("idealized-numerics") with the alias on: no error
assert_features("idealized-numerics") after disabling it: OpenDPException: Attempted to
 ↪ use function that requires "idealized-numerics", but not enabled. See
 ↪ https://docs.opendp.org/en/stable/api/user-guide/#feature-listing, then call
 ↪ enable_features("idealized-numerics")
assert_features("contrib", "honest-but-curious"): OpenDPException: Attempted to use
 ↪ function that requires "contrib", "honest-but-curious", but not enabled. See
 ↪ https://docs.opendp.org/en/stable/api/user-guide/#feature-listing, then call
 ↪ enable_features("contrib", "honest-but-curious")
```

The last three lines use `dp.assert_features`, which raises the error of a gated constructor without building anything. Every gated constructor calls it first. At the top of a script, one such line also states every flag that the script relies on. The fourth flag, `rust-stack-trace`, changes no guarantee, because it only adds the Rust frames to an error message, as the section on reading an exception shows.

15.3. What lies outside the guarantee

The guarantee covers what happens to the data inside a measurement, and four things happen outside it in every analysis. The first is loading the data, which the documentation leaves to a trusted curator. One reason is that a CSV reader infers the type of each column from its values. The script reads three two-row files that differ in one value, then a longer file whose odd value comes after the rows that the inference reads.

```
for raw in (b"x\n1\n2\n", b"x\n1\n2.5\n", b"x\n1\nabc\n"):
    print(f"csv {raw!r}: column x inferred as
     ↪  {pl.read_csv(io.BytesIO(raw)).schema['x']}")
late_text = b"x\n" + b"1\n" * 200 + b"abc\n"  # a text value after 200 numbers
failed = refusal(lambda: pl.read_csv(io.BytesIO(late_text), infer_schema_length=100))
print(f"csv with a text value on row 201, inferred from 100 rows:
 ↪  {first_line(failed)}")
as_strings = pl.read_csv(io.BytesIO(late_text), infer_schema=False).schema["x"]
print(f"the same file with infer_schema=False: column x read as {as_strings}")
```

```
csv b'x\n1\n2\n': column x inferred as Int64
csv b'x\n1\n2.5\n': column x inferred as Float64
csv b'x\n1\nabc\n': column x inferred as String
csv with a text value on row 201, inferred from 100 rows: ComputeError: could not parse
 ↪  `abc` as dtype `i64` at column 'x' (column number 1)
the same file with infer_schema=False: column x read as String
```

So one person's value decides whether the column is integer, float or text, and a late value decides whether the load succeeds at all. If the result of the load is visible outside, it is a release that nothing accounted for. The documentation gives two fixes. Read a format that carries its schema, as this book does with Parquet. Or read every column as text with `infer_schema=False`, then cast it yourself. The LFS loader in `practitioner_datasets` converts the CSV to Parquet once, which is the curator's step, outside any guarantee.

The second thing outside the guarantee is public information. Every bound, imputed value, key list and margin in this book comes from the domain or from a published source. A bound read from the private data is a release of the data, and the library cannot tell the two apart.

The third is code that runs outside the context, which is why the data and the context told you to delete your reference to the frame. Collecting the frame yourself is not a query, so nothing counts it. The fourth is the error channel. Whether a query succeeds or fails is itself an output, so a failure that depends on the data is a release. OpenDP guards against this inside a query, for example by replacing some Polars arguments so that an expression cannot fail on one value. The loading step and your own code have no such guard.

15.4. The error messages you will meet

The documentation's troubleshooting page has one entry, the type mismatch, and the earlier chapters met several more. This section gives each with its cause and its fix. The type mismatch comes first. OpenDP fixes the type of every argument from the domain of the data, and it refuses

an argument of another type instead of converting it. The script puts three integers behind a context and clamps them with float bounds.

```
float bounds on integer data: TypeError: inferred type is f64, expected i32. See
 ↪  https://docs.opendp.org/en/stable/getting-started/trouble-shooting.html#inferred-type-is-xxx-expe
integer bounds on integer data: scale=10.0
```

The message names the type it inferred from your argument and the type of the data. To fix it, pass bounds of the type of the data, `(0, 10)` for integers, or cast the data first. The documentation names one common cause, a scalar ε where a measurement expects an (ε, δ) pair. To show it, the script asks an approximate measurement, built with `make_approximate` as in Combinators, whether it stays within a scalar budget.

```
check(1.0, 1.0) on an approximate measurement: TypeError: Cannot coerce a non-tuple
 ↪  type to a tuple
check(1.0, (1.0, 0.0)) on the same measurement: True
```

The privacy loss of an approximate measurement is a pair, so its budget must be a pair too, and a pure budget is the pair $(\varepsilon, 0)$. The same message comes from `binary_search_param` with a scalar `d_out` on an approximate chain (Utilities and the Context API).

The next message comes from a domain that allows missing values. The library infers a list of Python floats as a domain that may contain NaN. A clamp refuses such a domain, because NaN is neither inside nor outside a bound.

```
clamp on floats that may be nan: OpenDPException: FailedFunction("Domain contains nan")
impute, then clamp: scale=80.0
```

The fix is the one of the first chapter, `impute_constant` before `clamp`, which maps the domain to one without NaN. In Polars the same need appears as `fill_null` before a `dp.sum`.

The next message comes from mismatched domains in a chain. The `>>` operator checks that the output domain of the left side equals the input domain of the right side, and refuses otherwise. The script clamps to one range and then chains a sum built for a wider one.

```
vectors = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = vectors >> dp.t.then_clamp((0.0, 10.0))
wide_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0.0, 100.0))),
 ↪  dp.symmetric_distance())
mismatch = refusal(lambda: clamp >> wide_sum)  # the clamp's output has bounds [0, 10],
 ↪  the sum wants [0, 100]
chained = clamp >> dp.t.then_sum()  # a partial constructor takes the clamp's output
 ↪  space, so they agree
```

```
clamp to [0, 10] then a sum built for [0, 100]: OpenDPException:
 ↪  DomainMismatch("Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
    output_domain: VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64))
```

```
    input_domain:  VectorDomain(AtomDomain(bounds=[0.0, 100.0], T=f64))
")
clamp to [0, 10] then then_sum(): sensitivity 10.0
```

The message prints both domains, so you can read that the bounds differ. The fix is the partial constructor `then_sum()`, which takes its input space from the output of the clamp (Transformations).

The remaining messages come from the Polars side, with the LFS behind a context of one query and a budget of $\varepsilon = 1$. The unit is the 36 rows of the data and the context.

```
def make_context(
    frame: pl.LazyFrame, delta: float | None = None, margins:
 ↪  Sequence[dp.polars.Margin] | None = None
) -> dp.Context:
    """The LFS behind a unit of 36 rows and a budget of epsilon = 1, with or without a
     ↪  delta, for one query."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=delta) if delta else
 ↪  dp.loss_of(epsilon=EPSILON),
        split_evenly_over=1,
        margins=margins,
    )
```

A float sum needs a margin with `max_length`, for the reason of the first section. Its sensitivity includes a rounding term that grows with the number of rows, so the proof needs a public bound on that number.

```
float sum without a margin: OpenDPException: MakeTransformation("must specify
 ↪  'max_length' in a margin with by=[]")
float sum with max_length=5,400,000: scale=2,881.04
```

The fix is the documentation's margin, `dp.polars.Margin(max_length=150_000 * 36)`. The scale is then a little above 2880, the sensitivity of the sum over $\varepsilon = 1$ in exact arithmetic. The extra is the rounding term at 5.4 million rows. A grouped sum needs the same bound in the margin of its grouping, which is what `by=[]` in the message refers to.

The next three messages come from one query. The script builds a count and tries to collect it before it releases it. Then it releases the count, collects the result twice, and submits another query to a context whose one share is spent.

```
count_query = context.query().select(dp.len(signed=True))
before_release = refusal(lambda: count_query.collect())  # the plan has no data; only a
 ↪  released OnceFrame does
once = count_query.release()
dp_count = int(once.collect()["len"][0])
twice = refusal(lambda: once.collect())  # a second collection would be a second
 ↪  release
```

```
exhausted = refusal(lambda: context.query().select(dp.len(signed=True)).release())
```

```
count: scale=36.0 95% accuracy=108.34
collect before release: ValueError: You must call `.release()` before executing a
↪  query.
collect a OnceFrame twice: OpenDPException: FailedFunction("OnceFrame has been
↪  exhausted")
query after the budget is spent: ValueError: Privacy allowance has been exhausted
```

A `LazyFrameQuery` is a plan without data, so `collect` on it would compute nothing private. The library refuses anyway, so that the habit of collecting a plan never meets a frame that holds data. You can collect the `OnceFrame` from `release` only once, because the noise is drawn when the plan runs, so a second run would be a second release. The third message is the one that the first chapter ended on. None of the three is a bug to work around, so keep the DataFrame from the first `collect` if you need a value twice.

The last message is from Contingency tables. A contingency table whose keys or cuts cover every column has no private key set to protect, so its release spends no δ. The library refuses a budget with a δ instead of leaving it unspent. The script uses $\delta = 10^{-7}$, the value of the grouping chapters.

```
contingency table with keys for every column under (epsilon, delta): ValueError: delta
↪  (1e-07) must be zero because keys and cuts span all columns
the same table under pure epsilon: ContingencyTableQuery
```

So state the budget as pure ε when the keys are all public, or as the zero-concentrated budget ρ of Utility and noise. Keep δ for the tables whose keys the library must discover. Two more messages from earlier chapters belong in this list. A plain `mean` is refused, because OpenDP has no proof for it. A grouping without keys or a δ is refused, because its key set is private (Grouping).

15.5. Reading an OpenDPException

Most of the messages above came as an `OpenDPException`, which is how an error in the Rust core reaches Python. Its `variant` is the kind of error, from a fixed list such as `DomainMismatch` or `FailedFunction`, which code can match on. Its `message` is the text, and its `raw_traceback` is the Rust stack at the point of the error, which is recorded even if you do not display it. The script raises the domain mismatch again and reads the three, then switches on `rust-stack-trace` and prints the exception again.

```
try:
    _ = clamp >> wide_sum
except dp.OpenDPException as err:
    print(f"variant: {err.variant}")
    print(f"message: {first_line(err.message or '')}")
    print(f"rust frames recorded: {err.raw_traceback is not None}")
    dp.enable_features("rust-stack-trace")
```

```
    print(str(err).strip())
    dp.disable_features("rust-stack-trace")
```

```
variant: DomainMismatch
message: Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
rust frames recorded: True
Continued Rust stack trace:
    opendp_combinators__make_chain_tt
    opendp::combinators::chain::make_chain_tt::<opendp::ffi::any::AnyDomain,
 ↪  opendp::ffi::any::AnyDomain, opendp::ffi::any::AnyDomain,
 ↪  opendp::ffi::any::AnyMetric, opendp::ffi::any::AnyMetric,
 ↪  opendp::ffi::any::AnyMetric>
    opendp::combinators::chain::mismatch_error::<opendp::ffi::any::AnyDomain>
  DomainMismatch("Intermediate domains don't match. See
 ↪  https://github.com/opendp/opendp/discussions/297
    output_domain: VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64))
    input_domain:  VectorDomain(AtomDomain(bounds=[0.0, 100.0], T=f64))
")
```

The frames read from the outside in: the entry point that Python called, the chaining function, and the helper that formats the mismatch. They help when the message does not say where in a long chain the library stopped, and they locate a bug in the Rust source. The list of variants may change between versions, so match on a variant only in tests you control. The errors that are not an `OpenDPException` come from the Python layer, a `TypeError` from argument conversion and a `ValueError` from the context and the query classes.

15.6. What a reviewer verifies

A release made with OpenDP claims that this table satisfies (ε, δ)-differential privacy for this privacy unit, and Table 15.1 collects the checks of that claim. The first is the unit, because an ε means nothing until you know what it protects. For example, `dp.unit_of(contributions=36)` protects a person only if nobody has more than 36 rows (Identifier truncation). The second is the budget, read back from the context.

The third is the flags, because with `honest-but-curious` the reviewer must read each privacy map that the library took on trust. The fourth is the source of each bound and key, and the fifth is what ran outside the context. The full edition's chapter on the definition in the full edition says what the finished claim must state.

Table 15.1.: What a reviewer verifies before trusting a release made with OpenDP.

Check	Question	Where to look
Unit	What does one privacy unit contain, and does anyone exceed it?	`privacy_unit` in the context; the identifier and any truncation

Check	Question	Where to look
Budget	What was spent, under which measure, and was any release collected twice?	`context.current_privacy_loss()`; one `collect` per `OnceFrame`
Features	Which flags were on, and if `honest-but-curious`, which maps were claimed?	the `enable_features` line; every `make_user_*` call and `lazy()`
Public information	Where did each bound, imputed value, key list and margin come from?	the constants of the script, traced to their sources
Outside the context	What touched the data before and after the context?	the load and its schema; any code holding the frame

15.7. Summary

Table 15.2.: The flags, the limits and the error messages of this chapter, with the rule of thumb for each.

Call or concept	What it does	Rule of thumb
`dp.enable_features("contrib")`	Opts in to components whose proofs are not yet vetted	Every script in this book needs it; record it with the release
`honest-but-curious`	Opts in to constructors whose guarantee rests on your arguments, such as plugins and `OnceFrame.lazy()`	Only where you wrote and proved the map, never where an untrusted analyst queries
`idealized-numerics`	Opts in to components whose proof assumes exact arithmetic	No constructor in this book needs it in 0.16.0; its alias `floating-point` warns
`dp.assert_features(...)`	Raises the flag error without building anything	One line at the top of a script states what it relies on
`rust-stack-trace`	Adds the Rust frames to an exception's text	Switch on to locate a bug; it changes no guarantee
Loading the data	Outside the guarantee; CSV inference reads private values	Read Parquet, or `infer_schema=False` and cast
`inferred type is f64, expected i32`	An argument's type differs from the data's	Match the data's type or cast first; a scalar ε for a pair is the common case
`Domain contains nan`	A float domain may hold NaN, and clamp refuses it	`impute_constant` or `fill_null` before `clamp`
`DomainMismatch`	Two chained components disagree on the domain between them	Use `then_*` constructors; read the two domains in the message

Call or concept	What it does	Rule of thumb
`must specify 'max_length' in a margin`	A float sum needs a public bound on the rows	`dp.polars.Margin(max_length=...)` from a public source
`You must call .release() before executing a query`	A plan was collected before it was released	`release()` first, then `collect()` once
`OnceFrame has been exhausted`	A released frame was collected twice	Keep the DataFrame from the first `collect`
`Privacy allowance has been exhausted`	Every share of the budget is spent	The refusal is the guarantee; a new context is a new budget
`delta (...) must be zero because keys and cuts span all columns`	A table with public keys spends no δ	Budget the query with pure ε or with ρ
`OpenDPException`	Carries `variant`, `message` and `raw_traceback`	Match on the variant only in tests; expect the list to change before 1.0

15.8. References

- OpenDP documentation: "Limitations" and the feature listing in the API user guide; "Troubleshooting" and "Next Steps" in Getting Started; the note on data loading in "Working with Tabular Data" and the margin and float-sum notes in "Essential Statistics".
- Casacuberta, Shoemate, Vadhan and Wagaman, "Widespread Underestimation of Sensitivity in Differentially Private Libraries and How to Fix It" (2022): the finite-precision sensitivity errors in earlier libraries, and the corrected proofs that OpenDP's sums follow.
- Mironov, "On Significance of the Least Significant Bits for Differential Privacy" (2012): the floating-point attack on the Laplace sampler.
- Jin, McMurtry, Rubinstein and Ohrimenko, "Are We There Yet? Timing and Floating-Point Attacks on Differential Privacy Systems" (2022): timing and floating-point side channels in differential privacy libraries.
- The full edition's chapters on the definition in the full edition and discrete and floating-point-safe mechanisms in the full edition derive the results this chapter states; the practitioner edition's Release and governance in the practitioner edition gives the record a reviewer asks for.

Part V.

Synthetic data

This part releases a dataset rather than a statistic. A synthetic dataset contains generated rows with statistics that approximate the real data, subject to a privacy budget. OpenDP first releases a contingency table of noisy counts for combinations of columns. It then fits a model to the noisy marginals and samples rows, a post-processing step that uses no additional budget.

Synthetic-data quality depends on the measured marginals and budget allocation. Contingency tables releases fixed-workload marginals with the `Fixed` algorithm and explains keys, cuts, thresholds and `ContingencyTable` error estimates. Synthetic data with AIM and MST selects marginals iteratively with AIM or along a spanning tree with MST, then generates rows. Evaluating and tuning synthetic data compares synthetic and real data on measured marginals, an unmeasured query and a downstream model, and examines how the budget and workload affect accuracy.

Table 15.3.: The chapters of this part and the questions they answer.

Chapter	Question it answers
Contingency tables	How do I release the noisy counts of a set of marginals, and what does the table object hold?
Synthetic data with AIM and MST	How do I let the library choose the marginals, and how do I generate synthetic rows?
Evaluating and tuning synthetic data	How good is the synthetic data, and what changes it?

By the end of this part, you can generate a private synthetic dataset, state its guarantee and measure its differences from the source data.

16. Contingency tables

Suppose you need a table of usual work hours by age, employment status and sex. A **contingency table** reports a count for every combination of column values. Grouping released a two-column table; adding more columns multiplies its cells and noise. Instead, release a few **marginals**, counts over two or three columns, and fit one model to them. The model then answers other queries over those columns without additional budget.

The chapter follows the documentation's contingency-table page. It sets up a differential privacy (DP) context with a zero-concentrated budget (zCDP), specifies public keys and cut points, and releases a fixed workload of marginals. It then inspects the resulting `ContingencyTable`, compares a projection with its error bar and adds a marginal omitted from the first release.

The script opendp_contingency_tables.py runs on the Labour Force Survey (LFS) sample with OpenDP 0.16.0 and `mbi` 1.1.0, the pinned Private-PGM version. Budgets, scales, thresholds, keys and error bars are deterministic. OpenDP's noise cannot be seeded, so the page labels released counts as one run.

```
uv run python code/part05-synthetic-data/opendp_contingency_tables.py
```

16.1. From grouped counts to marginals

Grouped counts from Grouping become difficult as columns are added. Each column multiplies the number of cells by its number of values; four raw columns in this sample already produce 4,648 cells. Noise per cell does not depend on its record count. As records spread across more cells, counts shrink while noise stays constant, until noise dominates most cells.

A marginal is a contingency table over a subset of columns. After binning, the two-way `AGE` by `HWUSUAL` marginal has sixteen cells with counts large enough to exceed their noise. A workload of low-order marginals preserves relationships within each subset but omits higher-order relationships. The documentation's workload contains `AGE` by `HWUSUAL` and `HWUSUAL` by `ILOSTAT`.

Separately released noisy marginals can disagree. Both marginals above include one-way `HWUSUAL` counts, but their noisy copies differ. OpenDP therefore fits one probability model to all columns, making its marginals jointly approximate the noisy measurements. All subsequent counts come from that model.

The fit uses **Private-PGM** (McKenna, Sheldon and Miklau, 2019) from the `mbi` package. It builds a graphical model with one factor per measured marginal and fits it by mirror descent, which preserves a valid distribution at each step. Because fitting uses only noisy marginals, it is post-processing and costs no additional budget (the full edition's chapter on query release).

Install `pip install 'opendp[mbi]'` to get `mbi` 1.1.0 and run the documentation's code. Each algorithm fits its model with `mbi.estimation.mirror_descent`, using 1,000 steps by default. The final check tests whether that suffices for this dataset.

16.2. The context under zCDP, with a delta

The context follows Assessing utility and choosing the noise with a larger budget and the book's privacy unit of 36 contributions per person (The data and the context). Its zCDP budget is a privacy parameter ρ of 0.2, with a slack δ of 10^{-7} for stable-key selection. The chapter's queries use $\rho = 0.1$ and $\rho = 0.05$, leaving part of the budget unspent.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The context: 36 contributions per person, rho = 0.2 of zCDP with delta = 1e-7,
     ↪  spent query by query."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # a person answers in at
 ↪  most 36 quarters
        privacy_loss=dp.loss_of(rho=RHO, delta=DELTA),  # the delta pays for the stable
 ↪  keys of ILOSTAT
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.2, 1e-07))
```

The zCDP budget suits a table that releases many counts: Gaussian costs compose by adding their ρ values, more tightly than pure-DP ε costs (Combinators and the full edition's chapter on concentrated differential privacy in the full edition). An ε budget with Laplace noise is also possible, but the error bars below would then be approximate.

The budget includes δ because `ILOSTAT` has no keys or cuts. OpenDP selects its keys by the stable-key method from Grouping, showing a key only when its noisy count exceeds a threshold. The threshold limits the probability that one person's rows alone create a key to at most δ. This slack pays only for key selection. OpenDP rejects nonzero δ when every column has keys or cuts, and rejects $\delta = 0$ when a column has neither. The script demonstrates both refusals.

```
delta with keys for every column: delta (1e-07) must be zero because keys and cuts span
 ↪  all columns
no delta with ILOSTAT unkeyed: delta (0.0) must be nonzero because keys and cuts don't
 ↪  span all columns
```

The toy-frame snippet demonstrates the first refusal. When all columns have public keys, no budget is spent discovering keys, so the query reports no one-way scale or threshold.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
frame = pl.LazyFrame({"SEX": [1, 2, 1, 2]})
fixed = dp.mbi.Fixed(queries=[dp.mbi.Count(("SEX",))])
unit = dp.unit_of(contributions=1)
pure = dp.Context.compositor(data=frame, privacy_unit=unit,
 ↪  privacy_loss=dp.loss_of(rho=0.1), split_evenly_over=1)
query = pure.query().contingency_table(keys={"SEX": [1, 2]}, algorithm=fixed)
print(query.oneway_scale, query.oneway_threshold)  # None None
slack = dp.loss_of(rho=0.1, delta=1e-7)
approximate = dp.Context.compositor(data=frame, privacy_unit=unit, privacy_loss=slack,
 ↪  split_evenly_over=1)
try:
    approximate.query().contingency_table(keys={"SEX": [1, 2]}, algorithm=fixed)
except ValueError as err:
    print(err)  # delta (1e-07) must be zero because keys and cuts span all columns
```

16.3. Keys and cuts

The model needs a fixed, finite set of values for every column, because its cells are indexed by them. OpenDP takes that set from one of three places. For a categorical column you pass **keys**, the public list of its values, as `keys={"SEX": [1, 2]}` from the survey's codebook. For a numeric column you pass **cuts**, the public bin edges, as `cuts={"AGE": [20, 40, 60]}`, which make four left-open intervals, from `(-inf, 20.0]` to `(60.0, inf]`. For a column with neither, here `ILOSTAT`, the library spends part of the budget on **stable keys**, the values whose noisy counts clear the threshold.

Each of the three sources can miss values, and the library handles every miss the same way. A value outside the key list, such as a code 3 in `SEX`, becomes `null`, and so does a stable key with too small a count. So `null` stands for "something else", and a true null in the data joins it. To keep true nulls apart, recode them to a value of their own with `with_columns` before the query, as the documentation advises. A cut column has no null bin, so a null `AGE` or `HWUSUAL` lands in the last bin. The check at the end confirms that on the 255 rows without an age and the 20,819 without usual hours.

The `ILOSTAT` codebook lists six codes, from 1 for working to 9 for a child under 15. Codes 4 and 5 do not occur here, so stable-key selection should find four codes. The `null` key represents the two absent codes and has no corresponding rows.

16.4. The Fixed algorithm and its workload

`dp.mbi.Fixed` releases a specified workload: a list of `Count` queries, one per marginal. Use it when you know which relationships to measure. The algorithms in the next chapter select marginals instead. The query specifies columns, keys, cuts and algorithm.

```
def workload_query(context: dp.Context) -> ContingencyTableQuery:
    """The documentation's query: keys for SEX, cuts for AGE and HWUSUAL, stable keys
     ↪  for ILOSTAT, two counts."""
    return (
        context.query(rho=TABLE_RHO, delta=DELTA)
        .select("SEX", "AGE", "HWUSUAL", "ILOSTAT")
        .contingency_table(
            keys={"SEX": [1, 2]},  # public: the codebook's two codes
            cuts={"AGE": [20, 40, 60], "HWUSUAL": [1, 20, 40]},  # left-open bins:
 ↪  (-inf, 20], (20, 40], ...
            algorithm=dp.mbi.Fixed(
                queries=[dp.mbi.Count(("AGE", "HWUSUAL")), dp.mbi.Count(("HWUSUAL",
 ↪  "ILOSTAT"))],
                oneway_split=ONEWAY_SPLIT,  # the share of rho for the one-way marginal
 ↪  of ILOSTAT
            ),
        )
    )
```

The query splits its budget between two uses. `oneway_split` assigns a share to one-way marginals for columns without keys; here, it discovers stable keys for `ILOSTAT`. The remaining budget funds the workload, divided equally among its counts unless weights are provided. By default, `Fixed` uses half the budget times the fraction of unkeyed columns. Here that is half of one quarter. The documentation sets the resulting one-eighth share explicitly.

```
oneway split: 0.125 of rho=0.1 -> rho=0.0125 one-way, rho=0.04375 per count
oneway scale: 227.68  oneway threshold: 1364
```

Inspect both attributes before release. The scale is the standard deviation of Gaussian noise on the one-way `ILOSTAT` counts. The threshold is the minimum noisy count for retaining a key. The privacy unit sets both: one person's 36 rows may share a status, so that person's contribution to one cell can be 36. This is the marginal's sensitivity. At budget ρ, the Gaussian scale for a count of sensitivity Δ is $\Delta/\sqrt{2\rho}$ (Assessing utility and choosing the noise). The script checks the scale for each budget share.

```
scale from the formula 36 / sqrt(2 x 0.0125): 227.68
scale of each count of the workload, 36 / sqrt(2 x 0.04375): 121.70
```

Each cell in the two measured marginals receives Gaussian noise with standard deviation 121.70. The threshold 1364 follows the method in Grouping. At scale 227.68, a key supported only by one person's 36 rows exceeds the threshold with probability at most δ. The privacy unit sets sensitivity to 36; truncating to one row per person and year (Identifier truncation) would reduce it.

16.5. The released table

```
table = query.release()
ilostat_std = table.std(("ILOSTAT",))
ilostat_accuracy = dp.gaussian_scale_to_accuracy(ilostat_std, alpha=ALPHA)
```

The release returns a `ContingencyTable` containing the model's key sets, cuts, stable-key thresholds and each column's key type.

```
keys: SEX=[1, 2, null]  AGE=[(-inf, 20.0], (20.0, 40.0], (40.0, 60.0], (60.0, inf]]
 ↪  HWUSUAL=[(-inf, 1.0], (1.0, 20.0], (20.0, 40.0], (40.0, inf]]  ILOSTAT=[1, 2, 3, 9,
 ↪  null]
cuts: AGE=[20.0, 40.0, 60.0]  HWUSUAL=[1.0, 20.0, 40.0]
thresholds: {'ILOSTAT': 1364}
schema: {'SEX': Int64, 'AGE': String, 'HWUSUAL': String, 'ILOSTAT': Int64}
```

Every key set without cuts ends in `null`, which collects values outside the specified keys. `SEX` therefore has a `null` key, although the data contain only 1 and 2. `ILOSTAT` has four observed codes and `null`. The schema shows that cut columns use string bin labels, so queries name bins rather than numeric values.

`project` sums out every column except those named and returns counts in key order. `project_melted` returns the same projection as a data frame. Both are post-processing. They can project onto unmeasured column sets, but those estimates have no release-based error bar.

`std` estimates a projection's standard deviation; `dp.gaussian_scale_to_accuracy` converts it to an error bar. Projecting a two-way marginal onto one column sums four noisy cells, doubling the standard deviation from 121.70 to 243.40. Thus each `ILOSTAT` count is within 477.06 of its truth with probability 0.95. This estimate excludes the one-way `ILOSTAT` measurement: it releases keys only, so OpenDP assigns no cell-wise precision to it. `HWUSUAL` appears in both measured marginals, and their estimates combine by inverse variance to 181.42.

```
std ILOSTAT: 243.40  95% accuracy: 477.06
std AGE x HWUSUAL: 121.70  95% accuracy: 238.53
std HWUSUAL: 181.42  95% accuracy: 355.58
```

16.6. Adaptive estimation

No measured marginal contains `SEX`, although its keys are known. The model therefore distributes the total evenly across its three keys, and `std` cannot provide an error bar.

```
std SEX: attrs (('SEX',)) are not covered by the query set
```

A second query adds the missing marginal without rebuilding the table from scratch. Passing the released table as `table=` preserves its keys, cuts, marginals and model, then measures the new marginal and refits the model. All columns now have keys, including those discovered in the first release, so this query must use $\delta = 0$ and spends nothing on key discovery.

```
def sex_query(context: dp.Context, table: dp.mbi.ContingencyTable) ->
↪  ContingencyTableQuery:
    """A second query that adds the marginal of SEX to the released table's model;
    ↪  every key is known, so delta = 0."""
    return (
        context.query(rho=SEX_RHO, delta=0.0)
        .select("SEX")
        .contingency_table(
            table=table,  # keep the keys, cuts, marginals and model of the first
↪  release
            algorithm=dp.mbi.Fixed(queries=[dp.mbi.Count(("SEX",))]),
        )
    )
```

The new one-way marginal uses $\rho = 0.05$ and sensitivity 36, giving scale 113.84 and a 223.13 error bar for each of the three `SEX` cells. The model now has four marginals, and the context has spent $\rho = 0.15$ of its 0.2 budget. Because the second query's budget was chosen after seeing the first table, the releases compose adaptively (Combinators). The new fit starts from the first model and runs another 1,000 mirror-descent steps.

```
std SEX after the second query: 113.84  95% accuracy: 223.13
marginals in the model: [('ILOSTAT',), ('AGE', 'HWUSUAL'), ('HWUSUAL', 'ILOSTAT'),
↪  ('SEX',)]
privacy loss spent: (0.15, 1e-07)
```

16.7. Checking the release against the truth

Because the data are public, the script can compare estimates with truth. Polars computes the one-way `ILOSTAT` counts and the `AGE` by `HWUSUAL` table using the documentation's cuts. As in the library, null ages and hours go in the last bin.

```
true ILOSTAT counts: 1=1,517,011  2=155,661  3=1,450,283  9=688,977
true AGE x HWUSUAL (a null AGE or HWUSUAL counts in the last bin):
  AGE (-inf, 20.0]: (-inf, 1.0]=2,733  (1.0, 20.0]=13,651  (20.0, 40.0]=95,686  (40.0,
↪  inf]=1,051,617
  AGE (20.0, 40.0]: (-inf, 1.0]=1,441  (1.0, 20.0]=31,415  (20.0, 40.0]=370,616  (40.0,
↪  inf]=258,601
  AGE (40.0, 60.0]: (-inf, 1.0]=1,285  (1.0, 20.0]=46,743  (20.0, 40.0]=437,820  (40.0,
↪  inf]=309,224
  AGE (60.0, inf]: (-inf, 1.0]=1,502  (1.0, 20.0]=28,084  (20.0, 40.0]=122,405  (40.0,
↪  inf]=1,039,109
```

The following one-run releases can be compared with the error bars above. Their errors also reflect model fitting.

```
released (one run):
  ILOSTAT 1: 1,515,589  (true 1,517,011, off by -1,422)
  ILOSTAT 2: 155,622  (true 155,661, off by -39)
  ILOSTAT 3: 1,450,332  (true 1,450,283, off by +49)
  ILOSTAT 9: 688,890  (true 688,977, off by -87)
  ILOSTAT null: 1,142  (true 0, off by +1,142)
  AGE x HWUSUAL largest |released - true|: 521  (accuracy 238.53)
  at AGE (60.0, inf] HWUSUAL (40.0, inf]: 1,038,588  (true 1,039,109)
  last bin of both, the nulls' cell: 1,038,588  (true 1,039,109, off by -521)
  SEX before the second query: 1,270,525 1,270,525 1,270,525  (uniform)
  SEX after: 1: 1,828,537  (true 1,828,992)  2: 1,982,506  (true 1,982,940)  null: 610
↪  (true 0)
  ILOSTAT after the second query:
    ILOSTAT 1: 1,516,288  (true 1,517,011, off by -723)
    ILOSTAT 2: 155,608  (true 155,661, off by -53)
    ILOSTAT 3: 1,450,301  (true 1,450,283, off by +18)
    ILOSTAT 9: 688,867  (true 688,977, off by -110)
    ILOSTAT null: 590  (true 0, off by +590)
```

The largest of the sixteen two-way errors is 521 rows: more than twice, but less than three times, the 238.53 error bar. It occurs in the last bin of both columns, which contains rows with null hours or age; this confirms how the library assigns nulls. The `null` keys of `ILOSTAT` and `SEX` have no true rows but receive positive estimates, because fitted counts cannot be negative. Expect upward bias for cells whose true count is zero.

Model fitting explains the larger errors. The working code 1 of `ILOSTAT` is 1,422 rows below truth, more than twice its 477.06 bar. Its empty `null` key instead holds 1,142 rows. The fit starts from a uniform model, which assigns one fifth of the rows to `null`. Because an empty key approaches zero only in the limit, 1,000 steps leave about a thousand rows there, missing from other keys.

The second query shows the same effect for `SEX`. Its fit starts from the first model, where the `null` key holds one third of the rows. After 1,000 steps, it still holds 610 rows, more than twice its bar. The two real keys are therefore short by 455 and 434 rows. The fit also continues to adjust `ILOSTAT`; its working code remains 723 rows below truth.

The script tests the fit by refitting the second table's four released marginals from scratch with 10,000 steps of `mbi.estimation.mirror_descent`, rather than the default 1,000. This uses no data, so it is post-processing and a development check, not part of a release.

```
  the same marginals refitted at 10,000 steps (dev time, post-processing):
    ILOSTAT 1: 1,516,829  (true 1,517,011, off by -182)
    ILOSTAT 2: 155,587  (true 155,661, off by -74)
    ILOSTAT 3: 1,450,277  (true 1,450,283, off by -6)
    ILOSTAT 9: 688,859  (true 688,977, off by -118)
    ILOSTAT null: 103  (true 0, off by +103)
    SEX: 1: 1,828,829  (true 1,828,992)  2: 1,982,798  (true 1,982,940)  null: 27
↪  (true 0)
    AGE x HWUSUAL largest |refit - true|: 295  (accuracy 238.53)
```

After more steps, the `null` keys of `ILOSTAT` and `SEX` hold 103 and 27 rows, within their bars. Code 1 is 182 rows from truth, and each real `SEX` key is within 163. The largest two-way error is

295 rows, less than three times its bar. Early stopping therefore biases empty cells upward and larger cells downward without warning in the release. A development check against public or historical data reveals this bias.

The script's test requires every two-way cell and count from the longer fit to lie within three error bars. For the default fit, it checks that each `null` key holds less than one tenth of a percent of the rows and that each real key's error is within three bars plus the rows remaining in `null`.

16.8. Summary

Table 16.1.: The calls and concepts of a contingency-table release, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.loss_of(rho=0.2, delta=1e-7)`	A zCDP budget with a slack for stable keys	Gaussian noise composes tightly over many marginals; the δ is needed exactly when a column has neither keys nor cuts
`keys={"SEX": [1, 2]}`	The public values of a categorical column	Values outside the list become `null`
`cuts={"AGE": [20, 40, 60]}`	Left-open bins of a numeric column	Three cuts make four bins; a null lands in the last one
A column with neither	Stable keys, bought with `oneway_split` of the budget	Rare values fall below the threshold and join `null`
`dp.mbi.Fixed(queries=[dp.mbi.Count(...), ...], oneway_split=1/8)`	A `Count` named in advance	The rest of the budget is shared equally over the counts, or by their weights
`query.oneway_scale`, `query.oneway_threshold`	The noise and the cut-off of the key discovery	Read them before releasing; both are `None` when every column is keyed
`table.keys`, `table.cuts`, `table.thresholds`, `table.schema`	What the model was fitted on	Cut columns come back as bin labels, not numbers
`table.project`, `table.project_melted`	Any count over the table's columns, from the model	Post-processing; a column no marginal covers comes back uniform
`table.std` with `dp.gaussian_scale_to_accuracy`	The error bar of a covered projection	Raises for an uncovered projection; a cell near zero is biased upward
`contingency_table(table=table, ...)`	Adds marginals to an existing model	Every key is then known, so $\delta = 0$

Call or concept	What it does	Rule of thumb
`mbi.estimation.mirror_descent(domain, marginals, iters=...)`	Refit the released marginals with more steps	Post-processing; the default 1,000 steps leave rows in the empty `null` keys on this data, so check the fit on public data

16.9. References

- OpenDP documentation: "Contingency Tables" in Getting Started, and the opening of "Synthetic Data", which lists the algorithms and the rules for null values.
- McKenna, Sheldon and Miklau, "Graphical-model based estimation and inference for differential privacy" (2019): Private-PGM, the model that one consistent table is fitted from, and the mirror-descent fit.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP and the Gaussian mechanism under it.
- The full edition's chapters on query release in the full edition, sensitivity in the full edition and concentrated differential privacy in the full edition derive the results this chapter states.

17. Synthetic data with AIM and MST

Contingency tables released a specified workload, which suits known questions. Synthetic-data users may not know future questions, so a fixed workload can omit important relationships. This chapter uses algorithms that choose marginals: **AIM** selects the marginal its current model fits worst at each round, while **MST** selects a spanning tree of column pairs. Both fit a model to noisy marginals. `synthesize()` draws rows from that model as post-processing, at no additional cost.

The chapter follows the documentation's synthetic-data page and the docstrings for `dp.mbi.AIM`, `dp.mbi.MST`, `dp.mbi.Sequential` and `ContingencyTable.synthesize`. It releases three tables over four survey columns, inspects the selected marginals, generates a synthetic frame of 3.8 million rows and states the guarantee. Comparing synthetic and real marginals reveals a null-handling problem, which the chapter then fixes.

The script opendp_aim_and_mst.py runs on the Labour Force Survey (LFS) sample with OpenDP 0.16.0, pinned `mbi` 1.1.0 and CPU JAX. Noise scales, thresholds, budget calculations, conversions to ε and data facts are deterministic. OpenDP's noise cannot be seeded, so selected marginals, counts, rows and distances vary; the page labels them as one run. The script takes about 75 seconds.

```
uv run python code/part05-synthetic-data/opendp_aim_and_mst.py
```

17.1. When you do not know the questions

A fixed workload spends budget only on specified marginals. For an omitted column pair, the model assumes independence; in the previous chapter, this made `SEX` uniform. With four columns, measuring every pair and triple would divide the budget among ten marginals, including many the model already fits well. The goal is to spend budget where the model is wrong.

AIM selects marginals adaptively. It first measures every one-way marginal with half the budget, which also releases keys for unkeyed columns. It then fits a model and runs rounds. Each round scores candidate marginals of up to three columns by model error adjusted for measurement noise. Candidates that overlap the workload receive greater weight, so the three-way workload marginals weigh most.

Report-noisy-max, a form of the exponential mechanism, uses one tenth of the round's budget to select a candidate (the full edition's chapter on selection). The other nine tenths measure that marginal with Gaussian noise, after which AIM refits the model.

Round budgets adapt. Each starts at the total budget divided by the planned number of rounds. If a measurement changes the model by less than its noise, the next round receives four times the

share. A privacy filter stops the rounds when the budget is spent. The `rounds` argument sets the plan: 16 per column by default, or 64 here. The script uses 10 to reduce runtime. `max_size` caps model memory in megabytes; it does not bind for these four small columns.

```
import opendp.prelude as dp

dp.enable_features("contrib")
aim = dp.mbi.AIM()
print(aim.queries, aim.measure_split, aim.max_size, aim.rounds)  # 3 0.9 80.0 None
print(dp.mbi.MST().num_selections, dp.mbi.MST().measure_split)  # None 0.9
print(dp.mbi.Count(("SEX", "ILOSTAT")))  # Count(by=('SEX', 'ILOSTAT'), weight=1.0)
```

`queries` sets either the maximum candidate size (3 by default) or a workload of weighted `dp.mbi.Count` objects. `measure_split=0.9` allocates 90% of each round to measurement. MST uses the same split and adds `num_selections` (Section 17.3).

17.2. The AIM release

The context follows the documentation and triples the budget because this chapter releases three tables. It uses the 36-contribution privacy unit from The data and the context. Its budget is a privacy parameter ρ of 0.3 under zero-concentrated differential privacy (zCDP), with Gaussian noise; the parameter bounds privacy loss on average (the full edition's chapter on concentrated differential privacy (DP)). The slack δ of 3×10^{-7} pays for selecting `ILOSTAT` keys, which the query does not specify.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The context: a privacy unit of 36 rows, a budget of rho = 0.3 with delta = 3e-7,
     ↪  one share per release."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or
↪   remove one person's 36 rows
        privacy_loss=dp.loss_of(rho=RHO, delta=DELTA),  # zCDP with Gaussian noise;
↪   delta pays for ILOSTAT's keys
    )
```

One function builds all three releases, which differ only by algorithm. Each query receives one-third of the budget: $\rho = 0.1$ and $\delta = 10^{-7}$. It selects four columns, supplies public `SEX` keys and `AGE` and `HWUSUAL` cuts, and discovers `ILOSTAT` keys by thresholding. The `fill` argument is used later.

```
def table_query(context: dp.Context, algorithm: Algorithm, fill: bool = False) ->
↪   ContingencyTableQuery:
    """The documentation's contingency-table query at rho = 0.1 and delta = 1e-7, with
     ↪  the algorithm plugged in."""
    query = context.query(rho=RHO_QUERY, delta=DELTA_QUERY)
    if fill:
        query = query.with_columns(fill_nulls())  # a row-by-row transformation before
↪   the table
```

```
    return query.select(*COLUMNS).contingency_table(
        keys=KEYS,  # public categories of SEX; ILOSTAT's are released with the
↪   threshold
        cuts=CUTS,  # bins for the two numeric columns
        algorithm=algorithm,
    )
```

The first release uses AIM with ten rounds. Before release, inspect the one-way scale and threshold; reading them incurs no cost.

```
aim_query = table_query(context, dp.mbi.AIM(rounds=ROUNDS))
oneway_scale, oneway_threshold = aim_query.oneway_scale, aim_query.oneway_threshold
table_aim, seconds_aim = timed_release(aim_query)
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.3, 3e-07))
query budget: rho=0.1 delta=1e-07
aim: oneway scale=227.6840 threshold=1364
aim: rho per one-way marginal=0.0125  scale 36/sqrt(2 rho)=227.6840
aim: rounds=10  rho per round=0.0050  measure=0.0045  select=0.0005  scale of a round's
↪  marginal=379.4733
aim keys ILOSTAT: [1, 2, 3, 9, None]  thresholds: {'ILOSTAT': 1364}
```

The budget split determines the scales. Half of the query's ρ funds the four one-way marginals, giving each 0.0125. One person can change at most 36 entries across a marginal, all in one cell in the worst case, so its Euclidean sensitivity is 36. Let Δ_2 denote that sensitivity and σ the Gaussian standard deviation. The mechanism is ρ-zCDP when

$$\sigma = \frac{\Delta_2}{\sqrt{2\rho}}, \tag{17.1}$$

and that formula gives the 227.68 the query reports.

The other half of the share is divided over the ten rounds. Nine tenths of the 0.005 of a round goes to measuring, and the formula gives 379.47 for the marginal of a round. The threshold of 1364 is the noisy count that a category of `ILOSTAT` must clear to be released. The keys are the four codes in the data plus the null that stands for anything else.

```
aim release (one run): 13.4 s
aim measured: ('AGE', 'HWUSUAL', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'AGE', 'HWUSUAL') std=379.47
aim measured: ('SEX', 'AGE', 'HWUSUAL') std=189.74
aim measured: ('SEX', 'AGE', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'HWUSUAL') std=379.47
aim measured: ('AGE', 'HWUSUAL') std=379.47
```

The table's `marginals` attribute lists each measurement, its clique of columns and its noise. Here AIM measured all four three-way marginals before any pair, as the weights predict. It measured

one marginal twice, with the second measurement at half the scale. The first changed the model by less than its noise, so the adaptive rule gave the next round four times its share. This exhausted the budget after seven measurements rather than ten. The selected marginals vary across runs; inspect this list after each release.

17.3. A spanning tree of pairs with MST

MST, winner of the 2018 NIST synthetic-data contest, is simpler than AIM. After the same one-way step, it scores column pairs by their disagreement with the current model. Noisy maximum selects pairs that connect columns not yet joined. With four columns, the default is three pairs forming a spanning tree. `num_selections` sets the number of pairs; its default is one fewer than the number of columns. MST measures the selected pairs at once with nine tenths of the remaining budget, then fits the model.

```
mst_query = table_query(context, dp.mbi.MST())
table_mst, seconds_mst = timed_release(mst_query)
```

```
mst: oneway scale=227.6840 threshold=1364
mst: pairs=3  rho per pair=0.0150  scale of a pair=207.8461
mst release (one run): 3.3 s
mst measured: ('HWUSUAL', 'ILOSTAT') std=207.85
mst measured: ('AGE', 'ILOSTAT') std=207.85
mst measured: ('SEX', 'ILOSTAT') std=207.85
mst pairs form a spanning tree: True
```

The one-way step has the same scale and threshold as AIM. The three pairs split nine tenths of the remaining budget, receiving 0.015 each; the scale is 207.85. This is more precise than an AIM round because MST commits to three measurements rather than reserving budget for later rounds. Here the tree is a star centred on `ILOSTAT`, which is most related to the other columns. The script checks that the selected pairs connect all four columns without a cycle.

17.4. A first look at the one-way marginals

Because the data are public, the script compares each synthetic one-way marginal with its real counterpart after binning numeric columns. It reports total variation distance, half the sum of absolute differences between cell shares, the largest cell error in rows and the marginal's standard deviation from `std`.

The last column is a bound. Write k for the number of cells and n for the number of rows. If every cell were off by three standard deviations, the distance would be

$$\frac{3k\sigma}{2n}. \tag{17.2}$$

A well-fitted marginal is usually far below this bound because cell errors have different signs and few reach three standard deviations.

```
one-way SEX (aim): tv=0.00007 largest cell error=2,813 rows  std=204.6 bound=0.00024
one-way AGE (aim): tv=0.00015 largest cell error=1,547 rows  std=201.1 bound=0.00032
one-way HWUSUAL (aim): tv=0.00200 largest cell error=11,310 rows  std=192.3
 ↪  bound=0.00030
one-way ILOSTAT (aim): tv=0.00039 largest cell error=1,670 rows  std=792.7
 ↪  bound=0.00156
one-way SEX (mst): tv=0.00004 largest cell error=2,779 rows  std=204.5 bound=0.00024
one-way AGE (mst): tv=0.00017 largest cell error=1,981 rows  std=204.5 bound=0.00032
one-way HWUSUAL (mst): tv=0.00257 largest cell error=13,620 rows  std=204.5
 ↪  bound=0.00032
one-way ILOSTAT (mst): tv=0.00040 largest cell error=1,963 rows  std=227.7
 ↪  bound=0.00045
```

The `SEX` and `AGE` shares fall within their bounds, but their largest cell errors of two to three thousand rows exceed the noise estimates. `HWUSUAL` is inaccurate even as shares: its distance exceeds the bound manyfold in both releases, and one cell differs by more than eleven thousand rows, over fifty standard deviations. Both algorithms measured this marginal at the same scale as the others, so noise does not explain the error. A systematic problem affects the column.

17.5. Nulls and the key set

Every table's model requires a fixed key set for each column. As the documentation explains, keys come from `keys`, `cuts` or stable-key release; values outside the set become null before counting. Cut columns have no null key. Thus the four `HWUSUAL` bins cover its entire key set, although 20,819 real rows have unrecorded hours.

```
rows: 3,811,932  AGE nulls: 255  HWUSUAL nulls: 20,819
HWUSUAL = 99 (not applicable): 2,294,921
```

In version 0.16.0, nulls follow different paths. For a cut column, the one-way step joins counts to bin labels and omits null rows. The multi-way step assigns a null bin the last index, counting those rows in the bin above 40 hours. The resulting measurements disagree on the row total, so the fitted model compromises. The script compares its one-way `HWUSUAL` marginal with real counts that place nulls in the last bin.

```
aim model total (one run): 3,806,638.8  real rows 3,811,932
HWUSUAL bins (aim, one run): fitted count against the real count with nulls in the last
 ↪  bin
  (-inf, 1.0]: fitted 8,953  real 6,961  difference +1,992
  (1.0, 20.0]: fitted 121,660  real 119,893  difference +1,767
  (20.0, 40.0]: fitted 1,028,785  real 1,026,527  difference +2,258
  (40.0, inf]: fitted 2,647,240  real 2,658,551  difference -11,311
```

The last bin is eleven thousand rows short, while each other bin is about two thousand rows high. The model's total, estimated from one-way marginals, is five thousand below the real count. The fit has spread the omitted null rows across bins.

The documentation's remedy is to replace true nulls with keys before tabulation, using `select` or `with_columns`. The query's `fill` option does this row by row without changing the privacy unit. The script maps null `HWUSUAL` values to 99, the survey's code for not applicable, and 255 null `AGE` values to zero, the youngest bin. Choosing a replacement is a modelling decision, but both steps must assign every row to the same key.

17.6. Guaranteeing marginals with Sequential

Adaptive selection may omit a marginal the release requires. `Sequential` runs algorithms in order on one table, each using the previous model. It divides the budget equally unless `weights` specifies another split. Placing `Fixed` before AIM guarantees the named marginals, while AIM uses the remaining budget where the model is inaccurate. The third release guarantees `ILOSTAT` by `SEX` and `AGE` pairs and fills the nulls.

```
sequential = dp.mbi.Sequential(
    algorithms=[
        dp.mbi.Fixed(queries=[dp.mbi.Count(by) for by in GUARANTEED]),  # these two
 ↪  marginals, whatever happens
        dp.mbi.AIM(rounds=ROUNDS),  # then AIM chooses the rest
    ],
)
sequential_query = table_query(context, sequential, fill=True)
table_seq, seconds_seq = timed_release(sequential_query)
```

```
sequential: rho per guaranteed marginal=0.0125  scale=227.6840
sequential: rho per AIM round=0.0025  scale of a round's marginal=536.6563
sequential release (one run): 12.1 s
sequential model total (one run): 3,811,712.7  real rows 3,811,932
sequential measured: ('SEX', 'ILOSTAT') std=227.68
sequential measured: ('AGE', 'ILOSTAT') std=227.68
sequential measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('SEX', 'AGE', 'HWUSUAL') std=536.66
sequential measured: ('SEX', 'AGE', 'HWUSUAL') std=268.33
sequential measured: ('SEX', 'AGE', 'ILOSTAT') std=536.66
sequential measured: ('HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('SEX', 'HWUSUAL') std=536.66
sequential guaranteed marginals present: True
```

The one-way step again receives half the budget, and the remainder is divided equally between the two algorithms. `Fixed` splits its quarter between two marginals, each with scale 227.68. AIM uses the other quarter for ten rounds, so each marginal has more noise than in the first release: 536.66 rather than 379.47. This is the cost of the guarantee; `weights` can change the allocation. The model's total is now 219 rows from truth, within one standard deviation of a one-way measurement, confirming that null handling caused the earlier discrepancy.

```
one-way SEX (sequential): tv=0.00004 largest cell error=240 rows  std=198.2
 ↪  bound=0.00023
one-way AGE (sequential): tv=0.00006 largest cell error=233 rows  std=200.7
 ↪  bound=0.00032
one-way HWUSUAL (sequential): tv=0.00009 largest cell error=273 rows  std=206.2
 ↪  bound=0.00032
one-way ILOSTAT (sequential): tv=0.00008 largest cell error=360 rows  std=271.5
 ↪  bound=0.00053
```

Every distance is at least three times below its bound, and no cell differs by more than 360 rows, less than two standard deviations. The largest `HWUSUAL` error fell from eleven thousand rows to 273.

17.7. Generating the rows

`ContingencyTable` stores the Private-PGM model fitted in the previous chapter: a graphical model whose marginals jointly approximate the noisy measurements. `synthesize()` draws a frame from this model. It uses only released values, so repeated calls are post-processing and spend no additional budget.

```
synthesize (aim, one run): shape (3806638, 4)  model total 3,806,638.8  11.9 s
┌──────┬──────────┬──────────┬──────────┐
│ SEX  ┆ AGE      ┆ HWUSUAL  ┆ ILOSTAT  │
│ ---  ┆ ---      ┆ ---      ┆ ---      │
│ i64  ┆ f64      ┆ f64      ┆ i64      │
╞══════╪══════════╪══════════╪══════════╡
│ 1    ┆ 19.813324 ┆ 0.482324 ┆ 1        │
│ 1    ┆ 19.500078 ┆ 0.94265  ┆ 1        │
│ 1    ┆ 19.338927 ┆ 0.741979 ┆ 9        │
│ 1    ┆ 19.031755 ┆ 0.118982 ┆ 9        │
│ 1    ┆ 19.232563 ┆ 0.676039 ┆ 1        │
└──────┴──────────┴──────────┴──────────┘
synthetic AGE from 19.00 to 61.00  HWUSUAL from 0.00 to 41.00
synthetic dtypes: {'SEX': Int64, 'AGE': Float64, 'HWUSUAL': Float64, 'ILOSTAT': Int64}
synthetic SEX values: [1, 2]  ILOSTAT values: [1, 2, 3, 9]
```

Without `rows`, the frame size is the model's noisy estimate of the row count. The columns retain their original types, and `SEX` and `ILOSTAT` use their key values. By default, `method="round"` scales expected counts to the requested row count and rounds randomly, so each synthetic marginal matches the model within one row per cell. `method="sample"` instead draws rows independently, adding sampling noise of about the square root of each count. Use it when rows should behave like a sample.

```
sample = table_aim.synthesize(rows=SAMPLE_ROWS, method="sample")  # draws rows
 ↪  independently from the model
```

```
synthesize rows=100,000 method=sample: shape (100000, 4)
```

The first rows have `AGE` just below 20 because the model stores bins, not original values. For cut columns, `synthesize()` draws uniformly within each bin and within one unit of the edge for open-ended bins. Synthetic ages therefore range from 19 to 61. Rows coded 99 receive hours just above 40.

The frame's mean age reflects the bins and sampling rule, not the original data. Use finer cuts when numeric values matter, accepting more cells per marginal and more noise per cell (Evaluating and tuning synthetic data).

The script seeds NumPy's global generator, used for rounding and within-bin draws. Thus the frame is deterministic given the released table.

17.8. The privacy statement

The context has spent all three shares. The script builds the release statement from its variables.

```
budget spent: (0.3, 3e-07)
rho=0.3 at delta=1e-07: epsilon=4.2709
whole context converted: epsilon=4.2709 delta=4e-07
```

Report a ρ budget in ε using the conversion in Assessing utility and choosing the noise. `make_zCDP_to_approxDP` and `make_fix_delta` convert the budget of 0.3 to $\varepsilon = 4.27$ with an additional δ of 10^{-7}. Applying both combinators to the context's `accountant` gives the same ε and total δ: three parts for key selection and one for conversion. The resulting $\varepsilon = 4.27$ guarantee for the three adaptive releases is loose.

```
print(
    f"This release of three contingency tables and the synthetic rows drawn from them
     ↪  satisfies "
    f"(rho = {RHO:g}, delta = {DELTA:g})-zero-concentrated differential privacy, which
     ↪  is "
    f"(epsilon = {epsilon:.2f}, delta = {total_delta:g})-differential privacy, "
    f"for adding or removing all of one person's survey responses, at most
     ↪  {CONTRIBUTIONS} of them, "
    f"under the central model, using OpenDP {dp.__version__} with Gaussian noise. "
    f"The keys of SEX and the cuts of AGE and HWUSUAL were fixed in advance; the keys
     ↪  of ILOSTAT were released "
    f"with a threshold of {oneway_threshold}. Generating the rows is post-processing
     ↪  and spent nothing."
)
```

Post-processing extends the guarantee to the synthetic rows without increasing privacy loss. The statement also records which keys and cuts were public; choosing cuts from private data would leak information. The 36-contribution privacy unit inflates every scale by a factor of 36. Identifier truncation and bounds shows that few people contribute often, so the script builds—but does not release—the same query with an identifier unit and a four-row contribution bound.

```
context_id = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=1, identifier="PIDENT"),  # neighbours add or
 ↪  remove one person
    privacy_loss=dp.loss_of(rho=RHO_QUERY, delta=DELTA_QUERY),
)
query_id = (
    context_id.query(rho=RHO_QUERY, delta=DELTA_QUERY)
    .truncate_per_group(TRUNCATE)  # at most 4 rows per person, so a person moves a
 ↪  count by at most 4
    .select(*COLUMNS)
    .contingency_table(keys=KEYS, cuts=CUTS, algorithm=dp.mbi.AIM(rounds=ROUNDS))
)
```

```
identifier unit, 4 rows per person: oneway scale=25.2982 threshold=142
identifier unit: scale ratio 9.00 = 36/4
```

The one-way scale falls nine-fold to 25.30 because each person can now change a count by at most 4 rows. The threshold falls from 1364 to 142, and all other release scales shrink by the same factor. This requires dropping every contribution after a person's fourth row.

17.9. The measured marginals against the real ones

The final check compares each measured multi-way marginal with its real counterpart, as in the one-way check. These should be the closest marginals because each release spent budget measuring them. The results are from the same run as above.

```
measured (AGE, HWUSUAL) (aim): tv=0.00200 largest cell error=2,992 rows  std=224.1
 ↪  bound=0.00141
measured (SEX, HWUSUAL) (aim): tv=0.00200 largest cell error=5,730 rows  std=242.4
 ↪  bound=0.00114
measured (AGE, HWUSUAL, ILOSTAT) (aim): tv=0.00219 largest cell error=1,382 rows
 ↪  std=379.5 bound=0.01195
measured (SEX, AGE, HWUSUAL) (aim): tv=0.00200 largest cell error=1,556 rows  std=169.7
 ↪  bound=0.00321
measured (SEX, AGE, ILOSTAT) (aim): tv=0.00093 largest cell error=699 rows  std=379.5
 ↪  bound=0.00896
measured (SEX, HWUSUAL, ILOSTAT) (aim): tv=0.00200 largest cell error=1,940 rows
 ↪  std=379.5 bound=0.00896
measured (AGE, ILOSTAT) (mst): tv=0.00116 largest cell error=2,366 rows  std=207.8
 ↪  bound=0.00164
measured (HWUSUAL, ILOSTAT) (mst): tv=0.00258 largest cell error=3,700 rows  std=207.8
 ↪  bound=0.00164
measured (SEX, ILOSTAT) (mst): tv=0.00041 largest cell error=1,085 rows  std=207.8
 ↪  bound=0.00123
measured (AGE, ILOSTAT) (sequential): tv=0.00027 largest cell error=387 rows  std=221.1
 ↪  bound=0.00174
measured (HWUSUAL, ILOSTAT) (sequential): tv=0.00068 largest cell error=1,035 rows
 ↪  std=351.3 bound=0.00276
```

```
measured (SEX, HWUSUAL) (sequential): tv=0.00017 largest cell error=338 rows  std=342.9
 ↪  bound=0.00162
measured (SEX, ILOSTAT) (sequential): tv=0.00019 largest cell error=335 rows  std=218.1
 ↪  bound=0.00129
measured (SEX, AGE, HWUSUAL) (sequential): tv=0.00094 largest cell error=712 rows
 ↪  std=240.0 bound=0.00453
measured (SEX, AGE, ILOSTAT) (sequential): tv=0.00099 largest cell error=751 rows
 ↪  std=536.7 bound=0.01267
measured (SEX, HWUSUAL, ILOSTAT) (sequential): tv=0.00073 largest cell error=567 rows
 ↪  std=536.7 bound=0.01267
```

In the two releases without null filling, every marginal containing `HWUSUAL` exceeds its bound. Pairs without that column also have larger errors than in the third release because the incorrect total affects every marginal. After null filling, each measured marginal is at least four times below its bound. The bound assumes every cell is three standard deviations from truth at once, an unlikely event; a three-way distance near one thousandth therefore indicates a good fit.

This check does not assess unmeasured queries or models trained on synthetic rows. Evaluating and tuning synthetic data tests both and examines the effects of budget, cuts and rounds.

17.10. Summary

Table 17.1.: The calls and concepts of synthetic data with AIM and MST, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.mbi.AIM(rounds=10)`	One-way marginals, then rounds that measure the marginal the model fits worst	Fewer rounds for a quick release; read `table.marginals` to see what it chose
`queries`, `measure_split`, `max_size`	The candidates or a weighted workload; the share of a round spent measuring; the memory cap	The defaults serve a handful of columns
`dp.mbi.MST()`	One-way marginals, then pairs that form a spanning tree, measured at once	Faster than AIM when pairs are what matter; it never measures a triple
`dp.mbi.Sequential(algorithms=[Fixed, AIM])`	Run algorithms in turn on one table, splitting the budget evenly or by `weights`	Put the marginals you must have in `Fixed`; AIM spends the rest
`query.oneway_scale`, `query.oneway_threshold`	The one-way step's noise scale and key threshold, before anything is spent	Under the identifier unit, 4 rows per person cost a ninth of 36

Call or concept	What it does	Rule of thumb
`table.synthesize(rows, method)`	Rows drawn from the fitted model; `round` matches its marginals, `sample` draws independently	Post-processing, so free; the row count defaults to the model's noisy total
Cut columns in the synthetic frame	Values drawn uniformly inside the bin, within one unit of the edge for the open bins	Right as bins, wrong as numbers; give finer cuts if the numbers matter
Nulls in a cut column	Dropped by the one-way step and counted in the last bin by the multi-way step in 0.16.0	Fill them with a public code in `with_columns` before the table
Total variation distance against `std`	How far a synthetic marginal is from the real one, against three standard deviations per cell	A measured marginal over the bound points to the data, not the noise

17.11. References

- OpenDP documentation: "Synthetic Data" in the statistical modelling guide, "Contingency Tables" in the tabular data guide, and the API reference for `opendp.extras.mbi` (`AIM`, `MST`, `Sequential`, `Fixed`, `ContingencyTable`).
- McKenna, Mullins, Sheldon and Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (2022): the selection score, the budget schedule and the experiments behind the defaults.
- McKenna, Miklau and Sheldon, "Winning the NIST Contest: A scalable and general approach to differentially private synthetic data" (2021): the MST mechanism and the Private-PGM engine that fits the model.
- McKenna, Sheldon and Miklau, "Graphical-model based estimation and inference for differential privacy" (2019): Private-PGM itself, the inference step that both algorithms share.
- The full edition's chapters on selection in the full edition, query release in the full edition and concentrated DP in the full edition derive the results this chapter states.

18. Evaluating and tuning synthetic data

Synthetic data with AIM and MST produced a synthetic table and privacy statement. The statement describes privacy, not data quality. This chapter evaluates quality in three ways: compare measured marginals with the real data, test an unmeasured table, and train a classifier on synthetic rows and test it on real rows. It repeats these checks at three budgets and with MST, then lists what to publish with the data.

The documentation ends after generating rows. This chapter adds evaluation methods from the AIM paper and a benchmark of synthetic-data generators. The Labour Force Survey (LFS) sample is public, so it can be compared with its source. For private data, use a public or historical stand-in, never the data being protected.

The script opendp_evaluating_synthetic_data.py runs on OpenDP 0.16.0 and `mbi` 1.1.0, reusing functions from the previous chapter's script. Budgets, scales, thresholds, conversions to ε and the classifier trained on real data are deterministic. OpenDP's noise cannot be seeded, so releases, fitted models and distances vary; the page labels them as one run. Four releases took 55 seconds.

```
uv run python code/part05-synthetic-data/opendp_evaluating_synthetic_data.py
```

18.1. What good means

The benchmark by Tao and coauthors evaluates synthetic data with three questions. Do measured marginals match the real ones? Do unmeasured queries, such as a count over all four columns or a share in a small group, remain accurate? Does a model trained on synthetic rows work on real rows? Each test can reveal failures missed by the others.

Only measured marginals have release-based error bars. Their known Gaussian scales let the table's `std` method estimate how far fitted counts may be from truth (Contingency tables). Unmeasured tables have no such bar, and neither do downstream models. Evaluate the second and third questions against real data. For private releases, use public or historical data of the same kind (the full edition's chapter on public data).

The evaluation uses the previous chapter's AIM release with nulls filled. It uses the same 36-contribution privacy unit (The data and the context), four columns, keys and cuts, and ten rounds. The budget is a privacy parameter ρ of 0.1 under zero-concentrated differential privacy (zCDP), with Gaussian noise; a slack δ of 10^{-7} pays for selecting `ILOSTAT` keys. Two functions build the context and query so later sections can vary the budget.

```
def make_context(frame: pl.LazyFrame, rho: float) -> dp.Context:
    """A context for one release at budget rho, with the previous chapter's unit and
     ↪  slack."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or
 ↪  remove one person's 36 rows
        privacy_loss=dp.loss_of(rho=rho, delta=DELTA),  # zCDP with Gaussian noise;
 ↪  delta pays for ILOSTAT's keys
    )


def table_query(context: dp.Context, rho: float, algorithm: Algorithm) ->
 ↪  ContingencyTableQuery:
    """The previous chapter's query with its nulls filled, at the budget rho, with the
     ↪  algorithm plugged in."""
    return (
        context.query(rho=rho, delta=DELTA)
        .with_columns(synth.fill_nulls())  # every null lands in a public key before
 ↪  anything is counted
        .select(*COLUMNS)
        .contingency_table(keys=synth.KEYS, cuts=synth.CUTS, algorithm=algorithm)
    )
```

The first line releases the table. The next applies the same preparation to real rows: fill nulls, bin numeric values and map codes outside the keys to null. Thus real and synthetic marginals have identical cells.

```
aim_query = table_query(make_context(frame, RHO), RHO, dp.mbi.AIM(rounds=ROUNDS))
table_aim, seconds_aim = synth.timed_release(aim_query)
real = synth.binned(frame, table_aim.keys, fill=True).collect()  # the real rows in the
 ↪  table's keys and bins
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.1, 1e-07))
aim rho=0.1: oneway scale=227.6840 threshold=1364  scale of a round's marginal=379.4733
 ↪  epsilon at delta=1e-07: 2.3484
rows: 3,811,932  keys ILOSTAT: [1, 2, 3, 9, None]
aim rho=0.1 release (one run): 13.7 s
aim rho=0.1 measured: ('AGE', 'HWUSUAL', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'AGE', 'HWUSUAL') std=379.47
aim rho=0.1 measured: ('SEX', 'AGE', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'ILOSTAT') std=189.74
```

The query and scales match the previous chapter: 227.68 for one-way marginals and 379.47 for each round's marginal. At a further δ of 10^{-7}, the budget converts to $\varepsilon = 2.35$. In this run, AIM measured all four three-way marginals and then a pair at half the scale.

18.2. The workload error

The first test measures each released marginal's distance from the real one. The AIM paper calls the workload-average distance the **workload error**, which AIM seeks to minimize. This is total variation distance: half the sum of absolute differences between cell shares. Equivalently, it is the largest share difference assigned to any set of cells. The worst distance below, 0.00102, bounds the share error for every query over those columns.

The function computes this distance for a list of cliques, the column sets covered by the marginals. It reads model counts with `project_melted` rather than drawing rows, which would add sampling noise (Section 18.6). It also reports the previous chapter's bound: the distance if every marginal cell were three standard deviations from truth.

```
def marginal_errors(
    table: ContingencyTable, real: pl.DataFrame, cliques: Sequence[tuple[str, ...]]
) -> list[MarginalError]:
    """Each clique's distance between the fitted model's marginal and the real one,
     ↪  with the table's own standard
    deviation for it and the bound that three standard deviations in every cell would
 ↪  give."""
    errors: list[MarginalError] = []
    for clique in cliques:
        fitted = table.project_melted(clique)  # the model's counts: no rows are drawn
        tv, worst = distance(synth.marginal(real, clique), fitted, clique)
        std = table.std(clique)
        bound = synth.tv_bound(synth.cells(table, clique), std, real.height)
        errors.append(MarginalError(clique, tv, worst, std, bound))
    return errors
```

Table 18.1.: The workload error of the AIM release at $\rho = 0.1$ in one run: the total variation distance between each one-way and each measured marginal of the model and the real one, the largest cell error, the table's own standard deviation, and the bound from it.

Marginal	Distance	Largest cell error (rows)	`std`	Bound
`SEX`	0.00004	158	196.2	0.00023
`AGE`	0.00005	146	219.9	0.00035
`HWUSUAL`	0.00006	246	219.9	0.00035
`ILOSTAT`	0.00013	307	201.0	0.00040
(`SEX`, `ILOSTAT`)	0.00022	425	178.9	0.00106
(`AGE`, `HWUSUAL`, `ILOSTAT`)	0.00102	509	379.5	0.01195
(`SEX`, `AGE`, `HWUSUAL`)	0.00081	503	379.5	0.00717
(`SEX`, `AGE`, `ILOSTAT`)	0.00072	568	379.5	0.00896
(`SEX`, `HWUSUAL`, `ILOSTAT`)	0.00065	470	379.5	0.00896

Every distance is at least three times below its bound, and no cell differs by more than a few hundred of 3.8 million rows. One-way standard deviations are smaller than the 227.68 one-way scale because the model combines each one-way measurement with all measured marginals

containing that column. Thus `ILOSTAT`, which appears in four marginals, has standard deviation 201.0.

Use this table to check the pipeline as well as the noise. A distance above its bound signals a systematic problem, such as the null handling in the previous chapter. Private data cannot supply the true marginals needed for these distances. The release does supply the `std` column, which provides the publishable error bars.

18.3. A query nobody measured

The second test concerns an unmeasured table. AIM considers marginals of at most three columns, so it never measures the 240-cell table over `SEX`, `AGE`, `HWUSUAL` and `ILOSTAT`. The model can still answer the query using a distribution that matches measured marginals and assumes no further relationships (the full edition's chapter on query release). But the release provides no error bar for this answer.

```
try:
    four_std = f"{table_aim.std(COLUMNS):.1f}"
except ValueError as err:
    four_std = str(err)  # no measured marginal covers all four columns, so the table
 ↪  has no error bar for it
```

```
four-way cells: 240  measured by AIM: False
std of the four-way table: attrs (('SEX', 'AGE', 'HWUSUAL', 'ILOSTAT')) are not covered
 ↪  by the query set
four-way (aim rho=0.1, one run): tv=0.00141 largest cell error=450 rows  best measured
 ↪  (SEX, ILOSTAT) tv=0.00022  worst measured tv=0.00102
cells of the share with at least 1,000 real rows: 26
share working by SEX, AGE, HWUSUAL (aim rho=0.1, one run): largest error=2.05 points at
 ↪  {'SEX': 1, 'AGE': '(-inf, 20.0]', 'HWUSUAL': '(1.0, 20.0]'}  real=100.00%
 ↪  model=97.95%
```

Here the four-way table is farther from truth than any measured marginal. Its error reflects noise in the measured triples and four-column dependence that no triple captures. The four measured triples overlap heavily, leaving little for the model to infer; therefore, the four-way distance stays near the worst measured distance. A release with fewer marginals relies more on model assumptions, as the MST results in Section 18.5 show.

Conditional shares are harder to estimate because they divide one count by another. From the four-way table, the script computes the employment share in 26 combinations of `SEX`, `AGE` band and `HWUSUAL` band with at least a thousand real rows. The largest error here is two percentage points for men under 20 who usually work 1 to 20 hours. All people in this cell work, but the model estimates a share of 97.95%. Noise is added to counts, not individuals, so small groups have larger relative errors. If analysts need small-group estimates, measure those groups, for example with `Fixed` inside `Sequential`.

18.4. Train on synthetic, test on real

The third test asks whether a model trained on synthetic rows performs on real rows, the benchmark's "train on synthetic, test on real" task. The classifier predicts whether `ILOSTAT` equals 1 from `SEX`, `AGE` band and `HWUSUAL` band. It is a scikit-learn logistic regression with eleven indicator columns, one for each feature key.

The script draws 200,000 synthetic rows independently with `method="sample"`. It also shuffles 400,000 real rows with a fixed seed. The first 200,000 train a real-data baseline; the remaining rows test both models.

```
rows = synth.synthesize(table, seed, rows=ROWS, method="sample")  # independent draws
 ↪  from the model
rows = synth.binned(rows.lazy(), table.keys, fill=False).collect()  # the numbers back
 ↪  in their bins
model = classifier(rows, table.keys, seed)  # train on synthetic
```

```
def classifier(train: pl.DataFrame, keys: dict[str, Any], seed: int) ->
 ↪  LogisticRegression:
    """A logistic regression that predicts ILOSTAT == 1 from SEX, the AGE band and the
     ↪  HWUSUAL band, fitted on
    `train`, which is either synthetic rows or real rows."""
    return LogisticRegression(max_iter=1000, random_state=seed).fit(design(train,
     ↪  keys), labels(train))


def accuracy(model: LogisticRegression, test: pl.DataFrame, keys: dict[str, Any]) ->
 ↪  float:
    """The share of the held-out real rows the model labels correctly."""
    return float((model.predict(design(test, keys)) == labels(test)).mean())
```

```
held-out real rows: 200,000  real training rows: 200,000  synthetic training rows:
 ↪  200,000
features: 11 indicators
share working in the held-out rows: 0.3976
train on real, test on real: accuracy=0.9313
standard error of an accuracy on 200,000 rows: 0.0006
train on synthetic (aim rho=0.1, one run), test on real: accuracy=0.9313  gap=+0.0000
 ↪  mean probability difference=0.0015
```

The real-data model classifies 93.13% of held-out rows correctly, while 39.76% of those rows are working. In this run, the synthetic-data model has the same accuracy, so the gap is zero and well within the 0.0006 standard error. The test allows a one-percentage-point gap, which could result from errors in a few small cells.

Accuracy is coarse: models make identical predictions whenever each cell's probability remains on the same side of one half. The script therefore also reports the models' mean predicted-probability difference, 0.0015 in this run. For a single model, direct private fitting (Statistical modelling) is often preferable to training on synthetic data.

18.5. What the budget and the algorithm buy

Tuning compares evaluation results across settings. The script runs AIM with one-fifth the budget, $\rho = 0.02$, and five times the budget, $\rho = 0.5$, then runs MST at the original budget. Each release has its own context because only one would be published. If all four used the same private data, they would compose to $\rho = 0.72$. Tuning on private data therefore spends budget; tune on a public stand-in and release once.

```
mst_query = table_query(make_context(frame, RHO), RHO, dp.mbi.MST())
table_mst, seconds_mst = synth.timed_release(mst_query)
mst_eval = evaluate("mst rho=0.1", table_mst, seconds_mst, real, test_real, baseline,
 ↪  seed)
```

```
aim rho=0.02: oneway scale=509.1169 threshold=3004  scale of a round's
 ↪  marginal=848.5281  epsilon at delta=1e-07: 0.9951
aim rho=0.5: oneway scale=101.8234 threshold=630  scale of a round's marginal=169.7056
 ↪  epsilon at delta=1e-07: 5.6709
mst rho=0.1: oneway scale=227.6840 threshold=1364  scale of a pair=207.8461  epsilon at
 ↪  delta=1e-07: 2.3484
mst rho=0.1 measured: ('HWUSUAL', 'ILOSTAT') std=207.85
mst rho=0.1 measured: ('AGE', 'ILOSTAT') std=207.85
mst rho=0.1 measured: ('SEX', 'ILOSTAT') std=207.85
share working by SEX, AGE, HWUSUAL (mst rho=0.1, one run): largest error=24.03 points
 ↪  at {'SEX': 2, 'AGE': '(-inf, 20.0]', 'HWUSUAL': '(-inf, 1.0]'}  real=100.00%
 ↪  model=75.97%
budget the four contexts add up to if all were released from the same data: rho=0.72
```

The previous chapter's scale formula uses sensitivity 36 divided by the square root of twice the budget. A 25-fold budget increase therefore divides each scale by five; one-way scales fall from 509.12 to 101.82. The smallest budget converts to ε of 1.00 and the largest to 5.67 (the full edition's chapter on choosing epsilon discusses defensible choices).

Table 18.2.: The same evaluation at three budgets with AIM and at one with MST, in one run: ε at $\delta = 10^{-7}$, the seconds the release took, the number of marginals measured, the total variation distances of the one-way marginals (mean), of the worst measured marginal and of the four-way table, the largest error of the share working, and the mean difference in predicted probability from the model trained on real rows.

Release	ε	Seconds	Measured	One-way mean	Worst measured	Four-way	Share error (points)	Probability difference
AIM, $\rho = 0.02$	1.00	6.4	4	0.00016	0.00157	0.00252	15.64	0.0016
AIM, $\rho = 0.1$	2.35	13.7	5	0.00007	0.00102	0.00141	2.05	0.0015
AIM, $\rho = 0.5$	5.67	7.1	6	0.00005	0.00052	0.00077	1.65	0.0014

Release	ε	Seconds	Measured	One-way mean	Worst measured	Four-way	Share error (points)	Probability difference
MST, $\rho = 0.1$	2.35	3.3	3	0.00021	0.00114	0.05610	24.03	0.0280

Larger budgets improve accuracy, but not uniformly. The worst measured marginal and four-way table improve across all three budgets, though less than the scales do. One-way marginals, already accurate at $\rho = 0.1$, improve least. Share error falls most at the low end, from 15.64 to 2.05 points, because noise affects small groups first. With more budget, AIM also measures more marginals: four, five and six in this run. All four classifiers have accuracy 0.9313, so budget changes are not visible in this task.

The algorithm also matters. Here MST measures three pairs, each containing `ILOSTAT`. It therefore treats `SEX`, `AGE` and `HWUSUAL` as independent given employment status. This is false: among working people, hours depend on sex and age. As a result, MST's four-way table is tens of times farther from truth than AIM's at the same budget.

MST's share error reaches 24 points for women under 20 who report at most one usual hour. Predicted probabilities differ by 0.028 on average, although accuracy is unchanged. Choose MST when pairwise queries suffice and release speed matters: it took 3.3 seconds, compared with 13.7 for AIM in this run.

The other settings are bin cuts and number of rounds. Finer cuts increase the number of cells in every marginal without changing noise scale, raising each marginal's error bound. More rounds divide the budget into smaller shares, allowing AIM to measure more marginals at lower precision.

18.6. How many rows, and how to draw them

The marginal evaluation reads the model rather than synthetic rows. Drawing a fixed number of independent rows adds sampling error to model error. The script estimates this error for each one-way marginal using the table's shares, for both 200,000 rows and the real dataset's size.

```
sampling noise on one-way SEX: expected tv at 200,000 rows=0.00090  at 3,811,932
 ↪  rows=0.00021
sampling noise on one-way AGE: expected tv at 200,000 rows=0.00152  at 3,811,932
 ↪  rows=0.00035
sampling noise on one-way HWUSUAL: expected tv at 200,000 rows=0.00100  at 3,811,932
 ↪  rows=0.00023
sampling noise on one-way ILOSTAT: expected tv at 200,000 rows=0.00140  at 3,811,932
 ↪  rows=0.00032
aim rho=0.1 one-way tv summed (one run): model=0.00029  round=0.00028  sample=0.00745
aim rho=0.1 four-way tv (one run): model=0.00141  round=0.00139  sample=0.00483
mst rho=0.1 four-way tv (one run): model=0.05610  round=0.18410  sample=0.05791
mst rho=0.1 pair ('AGE', 'HWUSUAL') tv (one run): model=0.01504  round=0.12370
 ↪  sample=0.01261
```

With 200,000 sampled rows, `SEX` sampling error is many times larger than the model error in Table 18.1, so the sample is less accurate than the release. Sampling error still exceeds model error at the real dataset's size because privacy noise at this budget is small relative to 3.8 million rows.

The default `method="round"` avoids sampling error; in this run, AIM's rounded marginals match the model. In `mbi` 1.1.0, however, each column is drawn by a row's rank within its parent-column group. Columns with the same parents are therefore sorted together and become dependent. In MST's star, `AGE` and `HWUSUAL` share parents; rounding changes their marginal distance from 0.01504 to 0.12370, while sampling keeps it near the model. Use sampling when joint row distributions matter. If marginal accuracy matters, draw as many rows as the model's total. Check pairwise marginals before trusting rounded rows.

18.7. What to publish

A synthetic dataset needs a guarantee statement and enough information for users to judge what it supports. The practitioner edition's chapter on release and governance gives the same guidance for trained models. The script supplies the numbers and identifies public information.

```
print(
    f"This synthetic dataset of {ROWS:,} rows was generated by AIM (OpenDP
     ↪ {dp.__version__}, mbi "
    f"{importlib.metadata.version('mbi')}, rounds={ROUNDS}, queries up to 3 columns)
     ↪ from one contingency table "
    f"over {', '.join(COLUMNS)} released under (rho = {RHO:g}, delta =
     ↪ {DELTA:g})-zero-concentrated differential "
    f"privacy, which is (epsilon = {epsilon:.2f}, delta = {2 * DELTA:g})-differential
     ↪ privacy, for adding or "
    f"removing all of one person's survey responses, at most {CONTRIBUTIONS} of them,
     ↪ under the central model. "
    f"The keys of SEX and the cuts of AGE and HWUSUAL were fixed in advance; nulls were
     ↪ recoded to "
    f"{synth.FILL}; the keys of ILOSTAT were released with a threshold of
     ↪ {aim_query.oneway_threshold}. "
    f"Drawing the rows by independent sampling is post-processing and spent nothing."
)
```

```
This synthetic dataset of 200,000 rows was generated by AIM (OpenDP 0.16.0, mbi 1.1.0,
 ↪ rounds=10, queries up to 3 columns) from one contingency table over SEX, AGE,
 ↪ HWUSUAL, ILOSTAT released under (rho = 0.1, delta = 1e-07)-zero-concentrated
 ↪ differential privacy, which is (epsilon = 2.35, delta = 2e-07)-differential
 ↪ privacy, for adding or removing all of one person's survey responses, at most 36 of
 ↪ them, under the central model. The keys of SEX and the cuts of AGE and HWUSUAL were
 ↪ fixed in advance; nulls were recoded to {'AGE': 0.0, 'HWUSUAL': 99.0}; the keys of
 ↪ ILOSTAT were released with a threshold of 1364. Drawing the rows by independent
 ↪ sampling is post-processing and spent nothing.
```

The statement covers the generated rows without an extra privacy cost because drawing from a released model is post-processing (the full edition's chapter on properties). It covers this table only; other releases from the same people would compose with it.

Table 18.3.: What to publish with a synthetic dataset, and where this chapter found each item.

Item	What to publish	In this chapter
The guarantee	ρ and δ, their conversion to ε, the privacy unit and the trust model, printed from the variables	$\rho = 0.1$, $\varepsilon = 2.35$, 36 contributions, central model
The columns	Each column with its keys or cuts, the null recoding, and which keys were released	`SEX` keys, `AGE` and `HWUSUAL` cuts, `ILOSTAT` keys at threshold 1364
The algorithm	Its name and parameters, the library versions, and how the rows were drawn	AIM with 10 rounds, OpenDP 0.16.0, `mbi` 1.1.0, 200,000 sampled rows
The error bars	The measured marginals and their `std`, which come from the release and cost nothing	Table 18.1, the `std` column
The utility	The marginal errors and the downstream check, measured on public or historical data	Never on the private data itself

The utility row is easy to mishandle. A distance computed from private data depends on that data; publishing it creates a release not covered by this statement. As Limitations and troubleshooting explains, the guarantee covers measured outputs, not separate computations. The `std` error bars are safe to publish. The distances here are safe only because the dataset is public.

This final chapter completes the workflow: release statistics, tables, models and synthetic rows through a context; evaluate quality on data you may inspect; and publish each release with a precise guarantee.

18.8. Summary

Table 18.4.: Evaluating and tuning synthetic data, what each step does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
Three questions	Measured marginals, unmeasured queries, a downstream model	Only the first has an error bar from the release
`table.project_melted(clique)`	The fitted model's counts for a marginal, without drawing rows	Evaluate the model first, then the rows
Total variation distance against `std`	Workload error of a marginal against three standard deviations in every cell	A distance over the bound points to the pipeline, not the noise

Call or concept	What it does	Rule of thumb
`table.std` on an uncovered table	Raises `ValueError`: not covered by the query set	No measurement, no error bar; check unmeasured queries on public data
Train on synthetic, test on real	A model fitted to synthetic rows, scored on held-out real rows	Accuracy is coarse; compare predicted probabilities too
One context per setting	Alternatives that you compare but release only one of	On the same private data their budgets add up; tune on a stand-in
The budget	Divides every scale by the square root of its ratio	Small groups and unmeasured tables gain most; one-way marginals least
AIM against MST	Triples chosen adaptively, or a tree of pairs	MST is fast but loses what no pair shows
`synthesize(rows, method="sample")`	Independent rows, with sampling error of their own	Draw as many rows as the model's total when the marginals must be close
`method="round"` in `mbi` 1.1.0	Matches the model's marginals but ties columns drawn from the same parents	Check the pairs before trusting rounded rows
What to publish	Guarantee, columns, algorithm, error bars, utility on public data	Never publish a distance computed on the private data

18.9. References

- OpenDP documentation: "Synthetic Data" in the statistical modelling guide, "Contingency Tables" in the tabular data guide, and the API reference for `opendp.extras.mbi.ContingencyTable` (`std`, `project_melted`, `synthesize`). The documentation gives no evaluation; this chapter's is its own.
- McKenna, Mullins, Sheldon and Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (2022): the workload error and the selection that minimizes it.
- Tao, McKenna, Hay, Machanavajjhala and Miklau, "Benchmarking Differentially Private Synthetic Data Generation Algorithms" (2021): marginal errors and downstream tasks as the measures of quality.
- The practitioner edition's chapter on release and governance in the practitioner edition: the statement of a guarantee and the evidence that goes with it.
- The full edition's chapters on post-processing and composition in the full edition, query release in the full edition, public data in the full edition and choosing epsilon in the full edition derive the results this chapter states.

A. Glossary

Acronyms used in the book and the pages that use them.

API Application programming interface. Used in Getting started, A first release, The data and the context, Grouping, Assessing utility and choosing the noise, The framework, A framework to understand differential privacy, Transformations, Measurements, Combinators, Utilities and the Context API inside, Extending the library, Plugins, Statistical modelling, Limitations and troubleshooting, Synthetic data with AIM and MST, Evaluating and tuning synthetic data.

CPU Central processing unit. Used in Synthetic data with AIM and MST.

CSV Comma-separated values. Used in The data and the context, Limitations and troubleshooting.

DP Differential privacy. Used in Preface, Essential statistics, Identifier truncation and bounds, Assessing utility and choosing the noise, A framework to understand differential privacy, Measurements, Combinators, Statistical modelling, Contingency tables, Synthetic data with AIM and MST.

JSON JavaScript Object Notation. Used in Utilities and the Context API inside.

LFS Labour Force Survey. Used in Preface, Getting started, The data and the context, Essential statistics, Grouping, Identifier truncation and bounds, Preparing microdata, Assessing utility and choosing the noise, A framework to understand differential privacy, Transformations, Measurements, Combinators, Utilities and the Context API inside, Plugins, Statistical modelling, Limitations and troubleshooting, Contingency tables, Synthetic data with AIM and MST, Evaluating and tuning synthetic data.

PCA Principal component analysis. Used in Statistical modelling.

RDP Rényi differential privacy. Used in Combinators.

SQL Structured Query Language. Used in Identifier truncation and bounds.

B. Index

adaptive composition, 112, 113, 122
approximate differential privacy, 77, 123

basic composition, 113

clipping, 26, 64, 65
composition, 1, 9, 16, 27, 35, 67, 77, 88, 112, 113, 122, 123, 138, 189

differential privacy, 3, 9, 16, 18, 27, 35, 45, 65, 67, 77, 88, 100, 112, 113, 122, 123, 125, 138, 147, 157, 168, 179, 189
discrete Gaussian distribution, 65, 100, 113
discrete Laplace distribution, 99, 100

empirical risk minimization, 147
exponential mechanism, 27, 138

fully adaptive composition, 112

Gaussian mechanism, 9, 27, 65, 77, 88, 101, 123, 138, 168
group privacy, 16

honest-but-curious, 111, 112, 138, 146, 156

Laplace mechanism, 9, 16, 27, 45, 77, 100

one-way marginal, 178, 185, 189

permute-and-flip, 88, 100
post-processing, 9, 16, 27, 35, 77, 88, 111, 113, 159, 167, 168, 179, 189
privacy filter, 112
privacy loss, 67, 113, 146
privacy odometer, 112
privacy profile, 77, 112, 113
pure differential privacy, 9, 16, 64, 77, 99, 112, 113, 122

randomized response, 100, 101
report-noisy-max, 100
Rényi differential privacy, 113

sensitivity, 9, 16, 26, 27, 35, 45, 56, 63, 64, 77, 88, 100, 101, 138, 146, 147, 157, 168
smooth sensitivity, 56
sparse vector technique, 100

total variation distance, 179, 182, 185, 188

variance, 45, 65, 88

zero-concentrated differential privacy, 18, 63–65, 77, 99, 112, 113, 122, 123, 167, 168